Методическое пособие
(учебно-методический комплекс)
по курсу

Разработка программного обеспечения для GNU/Linux

Москва
2025

УДК
ББК

Разработка программного обеспечения для GNU/Linux / Курячий Г.В., Балашов В.В., Батенков К.А. Методическое пособие (учебно-методический комплекс к спецкурсу).

Курс «Разработка программного обеспечения для GNU/Linux» предназначен для очного и дистанционного обучения студентов, обучающихся по специальности 0104000 — «Прикладная математика и информатика». В курсе рассматривается жизненный цикл создания программного продукта на примере практик, используемых при разработке приложений для операционных систем (ОС) семейства GNU/Linux.

Пособие по курсу «Разработка программного обеспечения для GNU/Linux» (далее — Пособие) включает в себя описание входных требований к слушателям, тематический план и программу курса, методики проведения занятий, планы занятий и практические задания к ним, оценочные средства для текущего контроля и итоговой аттестации студентов, требования к ресурсному обеспечению, инструкции по развертыванию виртуальной машины для выполнения заданий по курсу.

Курс основан на программной платформе отечественных операционных систем семейства «Альт», разработанного компанией «Базальт СПО». Знания и навыки, полученные в результате прохождения курса, могут быть применены и при работе с другими современными дистрибутивами операционной системы GNU/Linux.

Пособие разработано, апробировано и применяется при обучении студентов ВМК МГУ им. М.В. Ломоносова в рамках спецкурса кафедры автоматизации систем вычислительных комплексов (АСВК). Пособие подготовлено при финансовой поддержке ООО «Базальт СПО» (Договор пожертвования П37-2024 от 15.11.2024).

Материалы, составляющие учебно-методический комплекс, распространяются на условиях лицензии GNU FDL.

© Г.В. Курячий, В.В. Балашов, К.А. Батенков

Оглавление

Условные обозначения.....7
Введение.....8
1 Входные требования.....10
2 Тематический план курса.....11
3 Программа курса.....12
4 Методика чтения лекций.....13
5 Правила работы с домашними заданиями.....15
5.1 Выдача домашних заданий.....15
5.2 Оформление домашних заданий.....15
5.3 Оценивание домашних заданий.....16
6 Сборочное окружение и зависимости.....17
6.1 Стадии разработки приложения.....17
6.2 Цикл разработки.....17
6.3 Редактирование.....17
6.4 Сборочное окружение.....18
6.5 Зависимости.....18
6.6 Git.....19
6.7 Доступ к сборочной среде.....19
6.8 Домашнее задание.....20
7 Работа с терминалом и простой проект.....21
7.1 Терминал.....21
7.2 Управление выводом со стороны приложения.....21
7.3 База информации о терминалах Terminfo.....21
7.4 Ncurses.....22
7.5 Примитивный makefile.....22
7.6 Домашнее задание.....22
8 Многофайловая сборка.....24
8.1 Сборка из нескольких файлов.....24
8.2 Make.....25
8.3 GNU make.....25
8.4 Литература по GNU make.....26
8.5 Переменные.....26
8.6 Правила, применяемые по умолчанию.....27

8.7 Статические библиотеки....27
8.8 Динамические библиотеки....29
8.9 Другие системы сборки....29
8.10 Домашнее задание....30
9 Обработка текстов и сценарии....31
9.1 Языки склейки....31
9.2 Shell как язык склейки....31
9.3 Утилиты для работы с файловой системой и процессами....34
9.4 Утилиты для работы с текстом....34
9.5 Комбайны....34
9.6 Домашнее задание....34
10 Отладка....36
10.1 ELF....36
10.2 Общие правила отладки....36
10.3 GDB....36
10.4 Пример работы со стеком....38
10.5 Пакеты devel....39
10.6 Пакеты debuginfo....39
10.7 Debuginfod-сервер....40
10.8 Отладка ранее запущенного процесса....40
10.9 Удаленная отладка....40
10.10 Интерфейсы для gdb....41
10.11 Сценарии для gdb....41
10.12 Домашнее задание....42
11 Регулярные выражения....44
11.1 Шаблоны shell....44
11.2 Вводная информация о регулярных выражениях....44
11.3 Определение регулярного выражения....45
11.4 Диалекты (варианты синтаксиса) регулярных выражений....47
11.5 Поиск с заменой и обратные ссылки....48
11.6 Регулярные выражения в C....49
11.7 Нерегулярные выражения....51
11.8 Домашнее задание....51
12 Трассировка вызовов....53
12.1 Трассировка с помощью strace....53

12.2 Переменная LD_PRELOAD и подмена библиотек....54
12.3 Прочие инструменты трассировки....54
12.4 Домашнее задание....55
13 Адаптация к окружению при сборке....56
13.1 Общие понятия Autotools....56
13.2 Пример сборки проекта....58
13.3 Autoscan....58
13.4 Autoconf и configure....59
13.5 Autoheader....59
13.6 Config.h....59
13.7 Automake....61
13.8 Autoreconf....61
13.9 Тестирование....61
13.10 Сборка в отдельном дереве....61
13.11 Pkg-config....62
13.12 Cmake....62
13.13 Meson и Ninja....62
13.14 Домашнее задание....62
14 Интернационализация и локализация....65
14.1 Область применения....65
14.2 Gettext....66
14.3 Создание перевода....67
14.4 Обновление перевода....73
14.5 Различные формы множественного числа....77
14.6 Раздельное хранение исходных текстов и переводов....79
14.7 Домашнее задание....80
15 Работа со сторонними исходными текстами....81
15.1 Патч и его место в процессе разработки....81
15.2 Технология патчирования....81
15.3 Неточный контекст....86
15.4 Diffutils....89
15.5 Git и патчи....96
15.6 Домашнее задание....96
16 Библиотеки и тестирование....99
16.1 Сборка библиотек....99

16.2 Libtool.....100
16.3 Версионирование библиотек.....104
16.4 Тестирование.....105
16.5 Check.....106
16.6 Домашнее задание.....131
17 Информационное пространство и документирование.....132
17.1 Источники информации.....132
17.2 Doxygen.....133
17.3 Домашнее задание.....147
18 Установка и пакетирование.....149
18.1 Понятие установки.....149
18.2 Варианты установки.....149
18.3 Поддержка установки при сборке.....150
18.4 Введение в пакетирование.....151
18.5 Работа системы с пакетами.....151
18.6 Сборка пакета при помощи hasher.....152
18.7 Домашнее задание.....155
19 Материально-технические условия проведения курса.....157
19.1 Аппаратное обеспечение.....157
19.2 Программная среда для демонстраций и выполнения домашних заданий.....157

Условные обозначения

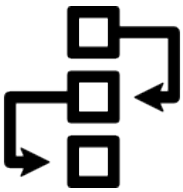
— требуется самостоятельное выполнение задания.

— исторические сведения.

— ограничения курса.

— краткое перечисление сведений.

— самостоятельно ранее приобретенные навыки.



— последовательность действий.

— на врезках: пропуск строк, аналогичных предыдущим и/или последующим.

Введение

Курс «Разработка программного обеспечения для GNU/Linux» предназначен для очного и дистанционного обучения студентов, обучающихся по специальности 0104000 — «Прикладная математика и информатика» на факультете Вычислительной математики и кибернетики Московского Государственного университета имени М. В. Ломоносова. В курсе рассматривается жизненный цикл создания программного продукта на примере практик, используемых при разработке приложений для операционных систем (ОС) семейства GNU/Linux.

Пособие по курсу «Разработка программного обеспечения для GNU/Linux» (далее — Пособие) включает в себя описание входных требований к слушателям, тематический план и программу курса, методики проведения занятий, планы занятий и практические задания к ним, оценочные средства для текущего контроля и итоговой аттестации студентов, требования к ресурсному обеспечению, инструкции по развертыванию виртуальной машины для выполнения заданий по курсу.

Курс основан на программной платформе отечественных операционных систем семейства «Альт», разработанного компанией «Базальт СПО». Знания и навыки, полученные в результате прохождения курса, могут быть применены и при работе с другими современными дистрибутивами операционной системы Linux.

Курс ориентирован на применение свободного программного обеспечения (СПО), распространяемого на условиях простой (неисключительной) лицензии, которая позволяет пользователю:

1. использовать программу для ЭВМ в любых не запрещённых законом целях;
2. получать доступ к исходным текстам (кодам) программы как в целях изучения и адаптации, так и в целях переработки программы для ЭВМ;
3. распространять программу (бесплатно или за плату, по своему усмотрению);
4. вносить изменения в программу для ЭВМ (перерабатывать) и распространять экземпляры изменённой (переработанной) программы с учётом возможных требований наследования лицензии;
5. в отдельных случаях (copyleft лицензия) распространять модифицированную компьютерную программу пользователем только на условиях, идентичных тем, на которых ему предоставлена исходная программа.

Примерами свободных лицензий являются:

1. *GNU general public license*[1]. Version 3, 29 June 2007 (Стандартная общественная лицензия GNU. Версия 3, от 29 июня 2007 г.) — лицензия copyleft;
2. *New Berkley Software Distribution license*[2] (Модифицированная программная лицензия университета Беркли).

Свободное программное обеспечение отлично подходит для целей обучения и для разработки собственных решений, потому что весь код доступен для изучения и мо-

1 Стандартная общественная лицензия GNU (GPL) [Электронный ресурс]. URL: https://www.gnu.org/licenses/gpl-3.0.html (дата обращения: 20.01.2025).

2 Wikipedia: BSD licenses [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/BSD_licenses (дата обращения: 20.01.2025).

дификации, а также для применения в некоммерческих и коммерческих целях. Однако всем, кто использует свободное программное обеспечение для построения своих программных продуктов, следует учитывать особенности лицензирования не только самих пакетов, но и входящих в их состав библиотек. Если вы используете библиотеку с лицензией GPL, это обязывает вас распространять свою программу под этой же лицензией, т.е. любой человек, который получит вашу программу легальным способом, может потребовать предъявить ему исходный код.

1 Входные требования

Перед началом обучения по курсу слушатель должен обладать следующими знаниями и навыками:

1. владение операционной системой Linux на уровне пользователя, в т.ч. работа в командной строке (bash, zsh);
2. умение работать в консольном редакторе (рекомендуется vim);
3. базовое знание языка Си;
4. владение системой управления версиями git на уровне, достаточном для ведения истории версий в публичном репозитории (github, gitlab и др.)

В состав материалов курса входит виртуальная машина с предустановленной операционной системой «Альт» и комплектом программного обеспечения, необходимым для выполнения практических заданий. Эта виртуальная машина должна быть доступна для скачивания и установки слушателями. Если слушатель планирует использовать иную операционную систему семейства Linux, он должен обладать навыками администрирования этой операционной системы и быть готовым устанавливать требуемые пакеты с учетом возможных различий их именования в используемой операционной системе и операционной системе «Альт».

2 Тематический план курса

№ п/п	Раздел курса	Лек-ции, часы	Самостоятель-ная работа, часы
1	Сборочное и окружение и зависимости	2	1
2	Работа с терминалом и простой проект	2	1
3	Многофайловая сборка	2	1
4	Обработка текстов и сценарии	2	1
5	Отладка	2	1
6	Регулярные выражения	2	1
7	Трассировка вызовов	2	1
8	Адаптация к окружению при сборке	2	1
9	Интернационализация и локализация	2	1
10	Работа со сторонними исходными текстами	2	1
11	Библиотеки и тестирование	2	1
12	Информационное пространство и документирование	2	1
13	Установка и пакетирование	3	1
	Итого	27	13

3 Программа курса

№ п/п	Раздел курса	Содержание разделов курса
1	Сборочное и окружение и зависимости	стадии разработки приложения, цикл разработки, редактирование, сборочное окружение, зависимости, git, доступ к сборочной среде
2	Работа с терминалом и простой проект	терминал, управление выводом со стороны приложения, база информации о терминалах terminfo, ncurses, примитивный makefile
3	Многофайловая сборка	сборка из нескольких файлов, make, GNU make, переменные, правила, применяемые по умолчанию, статические библиотеки, динамические библиотеки, другие системы сборки
4	Обработка текстов и сценарии	языки склейки, shell как язык склейки, утилиты для работы с файловой системой и процессами, утилиты для работы с текстом, комбайны
5	Отладка	ELF, общие правила отладки, GDB, пример работы со стеком, пакеты devel, пакеты debuginfo, debuginfo-сервер, отладка ранее запущенного процесса, удаленная отладка, интерфейсы для GDB, сценарии для GDB
6	Регулярные выражения	шаблоны shell, вводная информация о регулярных выражениях, определение регулярного выражения, диалекты (варианты синтаксиса) регулярных выражений, поиск с заменой и обратные ссылки, регулярные выражения в C, нерегулярные выражения
7	Трассировка вызовов	трассировка с помощью strace, переменная LD_PRELOAD и подмена библиотек, прочие инструменты трассировки
8	Адаптация к окружению при сборке	набор инструментов Autotools, сборка в отдельном дереве, Pkg-config, инструменты CMake, Meson и Ninja
9	Интернационализация и локализация	предмет интернационализации и локализации, инструмент gettext, создание и обновление перевода, поддержка форм множественного числа, раздельное хранение исходных текстов и переводов
10	Работа со сторонними исходными текстами	Патч и его место в процессе разработки, технология патчирования, применение патча в неточном контексте, набор инструментов diffutils, git и патчи
11	Библиотеки и тестирование	Сборка библиотек, версионирование библиотек, инструмент libtool, тестирование и его виды, инструмент check
12	Информационное пространство и документирование	Источники информации в проекте, инструмент doxygen, формирование различных видов документации при помощи doxygen
13	Установка и пакетирование	Понятие и варианты установки, поддержка установки при сборке, основные принципы пакетирования, сборка пакета при помощи hasher

4 Методика чтения лекций

Занятия данного курса целесообразно проводить в форме лекций, сопровождаемых демонстрацией. В основе такой лекции лежит развернутое комментирование лектором взаимодействия с операционной системой ALT Linux в форме ввода терминальных команд и получения отклика в виде выдачи в терминале или создания или изменения содержания файлов и каталогов. Экран компьютера, на котором производится демонстрация, показывается слушателям при помощи проектора. Также этот экран может транслироваться через видеосервисы Интернет при дистанционном чтении лекций.

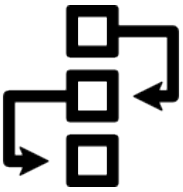
Подобная форма проведения лекций позволят слушателям наблюдать работу с программными средствами «вживую», а лектору — приводить проблемные ситуации и показывать возможные пути их разрешения.

Для курса организуется сайт, имеющий относительный адрес, содержащий его название, например /LecturesCMC/LinuxApplicationDevelopment2024. Для каждой лекции формируется страница с адресом, включающим ее порядковый номер и краткое название, например /LecturesCMC/LinuxApplicationDevelopment2024/01_TerminalProject. Данное краткое название является одновременно идентификатором занятия (указывается в начале раздела по теме).

Страница лекции используется лектором как краткий конспект лекции и источник фрагментов программного кода, которые лектор использует в ходе чтения лекции. Страница должна быть также доступна слушателям, которые, помимо конспекта и фрагментов кода, могут использовать находящиеся на странице ссылки на внешние материалы.

Наполнение страницы лекции может быть получено из настоящего материала, в котором скрыты участки с детальным описанием материала. Процедура скрытия описана далее в данном разделе.

Последовательность изложения материала лектором такова:



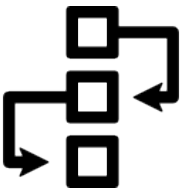
1. по каждой технологии (подтеме лекции) приводятся сведения, в целом описывающие проблематику;
2. после этого проводится живая демонстрация порядка взаимодействия с операционной системой и инструментальными средствами;
3. при необходимости уточнения определенных сложных моментов демонстрируется конспект (страница) лекции;
4. в конце рассмотрения технологии приводятся общие выводы и ограничения ее использования.

Участки лекции, отвечающие живой демонстрации, помечены зеленым фоном, на котором представлены команды взаимодействия и ответы терминала на них. Вводимые лектором команды отмечены приглашением вида [alt@localhost <текущий_каталог>] $, а после вводимых команд приведены примеры их вывода.

Синим фоном отмечены участки кода, содержащиеся в файлах, имена которых приведены в начальном комментарии.

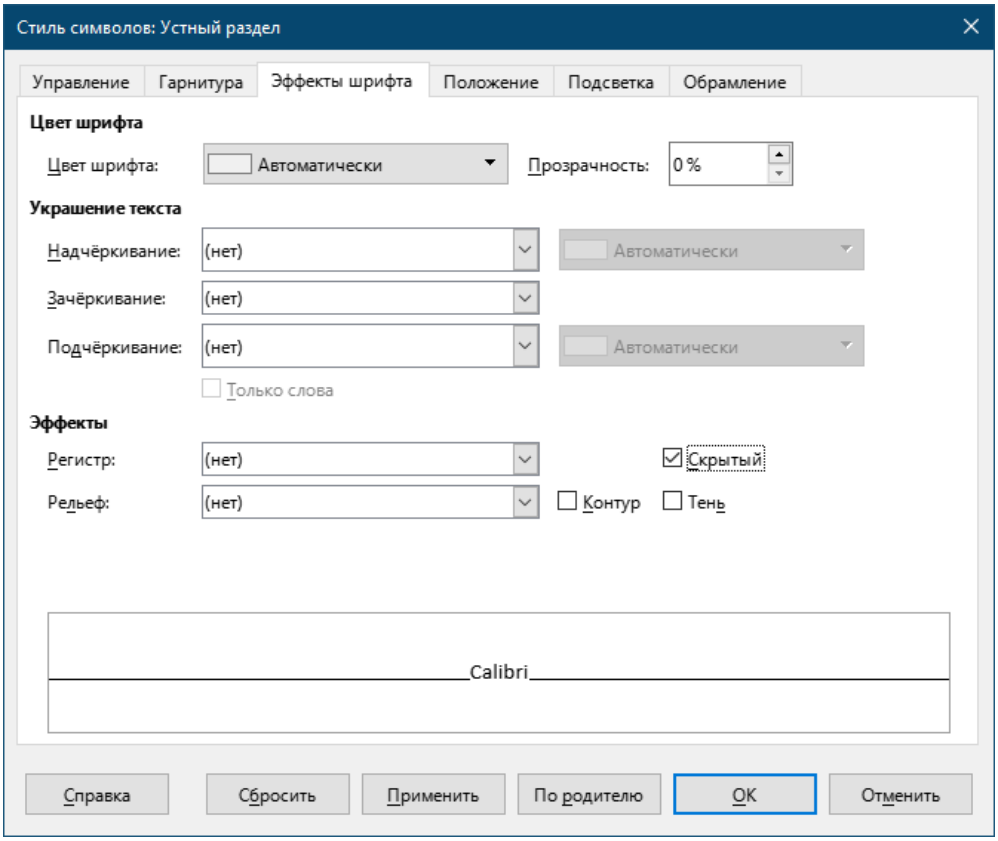
С помощью стиля символов выделены участки, подлежащие скрытию при формировании конспекта для слушателей. По сути, скрытая часть — это развернутые пояснения, предназначенные для использования лектором при подготовке к занятию.

Скрытие выполняется следующим образом:



1. открытие боковой панели «Стили» (Alt+2);
2. выбор раздела «Стили символов»;
3. изменение стиля «Устный раздел» (нажатие правой кнопки мыши);
4. в подразделе «Эффекты» установить маркер (галочку) «Скрытый»;
5. изменение стиля «Устный код» (нажатие правой кнопки мыши);
6. в подразделе «Эффекты» установить маркер «Скрытый».

Показать скрытые участки возможно сняв маркер «Скрытый» из подраздела «Эффекты» стилей «Устный раздел» и «Устный код».



5 Правила работы с домашними заданиями

5.1 Выдача домашних заданий

Домашнее задание выдается в конце каждой лекции, с необходимыми пояснениями от лектора. Описание домашнего задания содержится в конце страницы этой лекции и доступно слушателям на сайте курса.

5.2 Оформление домашних заданий

1. Завести публичный git-репозиторий на любом доступном сервисе:

- *GitHub*[3]*;*
- *GitLab*[4]*;*
- любой сайт (Gitweb/Gitea/Kalitea/SourceHut/…), поддерживающий интерфейс взаимодействия, пригодный для git clone;
- студентам ВМК можно воспользоваться *факультетским GitLab*[5] (вкладка LDAP).

2. Зарегистрировать этот репозиторий в качестве тикета (issue) в указанном лектором репозитории — таким способом выполняется запись на курс.

3. Создавать в этом репозитории подкаталоги, совпадающие с идентификатором лекции, так для /LecturesCMC/LinuxApplicationDevelopment2024/ 01_TerminalProject идентификатором является 01_TerminalProject, и размещать там решения:

- например, домашнее задание по теме с идентификатором 01_TerminalProject должно лежать в подкаталоге 01_TerminalProject;
- содержимое этого каталога должно позволять собрать и запустить решение способом, указанным в домашнем задании;
- в git-репозиторий нельзя размещать генераты — файлы, которые генерируются из других файлов в процессе сборки.

4. Соблюдать контрольные сроки, проверяемые по дате последнего коммита в каталоге репозитория. Если есть необходимость поработать над уже сделанным домашним заданием после срока сдачи (ради интереса, потому что нашли ошибку и т. п.), создать ещё один подкаталог (например, 01_TerminalProject_1), скопировать все файлы в него и там отрабатывать — это позволит избежать ложного вывода о нарушении контрольного срока.

5. Простое правило: чем решение позднее сдано, тем строже проверка. При этом в случае неуверенности в авторстве присланного решения будет проведено собеседование по представленной домашней работе.

---

3 GitHub home page [Электронный ресурс]. URL: https://github.com/ (дата обращения: 20.01.2025).

4 GitLab home page [Электронный ресурс]. URL: https://gitlab.com/ (дата обращения: 20.01.2025).

5 GitLab-сервер факультета ВМК МГУ [Электронный ресурс]. URL: https://git.cs.msu.ru/ (дата обращения: 20.01.2025).

5.3 Оценивание домашних заданий

Схема оценивания домашних заданий следующая:

1. Задание оценивается по пятибалльной шкале и умножается на коэффициент, зависящий от срока сдачи.
2. При сдаче задания в первые шесть дней после лекции, оценка умножается на 100.
3. При сдаче задания в следующую за этим неделю, оценка умножается на 50.
4. При сдаче задания в более поздний срок, оценка умножается на 25.

Решить необходимо все задачи, причем рациональнее делать это вовремя. Описанная выше схема оценивания выбрана для предотвращения накопления задолженностей к концу отчетного периода.

6 Сборочное окружение и зависимости

Идентификатор темы для сайта: 00_BuildEnv

Разработка приложения — не просто процесс написания кода на каком-то языке программирования с использованием IDE (integrated development environment — интегрированная среда разработки). В действительности разработка приложения — это довольно длинный процесс. Правила ведения проекта (в том числе группой разработчиков) обычно объясняют в рамках курса «Технологии программирования», который целесообразно прослушать в дополнение к настоящему курсу.

6.1 Стадии разработки приложения

Разработка:

1. Исходный код, документация, мультимедиа.
2. Сборка программного продукта.
3. Тестирование и отладка.

Развёртывание программного обеспечения (deployment management):

1. Локальное тестовое развертывание.
2. Репозиторий и доставка.
3. Установка и развёртывание.
4. Сопровождение.

6.2 Цикл разработки

Цикл разработки программного обеспечения состоит из трех шагов:

1. Редактирование.
2. Сборка и подготовка к запуску.
3. Тестирование и запуск.

6.3 Редактирование

Для редактирования программного кода необходим текстовый редактор.

Желательно, чтобы текстовый редактор для программирования поддерживал:

1. Синтаксис языка программирования.
2. Рецепты сборки.
3. Многофайловые проекты и связи между файлами.
4. Интеллектуальный ассистент разработчика (автодополнение кода и подсказки).
5. Отладчик.
6. Рецепты развертывания.
7. И другое…

6.4 Сборочное окружение

Предназначенный для процесса разработки дистрибутив Linux должен содержать довольно много компонентов:

1. Текстовый редактор.
2. Компилятор и компоновщик.
3. Дополнительные библиотеки или модули (для запуска приложения).
4. Сборочные варианты библиотек, в т.ч. их заголовочные файлы.
5. Отладочные версии библиотек (содержащие отладочную информацию).
6. Система контроля версий.
7. Средства обработки непрограммных компонентов (интернационализация i18n, мультимедиа и т. п.).
8. Средства оркестрации сборки сложного проекта (make, ninja, cmake, meson и т. п.).
9. Парсер библиотек и бинарных исполняемых файлов, прочие информационные утилиты.
10. Отладчик, трассировщик выполнения, профайлер и иные средства диагностики.
11. Инструменты пакетирования и развертывания (опционально).
12. И так далее…

6.5 Зависимости

Инструменты, от которых зависит разрабатываемая программа, условно делят на три части.

Инструментальные зависимости — программное обеспечение, необходимое для поддержки процесса разработки:

1. Текстовый редактор и дополнения к нему.
2. Система контроля версий (например, git).
3. Отладчик и профилировщик.
4. Руководства и иная документация.
5. Средства тестирования.

Инструментальные зависимости необходимы для удобства разработки, но не являются необходимыми для сборки.

Сборочные зависимости — программное обеспечение, необходимое для сборки работающего приложения из исходного текста:

1. Компилятор и компоновщик.
2. Средства автоматизации сборки (например, GNU make).
3. Devel-версии библиотек (в частности, файлы include).
4. Другие средства обработки исходников (генераторы документации и т. п.).

Без сборочных зависимостей никакой инструмент не соберет программный продукт. Построение сборочного окружения хорошо видно на примере GitHub actions, например — развертывание Ubuntu Linux с установкой некоторой последовательности пакетов.

Эксплуатационные зависимости — программное обеспечение, необходимое для запуска и использования установленного приложения:

1. Используемые разделяемые библиотеки или модули высокоуровневых языков программирования.

2. Запускаемые приложением другие приложения и сервисы, к которым приложение обращается.

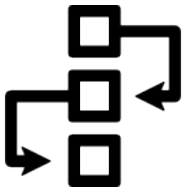
6.6 Git

Существует достаточно хорошо написанный на русском языке учебник по системе контроля версий *Git*[6].

Система контроля версий — программное обеспечение, которое поддерживает выполнение следующих задач при работе с кодом:

1. Версионирование.
2. Хранение.
3. Совместная разработка.

Цикл работы с Git:



1. Клонирование (git clone).
2. Синхронизация (git pull = git fetch + git merge).
3. Изменение.
4. Отметка файлов для коммита (git add).
5. Коммит (git commit).
6. Переход к синхронизации или изменению.
7. Публикация (git push).

Свойства Git:

1. Хранение всех версий исходников.
2. Хранение полного репозитория в локальной копии.
3. Отслеживание процесса разработки в виде ориентированного графа (в простейшем случае линейный): понятие «ветка», «слияние веток» и т. п.

6.7 Доступ к сборочной среде

Варианты сборочного окружения для выполнения заданий по данному курсу:

1. Собственный компьютер с установленной операционной системой Linux, при этом за ее настройку отвечает слушатель.
2. Собственный компьютер с *дистрибутивом Альт*[7] (*стартовые наборы для проектируемых дистрибутивов*[8] или *Rolling*[9]).
3. Виртуальная машина, например, VirtualBox или *WSL2*[10] (Windows Subsystem for Linux).

6 Pro Git book (перевод на русский) [Электронный ресурс]. URL: https://git-scm.com/book/ru/v2 (дата обращения: 20.01.2025).

7 Дистрибутивы ОС «Альт» [Электронный ресурс]. URL: https://getalt.org/ru/ (дата обращения: 20.01.2025).

8 Стартовые наборы ОС «Альт» [Электронный ресурс]. URL: https://www.altlinux.org/Starterkits/Download (дата обращения: 20.01.2025).

9 Регулярные сборки образов ОС «Альт» [Электронный ресурс]. URL: https://www.altlinux.org/Regular (дата обращения: 20.01.2025).

10 Установка Linux в Windows с помощью WSL [Электронный ресурс]. URL: https://learn.microsoft.com/ru-ru/windows/wsl/install (дата обращения: 20.01.2025).

4. Доступ к удаленному серверу, настроенному лектором (необходима заявка о создании учетной записи; понадобится владение ssh, консольным текстовым редактором, например, vim, nano, mc micro; все необходимые пакеты уже установлены).

6.8 Домашнее задание

1. Обеспечить себе доступ к Linux-системе, на которую можно устанавливать произвольное программное обеспечение из репозитория, или к предварительно настроенной системе, где необходимое для данного курса программное обеспечение уже установлено.

2. Завести публичный git-репозиторий и зарегистрировать его в качестве тикета (issue) в выбранном репозитории: репозиторий можно завести на любом публичном хостинге, поддерживающем git clone (GitHub, GitLab, GitLab на факультете Вычислительной математики и кибернетики для студентов или любой аналогичный ресурс); скорее всего понадобится сгенерировать и зарегистрировать ssh-ключ. Примерный вид сгенерированного ключа следующий:

```
ED25519 key fingerprint is
    MD5:55:1e:6a:4c:32:99:1b:fb:fa:d3:15:b9:a5:6b:94:53.
+--[ED25519 256]--+
|        oo. o    |
|        += + .   |
|         += .  . |
|        oo    o E|
|        S.     B
B
oo
o
|         ... .   |
MD
```

7 Работа с терминалом и простой проект

Идентификатор темы для сайта: 01_TerminalProject

В рамках данной темы рассматривается устройство текстового терминала и принципы работы с ним, без детального описания используемых программных инструментов.

7.1 Терминал

Исторически, терминал — это устройство, которое обеспечивает ввод и вывод байтов на компьютер. Название «терминал» появилось, поскольку на этом месте компьютер заканчивается, а начинается аппаратура ввода-вывода.

Режимы работы терминала:

1. raw mode (необработанный режим): последовательность байтов;
2. cooked mode (обработанный режим):
   - управление со стороны операционной системы (сигналы с клавиатуры):
     stty / stty —a;
   - преобразование ввода-вывода:
     - перевод строки (^M / ^J);
     - примитивное редактирование ввода-вывода (^U / ^W / ^H).

7.2 Управление выводом со стороны приложения

Варианты управления выводом:

1. унаследованные способы управления (подчёркивание, двойная печать и т. п.);
2. задание отображаемого текста (printf, echo);
3. изменение атрибутов текста;
4. позиционирование курсора;
5. трансляция в терминал ввода с дополнительных устройств (например, мыши).

7.3 База информации о терминалах Terminfo

Особенности базы Terminfo:

1. унификация управляющих последовательностей;
2. доступ через infocmp;
3. вывод через tput;
4. проблемы при неправильном значении переменной $TERM.

7.4 Ncurses

Ncurses — свободно распространяемая реализация Curses, библиотеки для работы с произвольным терминалом. Базовые свойства ncurses:

1. независимость от типа терминала;
2. экран = матрица символов (а не телетайп с последовательным вводом);
3. функции вывода заполняют эту воображаемую матрицу (а не настоящий экран), экран обновляется отдельной командой.

Источники информации о curses:

1. *простая методичка*[11];
2. *учебник на русском*[12] (по старому curses, зато простой);
3. *учебник на английском*[13];
4. *учебник от автора ncurses*[14] (изначально — от самого *ESR*а[15]);
5. *перевод*[16].

7.5 Примитивный makefile

Компилировать проект вручную — занятие весьма утомительное, особенно когда исходных файлов становится больше одного, и для каждого из них надо каждый раз набивать команды компиляции и компоновки. Для того, чтобы не повторять одни и те же рутинные действия вручную, возможно использовать функционал makefile и программы make.

Make — утилита, предназначенная для автоматизации преобразования файлов из одной формы в другую. Правила преобразования задаются в скрипте с именем Makefile, который должен находиться в корне рабочего каталога проекта. Сам скрипт состоит из набора правил, которые в свою очередь описываются:

1. целями (результат выполнения правила);
2. реквизитами (входные данные для выполнения правила и получения целей);
3. командами (выполняю преобразование реквизитов в цели).

7.6 Домашнее задание

Предполагается, что на запись на курс выполнена и каждый слушатель сообщил преподавателю адрес своего репозитория для домашних заданий.

11 Введение в ncurses [Электронный ресурс]. URL: http://grep.cs.msu.ru/dkhramov/Comp.NcursesTutorial.html (дата обращения: 20.01.2025).

12 CURSES/TERMINFO [Электронный ресурс]. URL: http://grep.cs.msu.ru/BESTA/ruprog/curter.doc.txt (дата обращения: 20.01.2025).

13 NCURSES Programming HOWTO [Электронный ресурс]. URL: https://tldp.org/HOWTO/html_single/NCURSES-Programming-HOWTO (дата обращения: 20.01.2025).

14 Writing Programs with NCURSES [Электронный ресурс]. URL: https://invisible-island.net/ncurses/ncurses-intro.html (дата обращения: 20.01.2025).

15 Wikipedia: Реймонд, Эрик Стивен [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Реймонд %2C_Эрик_Стивен (дата обращения: 20.01.2025).

16 Введение в программирование с curses [Электронный ресурс]. URL: http://grep.cs.msu.ru/ncurses.html (дата обращения: 20.01.2025).

1. Сделать в репозитории подкаталог, идентифицирующий данную тему, например 01_TerminalProject, и поместить в него решение описанной далее задачи.

2. С помощью интернета и здравого смысла написать на ncurses программу Show.c, которая постранично просматривает файл (слишком длинные строки усекаются или переносятся — как вам удобнее):

- имя файла передавать параметром командной строки;
- файл должен показываться в окне, а в первой строке экрана должно содержаться его имя;
- весь файл хранить в памяти, например, одним куском или в виде списка строк;
- при нажатии пробела файл прокручивается дальше (если есть такая возможность), при нажатии ESCпрограмма завершается;
- имеет смысл очищать окно при помощи werase(), а потом всё заново в него выводить;
- многобайтовую кодировку можно не поддерживать;
- исполняемый файл должен называться Show;

3. Сделать Makefile и добиться работоспособности make (обратите внимание на использование табуляций или символа, заданного в RECIPEPREFIX);

4. Не забыть опубликовать решение в репозитории;

5. Дополнительно, для тех, кому стало интересно:

- при нажатии «стрелки вправо» выводить строки файла, начиная со следующей колонки (так можно посмотреть урезанные длинные строки; начала строк при этом не видны); «стрелку влево» обработать аналогично;
- реализовать обработку PgUp, PgDown и стрелок вверх/вниз — прокрутка на один экран вперёд/назад, на одну строку вперёд-назад, остальные клавиши игнорировать;
- … далее везде.

8 Многофайловая сборка

Идентификатор темы для сайта: 02_Multifile

Данная тема посвящена решению проблемы взаимоувязки значительного числа исходных файлов между собой и с результатами их обработки.

Многофайловая организация проекта — это представление проекта в виде набора исходных файлов.

8.1 Сборка из нескольких файлов

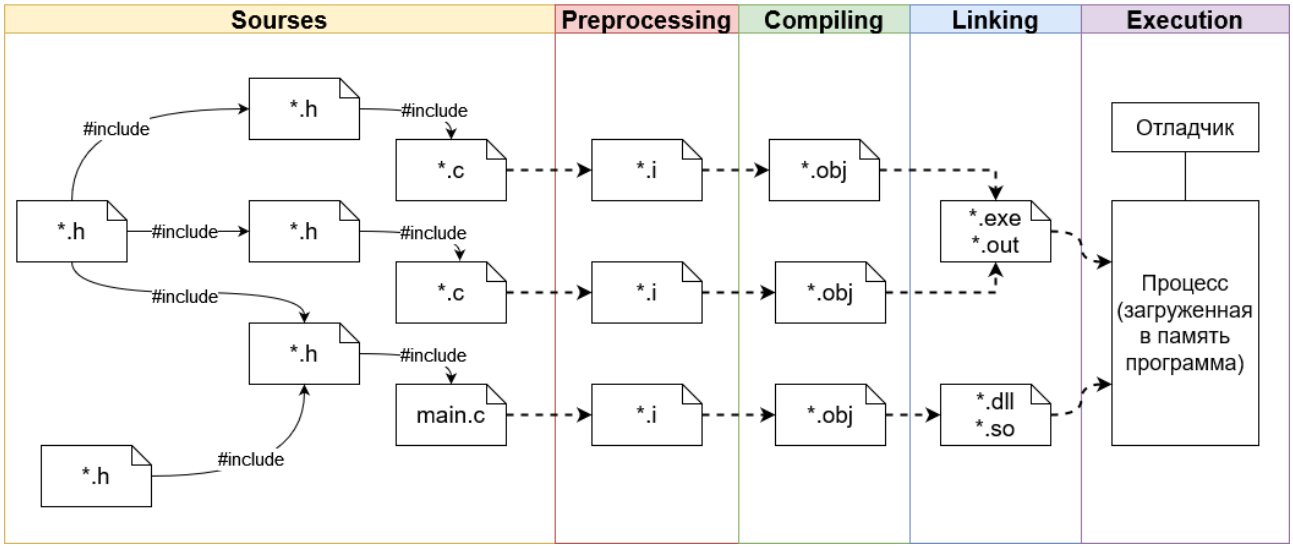
Для чего проекты делятся на файлы:

1. простота понимания кода;
2. ограничение области видимости имён функций («пространства имен» в языке C);
3. скорость компиляции (при локальных изменениях);
4. снижение использования памяти при компиляции.

Сборка (build) — процесс получения программы или библиотеки из исходного кода. Сборку разделяют на 3 основных этапа:

1. препроцессирование (preprocessing) — выполнение директив препроцессора (#include, #define и пр.), у результата обычно расширение .i;
2. компиляция (compiling) — перевод в машинный код, у результата обычно расширение *.obj или *.o (он же объектный файл);
3. компоновка (linking) — у результата обычно расширение .exe/.out (исполняемый файл) и .dll/.so (файл динамической библиотеки), либо без расширения.

Если один из этапов завершился неуспешно, то будет выведено сообщение компилятора или компоновщика с диагностикой ошибок.



8.2 Make

Основная проблема многофайловой сборки — это сложность ее выполнения вручную, при этом помещение всех команд компиляции в командный сценарий приводит к ненужным действиям при его выполнении, например перекомпиляции не изменившихся исходных файлов.

Make перекомпилирует только обновлённые исходники, для чего строит и анализирует динамический граф зависимостей объектов. Makefile — это по сути сборник рецептов, из которых получается выстроить причинно-следственные взаимосвязи, иногда достраивая не описанные явно. На основе данного графа проводится анализ, какие из целей устарели по отношению к источникам сборки и требуют пересборки. После этого запускаются только те рецепты и в том порядке, которые указаны в графе зависимостей.

Кроме того, make поддерживает параллельную сборку. Возможен запуск сразу нескольких компиляций, при условии отсутствия зависимостей между ними.

8.3 GNU make

GNU make является GNU-реализацией программы make, широко используемой в мире UNIX. По сравнению с традиционным набором функций, GNU make включает в себя ряд расширений. GNU make не полностью совместим с make, например — обратная косая черта не обрабатывается в соответствии с POSIX, а поскольку вызовы командной оболочки выполняются без опции '-e', GNU make не всегда отменяет дальнейшее выполнение сборки после неудачного выполнения команд.

Возможно раздельное описание правил для компиляции и компоновки:

```
# makefile

prog:   const.o fun.o prog.o
      cc const.o fun.o prog.o -o prog

fun.o:  fun.c
      cc fun.c -c -o fun.o

prog.o: prog.c
      cc prog.c -c -o prog.o

const.o:    const.c
      cc const.c -c -o const.o

clean:
      rm -f prog a.out *.o
```

8.4 Литература по GNU make

Рекомендуемая для изучения литература:

1. *один из первых учебников*[17] make (достаточно подробный и с объяснениями);

17 MAKE [Электронный ресурс]. URL: http://grep.cs.msu.ru/BESTA/ruprog/make.doc.txt (дата обращения: 20.01.2025).

2. *Managing Projects with GNU Make*[18], 3rd Edition;
3. *Documentation*[19];
4. Перевод документации в библиотеке *Linux по-русски*[20];
5. Владимир Игнатов *Эффективное использование GNU Make*[21];
6. примеры *написания и использования make*[22].

8.5 Переменные

Традиционно, порождаемые при сборке файлы делят на совсем не нужные на выходе сборки (TRASH) и собственно целевые файлы проекта (GENERATES), которые всё равно стоит удалять, если в репозитории должны остаться только исходники.

```
# makefile

GENERATES = prog README
TRASH = *.o *~ o.*

%.o:    %.c
      cc $< -c -o $@

all:    README prog

prog:   const.o fun.o prog.o
      cc $^ -o $@

README: prog
      ./$< 2> $@

clean:
      rm -f $(TRASH)

distclean:      clean
      rm -rf $(GENERATES)
```

8.6 Правила, применяемые по умолчанию

У make есть огромное количество правил по умолчанию. Их можно посмотреть с помощью команды make -p. Одно из правил по умолчанию задает генерацию объектных файлов .o по исходным файлам .c при помощи компилятора cc. В связи с наличием такого правила по умолчанию, прописывать его в makefile не обязательно.

18 Managing Projects with GNU Make, 3rd Edition [Электронный ресурс]. URL: https://www.oreilly.com/library/view/managing-projects-with/0596006101/ (дата обращения: 20.01.2025).

19 GNU make [Электронный ресурс]. URL: https://www.gnu.org/software/make/manual/make.html (дата обращения: 20.01.2025).

20 GNU Make. Программа управления компиляцией [Электронный ресурс]. URL: https://rus-linux.net/nlib.php?name=/MyLDP/algol/gnu_make/gnu_make_3-79_russian_manual.html (дата обращения: 20.01.2025).

21 Эффективное использование GNU Make [Электронный ресурс]. URL: https://www.opennet.ru/docs/RUS/gnumake/ (дата обращения: 20.01.2025).

22 Пример написания и использования Makefile [Электронный ресурс]. URL: https://uneex.org/FrBrGeorge/MakefileExample (дата обращения: 20.01.2025).

8.7 Статические библиотеки

Библиотека — это набор скомпонованных особым образом объектных файлов. Библиотеки подключаются к основной программе во время компоновки. По способу компоновки библиотеки подразделяют на архивы (статические библиотеки, static libraries) и совместно используемые (динамические библиотеки, shared libraries).

Библиотеки служат важной цели: дают возможность использовать одни и те же механизмы в разных программах. В Linux библиотеки используются повсеместно, поскольку это очень удобный способ "не изобретать велосипеды". Даже ядро Linux в каком-то смысле представляет собой библиотеку механизмов, называемых системными вызовами.

Статическая библиотека — это просто архив объектных файлов, который подключается к программе во время компоновки. Эффект такой же, как если бы вы подключали каждый из файлов отдельно.

Статические библиотеки делают программу более автономной: программа, скомпонованная со статической библиотекой, может запускаться на любом компьютере, не требуя наличия этой библиотеки (она уже "внутри" бинарного файла).

Динамическая библиотека компонуется с исполняемым файлом при его запуске. Поскольку в файловой системе хранится единственный экземпляр динамической библиотеки, не тиражируемый по всем использующим ее исполняемым файлам, такие библиотеки также называются совместно используемыми (или разделяемыми).

В Linux статические библиотеки обычно имеют расширение .a (Archive), а динамические библиотеки имеют расширение .so (Shared Object). Хранятся библиотеки, как правило, в каталогах /lib и /usr/lib.

8.8 Динамические библиотеки

Второй вариант сборки — использование динамической библиотеки.

Динамическая (.so) библиотека имеет совсем другой формат, чем статическая, и она подгружается в память в момент запуска программы, а в самой программе не содержится. В некоторых случаях динамическую библиотеку даже можно запустить.

В отличие от статических библиотек, код совместно используемых (динамических) библиотек не включается в бинарный файл. Вместо этого в него включается только ссылка на библиотеку.

Программа, скомпонованная с динамической библиотекой, требует наличия этой библиотеки на том компьютере, где она запускается, поскольку в бинарном файле программы содержится не сам код библиотеки, а ссылка на этот код. Несмотря на такую зависимость, динамические библиотеки обладают двумя существенными преимуществами. Во-первых, файл, скомпонованный с совместно используемой библиотекой меньше размером, чем файл той же программы с подключенной к нему статической библиотекой (статически скомпонованный). Во-вторых, любая модернизация динамической библиотеки отражается на всех программах, использующих ее. Таким образом, если некоторую библиотеку используют несколько программ, то исправление какой-нибудь ошибки в ней или любое другое улучшение библиотеки ав-

томатически улучшает все программы, которые используют эту библиотеку. Именно поэтому динамические библиотеки называют совместно используемыми. Чтобы применить изменения, внесенные в статическую библиотеку, нужно пересобрать все программы.

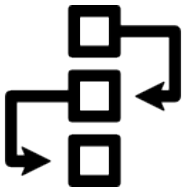
8.9 Другие системы сборки

Системы сборки кода — это специальные программы, которые собирают и пересобирают код проекта в автоматическом режиме по заранее заданным правилам. Эти системы определяют зависимости между файлами с исходным кодом и выходными файлами (программами, библиотеками и конфигурационными файлами) и в нескольких параллельных процессах выполняют команды компиляции для всех изменившихся со времени последней сборки файлов, соблюдая зависимости. Вторая задача систем сборки — это поиск в операционной системе и подключение к проекту библиотек и программ, что выполняется различными способами в зависимости от операционной системы.

В новых языках программирования (Rust, Go) параллельная сборка и поиск зависимостей уже встроены на уровне средств языка, но для давно существующих языков (C, C++, Fortran) это затруднительно сделать, поэтому для них используют отдельные системы сборки. Кратко опишем эти системы.

Система сборки является самым важным элементом любого проекта. Эта программа генерирует команды для сборки исходного кода, и чем быстрее эта система собирает код и чем больше рутинных операций автоматизирует, тем быстрее идет разработка, и тем проще настроить непрерывную интеграцию — автоматизированную сборку и тестирование вашей программы. В задачи системы сборки входит:

1. поиск зависимостей между файлами проекта;
2. формирование команд для генерации файлов (промежуточных и итоговых);
3. выполнение этих команд (возможно, во внешней командной оболочке);
4. формирование различных версий кода в зависимости от платформы, на которой происходит сборка.



8.10 Домашнее задание

1. В репозитории с домашним заданием создать подкаталог 02_Multifile и поместить туда код из примера применения make; убедиться, что он работает или исправить ошибки, если они есть.

2. Отредактировать Makefile так, чтобы в цели по умолчанию:

   а. собирались две библиотеки: liboutput_static.a и liboutput.so из файлов fun.o и const.o (одна статическая, другая — динамическая);
   б. собирались три бинарных файла: prog (из .o-файлов), prog-a (из prog.o и liboutput_static.a) и prog-so (из prog.o и liboutput.so).

3. Отредактировать Makefile так, чтобы имелась цель test, для достижения которой каждый из трёх бинарных файлов запускается:

   а. сначала без параметров (при запуске без параметров в stderr выводится help — необходимо обратить внимание на перенаправление и на то, что argv[0] у программ не совпадают; перенаправление обоих выходных потоков (stderr и

stdout) можно делать в виде ./prog > outfile 2>&1; чтобы выводы совпали, в исходном тексте разрешается вместо argv[0] использовать одинаковую константу, например, prog);

б. затем с одним каким-нибудь параметром;

в. наконец, с тремя параметрами (для каждого бинарного файла параметры одинаковые).

4. Сравнить выводы соответствующих запусков, например, с помощью cmp file1 file2, который вернёт ненулевой статус ошибки, если файлы не одинаковы.

5. Отредактировать Makefile так, чтобы все генераты (включая файлы с выводами из тестов) удалялись по команде make clean.

9 Обработка текстов и сценарии

Идентификатор темы для сайта: 03_Text

В большинстве тем данного курса рассматриваются отдельные виды инструментов, но в процессе разработки эти инструменты необходимо использовать совместно. Для этой цели существуют языки склейки, типичным примером которых является язык командной оболочки — shell. Знание shell необходимо также для написания рецептов многофайловой сборки (например, в makefile), т. к. при сборке выполняются вызовы команд операционной системы, в которой находится сборочное окружение.

9.1 Языки склейки

Связующий код (glue code, склеивающий код, язык склейки) — программный код, который служит исключительно для сопряжения разных частей кода, и при этом не реализует никакую иную прикладную функцию. Связующий код может рассматриваться как программа-шаблон, содержащая обращения к функционально нагруженному коду.

Наглядной абстракцией для «склейки» компонентов является конвейерная организация работы программ, реализуемая в UNIX при помощи конструкции «|». При помощи этой конструкции можно объединить набор узкоспециализированных утилит для решения требуемой задачи. При выстраивании программ в конвейер, объем связующего кода увеличивается линейно от количества связываемых программ, но в общем случае эта зависимость может приближаться к квадратичной.

9.2 Shell как язык склейки

Командная оболочка (shell) — это программа, которая организует среду для выполнения других программ и команд.

Командная оболочка имеет свои встроенные команды, арифметические операторы и другие синтаксические конструкции. Её основная задача — упрощать запуск других программ.

Командная оболочка ищет программы в текущем каталоге и в путях, указанных в переменной среды PATH, а также управляет сменой текущего каталога и переменными окружения.

Основная задача оболочки — интерпретировать команды пользователя и выполнять их, независимо от того, внешние это программы или внутренние команды.

В shell большинство операций выполняется с текстом. Даже арифметические операции проводятся над текстовыми данными, которые проверяются на принадлежность к числам или более простым арифметическим выражениям. Ввод и вывод файлов также текстовый, подстановка представляет собой склейку текстовых данных. Таким образом, помимо функций командной строки shell поддерживает значительный набор возможностей для работы с текстом.

Основные возможности shell:

1. выполнение арифметических операций;

2. экспорт переменных;
3. использование переменных и их подстановки с простейшей постобработкой;
4. поддержка встроенных команд и программ-команд;
5. перенаправление ввода-вывода (вывод в строковое выражение, конвейеры);
6. условный оператор и циклы;
7. генерация имён файлов;
8. шаблоны;
9. обработка сигналов.

9.3 Утилиты для работы с файловой системой и процессами

GNU Core Utilites — это набор утилит для выполнения базовых пользовательских операций: создание каталога, вывод файла на экран и т. д. По замыслу разработчиков, эти утилиты должны быть доступны в любой операционной системе, что и наблюдается в настоящее время. Coreutils содержит такие часто используемые утилиты, как cat, tail, echo, wc и многие другие.

Наиболее значимые утилиты:

1. классические — cp, mv, ls, cat, echo и т. п.;
2. find;
3. xargs;
4. mktemp.

9.4 Утилиты для работы с текстом

Основные утилиты (в скобках указаны названия пакетов):

1. преобразователи текста: cut, tr, tail, head, sort, paste, join (GNU Core Utilities);
2. преобразователи форматов: hexdump (util-linux), xxd (vim) и od (coreutils);
3. moreutils.

9.5 Комбайны

Основные комбайны (многофункциональные инструменты для работы с текстом) в Linux:

1. grep — поиск;
2. sed — поиск с заменой;
3. awk (или perl) — написание программ, управляемых контекстом (более компактных, чем на других языках программирования);
4. m4.

9.6 Домашнее задание

1. В репозитории с домашним заданием создать подкаталог 03_Text, в котором разработать shell-сценарий randomize.sh, который:

a. принимает один необязательный параметр — время задержки в секундах (вещественное) перед выводом очередного символа;

б. считывает со стандартного ввода небольшой файл с ASCII-графикой в кодировке ASCII и без символов табуляции;
в. выводит его посимвольно в случайном порядке, при этом каждый символ выводится в ту же позицию (относительно левого верхнего угла терминала), в которой этот символ находится в файле (относительно начала первой строки файла);
г. в результате на экране должна постепенно появляться та же ASCII-картинка, что и во входном файле.

2. Требуемые инструменты:

- перенаправление ввода-вывода;
- tput (cup и clear) из пакета termutils;
- арифметика в shell (конструкция вида $((…)));
- параметры командной строки shell-сценария (конструкции вида $1, $2 и т. п.);
- цикл while в shell;
- od или hexdump (с od необходимо использовать ключ -v);
- tr или sed;
- shuf;
- sleep.

3. Запрещено пользоваться другими языками программирования, кроме shell и, при необходимости, sed.

4. Скрипт по возможности должен выполняться стандартным shell: не bash, а, например dash, или ash (обратите внимание, что встроенное echo в ash не работает с ключом -e, а /bin/echo работает).

5. Следует научиться работать с псевдографикой и русскими буквами, поскольку русские буквы в utf-8 занимают два байта, а псевдографика — три, что требует предварительной перекодировки потока (iconv) в usc-2, для которой размер всех символов будет одинаковым.

6. Необязательное усложнение: выводить ASCII-графику посередине экрана, что потребует:

- подсчета во входном потоке количества строк и максимальной ширины строки (тривиальная задача);
- передачи полученных данных до передачи остальных (сложная задача);
- определения размера экрана командой stty, причём поскольку стандартный ввод — это не терминал, то можно использовать перенаправление вида stty size < /dev/tty);
- использования временного файла (mktemp) с последующим обязательным его удалением.

10 Отладка

Идентификатор темы для сайта: 04_Debugging

Достаточно полное изложение вопросов, связанных с отладкой приложений под Linux, представлено в книге *Linux Applications Debugging Techniques*[23]. В книге сделан акцент на использовании доступных в среде Linux средств диагностики и отладки программ, а также на работе с проблемами, специфическими для программ, использующих возможности среды Linux. Всего одна глава отведена непосредственно отладке кода, а полное освоение потребует достаточно хорошего знания структуры бинарных файлов.

10.1 ELF

ELF (Executable and Linkable Format) — формат исполняемых и связываемых файлов, определяющий структуру бинарных файлов, библиотек и файлов ядра (core files). Спецификация формата позволяет операционной системе корректно интерпретировать содержащиеся в файле машинные команды. Файл ELF, как правило, является выходным файлом компилятора или компоновщика и имеет двоичный формат. С помощью подходящих инструментов он может быть проанализирован и изучен.

Анализ содержимого файлов ELF необходим для:

1. общего понимания работы операционной системы;
2. разработки программного обеспечения;
3. цифровой криминалистики и реагирования на инциденты (DFIR — Digital Forensics and Incident Response);
4. исследования вредоносных программ (анализ бинарных файлов).

10.2 Общие правила отладки

Основные правила сборки программы, выполнение которые значительно упрощает процесс отладки программы, следующие:

1. отсутствие оптимизации (ключ -O0);
2. наличие отладочной информации (debuginfo) в двоичном коде программы и используемых библиотек;
3. наличие привязки двоичного кода к исходному тексту.

Для непосредственного процесса отладки необходим GDB (GNU Debugger) — инструмент для отладки программ в Linux, который позволяет анализировать и отлаживать программы, написанные на языках C, C++, Ada и Fortran.

23 Linux Applications Debugging Techniques [Электронный ресурс]. URL: https://en.wikibooks.org/wiki/Linux_Applications_Debugging_Techniques (дата обращения: 20.01.2025).

10.3 GDB

Программа GDB — это популярный отладчик для программ, написанных на языке программирования C и C++. Отладчик позволяет выполнить программу пошагово, посмотреть значения всех переменных на каждом из этапов выполнения, а если это необходимо, то дизассемблировать код и посмотреть состояние регистров процессора:

```
[alt@localhost 04_deb]$ gdb a.out
GNU gdb (GDB) 10.1-alt1 (ALT Sisyphus)
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
Reading symbols from a.out...
(gdb)
```

Основные команды gdb представлены в *документации*[24]:

1. выполнение: run, continue, next, step, finish, advance;
2. точки останова: по номеру строки (breakpoint; b main + run — это start); по изменению ячейки или выражения (watchpoint); по C++-исключению или системным вызовам (catchpoint);
3. просмотр ячейки, выражения, в т. ч. адресного — print, display; показ участка исходного кода вокруг точки останова — list;
4. запись в файл dump;
5. стек вызовов: просмотр — backtrace; переход между уровнями up, down;
6. *обратная отладка*[25];
7. *запись и воспроизведение*[26].

10.4 Пример работы со стеком

В данном разделе рассматривается стек как область памяти для временного хранения данных (не путать со стеком вызовов).

Код для проверки работоспособности:

```
// err.c

#include <stdio.h>

int tst(long *p) {
  long x;
  x = (long) p;
  return 0;
}

int tst2() {
```

24 GDB: The GNU Project Debugger [Электронный ресурс]. URL: https://www.gnu.org/software/gdb/documentation/ (дата обращения: 20.01.2025).

25 Running programs backward [Электронный ресурс]. URL: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Reverse-Execution.html (дата обращения: 20.01.2025).

26 Recording Inferior's Execution and Replaying It [Электронный ресурс]. URL: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Process-Record-and-Replay.html (дата обращения: 20.01.2025).

```
  long int *a;
  *a = 42;
  return 0;
}

int main(int argc, char *argv[]) {
        long b=7;
        tst(&b);
        tst2();
        printf("⇒ %ld\n", b);
        return 0;
}
```

10.5 Пакеты devel

Пакет devel для некоторой библиотеки содержит файлы, необходимые для разработки приложений с использованием этой библиотеки. К имени таких пакетов добавляется суффикс -devel.

В таких пакетах обычно содержатся:

1. заголовочные файлы C/C++;
2. документация;
3. библиотеки (.a, .so), в т. ч. собранные с отладочной информацией.

10.6 Пакеты debuginfo

Для отладки программы, использующей некоторые библиотеки, нет необходимости устанавливать devel-пакеты этих библиотек, вполне достаточно иметь в распоряжении пакеты debuginfo, содержащие отладочную информацию для библиотек.

10.7 Debuginfod-сервер

Если пакеты debuginfo установить затруднительно, то можно использовать из пакета elfutils *debuginfod-сервер*[27], который предоставляет отладочную информацию без установки таких пакетов.

Сам пакет elfutils содержит набор утилит и библиотек для работы с файлами ELF, а *debuginfod*[28] — это способ распределения файлов debuginfo.

10.8 Отладка ранее запущенного процесса

GDB можно присоединить к уже запущенным процессам с помощью команды attach. После присоединения к процессу gdb остановит его выполнение. Это позволяет подготовить сеанс отладки с помощью команд gdb, например, установить точки останова или точки наблюдения. Затем можно использовать команду continue, чтобы про-

27 ALT debuginfod server [Электронный ресурс]. URL: https://debuginfod.altlinux.org/ (дата обращения: 20.01.2025).

28 Elfutils debuginfod [Электронный ресурс]. URL: https://sourceware.org/elfutils/Debuginfod.html (дата обращения: 20.01.2025).

цесс продолжил выполнение. Несмотря на то, что gdb — это последовательный отладчик, он может исследовать параллельные программы, присоединившись к отдельным процессам программы.

Для демонстрации работы необходимо запустить процесс, например задержки, в одном терминале:

```
[alt@localhost 04_deb]$ sleep 5 | ps -ed | grep sleep
  10605 pts/1    00:00:00 sleep
```

а перехват в другом:

```
[alt@localhost 04_deb]$ gdb -q -p 10605
Attaching to process 10605
ptrace: Операция не позволена.
(gdb)
```

Подключение не удалось, поскольку оно запрещено модулем безопасности Yama для Linux, разработанным компанией Canonical. Для проверки текущих значений трассировки выполнить (от имени администратора root):

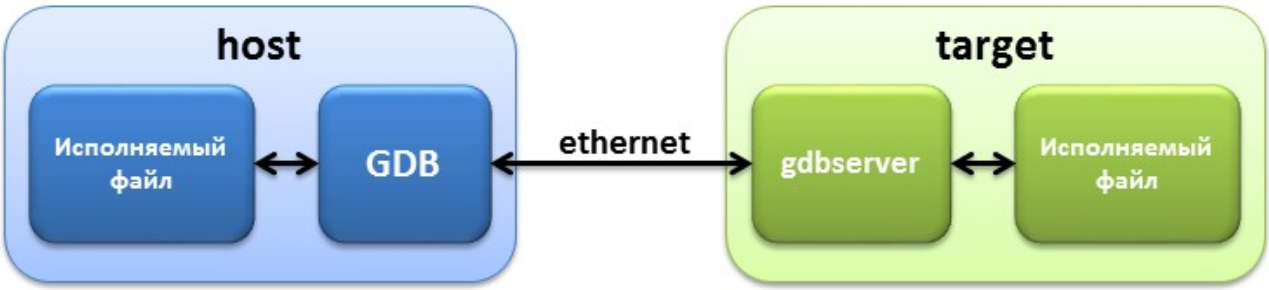
```
[alt@localhost 04_deb]$ sudo sysctl -a | grep ptrace
[sudo] password for alt:
kernel.yama.ptrace_scope = 1
```

10.9 Удаленная отладка

Задача пошаговой отладки становится не такой тривиальной в случае программирования под Linux на встраиваемых системах (таких как rasberry pi, virt2real или промышленные процессорные модули).

Данную задачу в Linux призвана решать стандартная связка программ gdb и gdbserver. Идея в том, чтобы писать программу на компьютере (host в терминологии GDB), там же ее компилировать, и заливать на целевое устройство (target). Далее запускать на целевом устройстве (target) отлаживаемый файл и gdbserver, а на хосте запускать GDB и работать с ним.

Схема взаимодействия при такой отладке кратко представлена на рисунке.



Исполняемый файл — это удаленно отлаживаемая программа. Для корректной работы, экземпляр программы должен находиться не только на целевом устройстве (target), но и на компьютере (host), с которого идет отладка.

GDB — программа, которая непосредственно выполняет процесс отладки, а gdbserver — ответная часть gdb, которая запускает исполняемый файл в режиме отладки.

10.10 Интерфейсы для gdb

Перечень наиболее часто используемых интерфейсов для GDB:

1. TUI (Text User Interface) — встроенный в gdb текстовый интерфейс с предпросмотром кода исполняемой программы;
2. CGDB — это основанный на curses интерфейс к отладчику GDB;
3. графические интерфейсы к GDB для Linux, например Seer, GeDe, gf;
4. web-интерфейсы, например gdbgui, GDBFrontend.

10.11 Сценарии для gdb

Для gdb есть возможность писать сценарии отладки, которые будут автоматически выполнять команды gdb. Это особенно востребовано при многократных попытках отладки одной и той же части кода несколько раз. Вместо того, чтобы вручную устанавливать точку останова, выполнять код и печатать значения переменных каждый раз при загрузке двоичного файла, возможно использовать сценарий gdb для автоматизации всего процесса.

Есть несколько полезных публикаций:

1. описание в *руководстве gdb*[29];
2. *обучающая статья*[30];
3. несколько наглядных *примеров*[31].
4. сценарии для gdb на *python*[32] (в *документации*[33]).

10.12 Домашнее задание

1. Установить в сборочное окружение -debuginfo версии библиотек. В разных дистрибутивах они могут называться по-разному, совпадать с -devel версиями и даже отсутствовать. В ALT библиотеки называются lib_____-debuginfo-<версия> и располагаются в отдельной секции репозитория. Пример sources.list с сервера практикума:

```
frbrgeorge@linuxprac ~/src $ grep "^[^#]" /etc/apt/sources.list.d/yandex.list
rpm [alt] http://mirror.yandex.ru/altlinux Sisyphus/x86_64 classic debuginfo
rpm [alt] http://mirror.yandex.ru/altlinux Sisyphus/x86_64-i586 classic
rpm [alt] http://mirror.yandex.ru/altlinux Sisyphus/noarch classic
```

2. Просмотреть материалы на страницах make-your-debugging-easier и gdb-scripting.

29 Command Files [Электронный ресурс]. URL: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Command-Files.html (дата обращения: 20.01.2025).

30 GDB: A quick guide to make your debugging easier [Электронный ресурс]. URL: https://johnysswlab.com/gdb-a-quick-guide-to-make-your-debugging-easier/ (дата обращения: 20.01.2025).

31 Scripting GDB [Электронный ресурс]. URL: https://sdimitro.github.io/post/scripting-gdb/ (дата обращения: 20.01.2025).

32 An introduction to GDB scripting in Python [Электронный ресурс]. URL: https://blog.vmsplice.net/2020/02/an-introduction-to-gdb-scripting-in.html (дата обращения: 20.01.2025).

33 Python API [Электронный ресурс]. URL: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Python-API.html (дата обращения: 20.01.2025).

3. Написать программу-генератор арифметической прогрессии range.c, принимающую от одного до трёх параметров по аналогии с range() на python:

- без параметров выводит help;
- с одним параметром N выводит построчно последовательность [0, 1, …, N-1];
- с двумя — M, N — последовательность [M, M+1, … N-1];
- с тремя — M, N, S — последовательность [M, M+S, M+2S, … N-1];

4. Написать следующие сценарии gdb (формат вывода произвольный):

- запустить ./range с параметрами 1 12 и вывести содержимое переменных, отвечающих на начало, конец, шаг и текущее значение элемента прогрессии, только когда этот элемент кратен 5;
- запустить ./range с параметрами -100 100 3 и вывести содержимое переменных, отвечающих на начало, конец, шаг и текущее значение элемента прогрессии только для 28, 29, 30, 31, 32, 33, 34, и 35 по счёту элементов прогрессии.

5. Написать Makefile, в котором содержатся следующие действия:

- компиляция бинарного файла с ключами -O0 -g;
- цель test — запуск этих сценариев gdb и сравнение их вывода с эталонным;
- цель clean — удаление всех генератов;
- gdb часто выводит много лишней динамической информации, например, адреса, которые вполне могут не повторяться при новом запуске;
- возможно использовать в сценариях gdb связку echo @@@ и print …, после чего отыскивать (grep) строки, начинающиеся на @@@, и с эталоном сравнивать только их;
- применение --batch и --quiet скорее необходимо, а вот logging нет;
- чтобы вывод утилиты range не порождал лишней информации, его целесообразно перенаправлять в /dev/null прямо в самой команде run.

6. В случае достаточности навыков программирования на C, дописать функции и типы данных к данной программе и решить домашнее задание с ней:

```
int main(int argc, char *argv[]) {
        range I;
        argparse(argc, argv, &I.start, &I.stop, &I.step);
        for(range_init(&I); range_run(&I); range_next(&I))
                printf("%d\n", range_get(&I));
        return 0;
}
```

7. Создать в репозитории с решениями подкаталог 04_Debugging и разместить в нем все исходники решения описанной выше задачи.

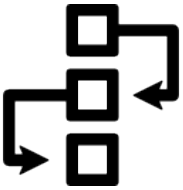
11 Регулярные выражения

Идентификатор темы для сайта: 05_Regexps

Одним из принципов UNIX-систем является широкое использование текстовых данных: конфигурационные файлы, входные и выходные данные программ в Linux часто организованы в виде обычного текста. Регулярные выражения — это мощный инструмент для манипуляции текстовой информацией.

Регулярные выражения — это специальным образом записанные строки, используемые для поиска символьных шаблонов в тексте. Чем-то они похожи на групповые символы в оболочке, но их возможности куда шире. Многие утилиты для работы с текстом в Linux и языки программирования включают в себя механизм регулярных выражений. Здесь возникают проблемы: разные программы и языки оперируют различными диалектами регулярных выражений.

Исходя из принципов текстоориентированности систем Linux, схема применения регулярных выражений следующая:



1. объектами являются последовательности строк;
2. шаблон также представляет собой строку, состоящую из обычных символов и спецсимволов;
3. при поиске по регулярному выражению производится подстановка (под)строк в шаблон (matching, searching);
4. выполняется фильтрация строк как соответствующих или не соответствующих шаблону.

11.1 Шаблоны shell

Язык шаблонов shell довольно ограничен, поскольку предназначен для достаточно простого анализа строк:

1. механизм сопоставления с шаблоном *glob*[34], используемый для подстановки имени файла в командной строке;
2. генерация имён файлов и шаблоны в case.

11.2 Вводная информация о регулярных выражениях

Наиболее полное описание механизма регулярных выражений представлено в книге Jeffrey Friedl *Mastering Regular Expressions*[35] («книга о сове»). Данное издание достаточно полно описывает практику использования регулярных выражений, в том числе дает понимание как интерпретируется шаблоны регулярных выражений при их подстановке, а также интуитивное понимание необходимости использования тех или иных выражений.

34 glob(7) — Linux manual page [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man7/glob.7.html (дата обращения: 20.01.2025).

35 Mastering Regular Expressions [Электронный ресурс]. URL: https://regex.info/book.html (дата обращения: 20.01.2025).

Базовым инструментом регулярных выражений в командной строке Linux является утилита grep:

```
[alt@localhost 05_reg]$ cal
     Ноябрь 2024
Пн Вт Ср Чт Пт Сб Вс
              1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30

[alt@localhost 05_reg]$ cal | grep '8.*0'
 4  5  6  7  8  9 10
18 19 20 21 22 23 24
25 26 27 28 29 30
```

11.3 Определение регулярного выражения

Виды определения регулярных выражений:

1. атомарное;
2. повторители;
3. составное регулярное выражение;
4. позиционирование;
5. принцип группирования: самый левый, самый длинный.

Атомарный вид выражений включает в себя:

1. любой неспециальный символ: "E" → «E»;
2. точка "." — любой один символ:
   - "." → «E»;
   - "." → «:»;
   - "." → «.»;
4. диапазон — любой один символ из диапазона:
   - "[quack!]" → «a»;
   - "[quack!]" → «!»;
   - "[a-z]" → «q» (любая строчная буква);
   - "[a-z]" → «z» (любая строчная буква);
   - "[a-fA-F0-9]" → «f» (любая шестнадцатеричная цифра);
   - "[0-9A-Fa-f]" → «D» (любая шестнадцатеричная цифра);
   - "[bdefAEDFC0-9Bca]" → «4» (любая шестнадцатеричная цифра);
5. выключенный диапазон — любой один символ не из диапазона:
   - "[^quack!]" → «r»;
   - "[^quack!]" → «#»;
   - "[^quack!]" → «A»;
6. группировка: любое составное регулярное выражение в скобках "\(" и "\)".

Повторители включают в себя:

1. "атомарноеРВ*" — 0 или больше вхождений атомарногоРВ;
2. "a*" → «aaa»;
3. "a*" → «»;
4. "a*" → «a»;
5. "[0-9]*" → «7»;

6. "[0-9]*" → «»;
7. "[0-9]*" → «1231234»;
8. ".*" → любая строка!

Варианты составного регулярного выражения:

1. последовательность атомарных регулярных выражений (подставляется в строку, которую можно разделить на подстроки, в каждую из которых подставляется очередное атомарное регулярное выражение):

- "boo" → «boo»;
- "r....e" → «riddle»;
- "r....e" → «r re e»;
- "[0-9][0-9]*" → неотрицательное целое;
- "[A-Za-z_][A-Za-z0-9_]*" → Идентификатор (например, в C);

2. группировка превращает составное выражение в атомарное, так что после него можно использовать повторитель:

- "\([A-Z][a-z]\)*" → «ReGeXp»;
- "\([A-Z][a-z]\)*" → «»;
- "\([A-Z][a-z]\)*" → «Oi».

Способы позиционирования:

1. "^regexp" — только в начале строки;
2. "regexp$" — только в конце строки.

Принцип группирования самый левый, самый длинный:

1. из всех успешных сопоставлений составного регулярного выражения в строке выбираются те, в которых сопоставление первого атомарного регулярного выражения ближе всего к началу;
2. из выбранных сопоставлений выбираются те, в котором это сопоставление — самое длинное;
3. продолжается далее для всех последующих атомарных регулярных выражений.

Алгоритм недетерминированного конечного автомата (с откатом) "aabb" ? "abb*":

1. "aabbb" → "abb*";
2. нет соответствия для "b", откат на следующее соответствие для "a";
3. "aabbb" → "abb*";
4. "aabbb" → "abb*";
5. {*} "aabbb" → "abb*".

11.4 Диалекты (варианты синтаксиса) регулярных выражений

Существуют разные подходы к использованию "\\" в диалектах регулярных выражений POSIX, GNU, extended regex, Lisp и т. д.

Так, в командных оболочках почти всегда есть расширенные регулярные выражения (*extended regex*[36]) — sed -E, grep -E, awk и т. п. Основные отличия extended regex:

1. круглые и фигурные скобки — это спецсимволы, а "\)" … "\{" — это скобки;
2. спецповторители "+", "?" и "{число}"/"{число,число}", а "\+ и "\?" — это «+» и «?»;

36 regex(7) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man7/regex.7.html (дата обращения: 20.01.2025).

3. классы символов: "[[:alnum:].]+" → «2.3e1.2dsf3e..12asdf3.e123e.»;
4. альтернатива "|": хотя бы одно из выражений "[ab]+|[cd]+" → «ababaabab» или "[ab]+|[cd]+" → «dcccdccdc»

В разных языках отличаются правила квотирования, независимо от того является строка регулярным выражением или нет:

```
[alt@localhost 05_reg]$ /bin/echo \\
\
[alt@localhost 05_reg]$ /bin/echo \\\
> q
\q
[alt@localhost 05_reg]$ /bin/echo \\\\
\\
[alt@localhost 05_reg]$ /bin/echo \\\\\\
\\\
[alt@localhost 05_reg]$ echo \\\\\\
\\\
```

В оболочке zsh все немного по-другому:

```
[alt@localhost 05_reg]$ zsh
alt@localhost ~/Lin24/05_reg $ echo \\\\\\
\\
alt@localhost ~/Lin24/05_reg $ echo \\\\
\
alt@localhost ~/Lin24/05_reg $ echo \\
\
```

11.5 Поиск с заменой и обратные ссылки

Понятия кармана "\(…\)" и подстановки "\номер" в Linux связаны с регулярными выражениями и используются в инструментах поиска с заменой.

Карман — это любое составное регулярное выражение в скобках (и).

Подстановка — это замена части регулярного выражения на текст, сопоставленный ранее заданному карману, адресуемому номером кармана.

Пример поиска с подстановкой:

```
[alt@localhost 05_reg]$ cal | grep "Ч\(.\) П\1"
Пн Вт Ср Чт Пт Сб Вс
[alt@localhost 05_reg]$ cal | grep "\(.\)0 \11 \12"
 9 10 11 12 13 14 15
16 17 18 19 20 21 22
```

Команда *sed* — это потоковый редактор текста, работающий по принципу замены. Его можно использовать для поиска, вставки, замены и удаления фрагментов в файле. С помощью этой утилиты возможно также редактировать файлы, не открывая их. Замена выполняется двумя типами команд:

1. s/что_заменять/на_что_заменять/опции — замена символов (поддерживаются регулярные выражения);
2. y/символы/символы — замена символов из первой части на соответствующие символы из второй части.

Существуют определенные правила нумерации карманов, в т. ч. вложенных:

```
[alt@localhost 05_reg]$ date | sed 's/\([0-9][0-9]*\)/<\1>/'
Пн <23> дек 2024 00:32:14 MSK
[alt@localhost 05_reg]$ date | sed 's/\([0-9][0-9]*\)/<\1>/g'
Пн <23> дек <2024> <00>:<32>:<21> MSK
[alt@localhost 05_reg]$ date | sed -E "s/([[:digit:]]+)(.*)([[:digit:]]+)/<1=\
    1><2=\2><3=\3>/"
Пн <1=23><2= дек 2024 00:32:3><3=5> MSK
[alt@localhost 05_reg]$ date | sed -E "s/(([[:digit:]]+).*)([[:digit:]]+)/<1=\
    1><2=\2><3=\3>/"
Пн <1=23 дек 2024 00:32:4><2=23><3=5> MSK
```

11.6 Регулярные выражения в C

Регулярное выражение в C — это последовательность символов, которая используется для поиска определённого шаблона или сопоставления его со строками. Для работы с регулярными выражениями в C используется библиотека POSIX. Некоторые выражения, которые в ней используются:

1. ** []** — находит символы или числа, написанные внутри этих скобок;
2. [:number:] — находит любую цифру в числах;
3. [:lower:] — находит строчные буквы алфавита;
4. [:word:] — находит конкретное слово, которое может быть комбинацией букв, цифр или подчёркиваний в данном тексте.

Для компиляции регулярного выражения в C используется функция regcomp(). Она принимает три параметра: указатель на место в памяти, где хранится шаблон для сопоставления, указатель на шаблон строкового типа и флаг для указания типа компиляции.

Для сопоставления строки с шаблоном используется функция *regexec*[37]. Она принимает пять аргументов: предкомпилированный шаблон, строку, в которой нужно искать шаблон, информацию о расположении совпадений и флаги для указания изменений в поведении сопоставления.

Реализация утилиты аналогичной cat на C использует функцию *getline*[38] для ввода строки в Linux. Она считывает целую строку, сохраняя адрес буфера, содержащего текст, в указанный параметр. Буфер завершается символом null и содержит символ новой строки, если был найден разделитель для новой строки.

Освобождение памяти выполняется функцией free:

```
//gre.c

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
        char *buf;
        size_t len = 0;
        int chars;
```

37 regex(3) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man3/regcomp.3.html (дата обращения: 20.01.2025).

38 getline(3) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man3/getline.3.html (дата обращения: 20.01.2025).

```
        for(buf = NULL; (chars = getline(&buf, &len, stdin)) != -1; buf =
NULL) {
                puts(buf);
                free(buf);
        }
        return 0;
}
```

Реализуем несколько ограниченный вариант grep (с освобождением памяти из-под скомпилированного регулярного выражения с помощью regfree):

```
//gre.c

#include <stdio.h>
#include <stdlib.h>
#include <regex.h>

int main(int argc, char *argv[]) {
        char *buf;
        size_t len = 0;
        int chars;
        regex_t regex;

        regcomp(&regex, argv[1], 0);

        for (buf = NULL; (chars = getline(&buf, &len, stdin)) != -1; buf =
NULL) {
                buf[chars - 1] = 0;
                if (regexec(&regex, buf, 0, NULL, 0) == 0)
                        puts(buf);
                free(buf);
        }
        regfree(&regex);
        return 0;
}
```

Для работы с карманами необходимо regexec передать несколько параметров. Для хранения карманов использовать массив:

```
//gre.c

#include <stdio.h>
#include <stdlib.h>
#include <regex.h>

#define MAXGR 10

int main(int argc, char *argv[]) {
        char *buf;
        size_t len = 0;
        int chars, b, e;
        regex_t regex;
        regmatch_t bags[MAXGR];

        if (regcomp(&regex, argv[1], 0))
            exit(1);
```

```
        for (buf = NULL; (chars = getline(&buf, &len, stdin)) != -1; buf =
NULL) {
                buf[chars - 1] = 0;
                if (regexec(&regex, buf, MAXGR, bags, 0) == 0) {
                        puts(buf);
                        for(int i=0; i<MAXGR && bags[i].rm_so>=0; i++) {
                                b = bags[i].rm_so, e = bags[i].rm_eo;
                                printf("%d: %.*s\n", i, e - b, buf + b);
                        }
                }
                free(buf);
        }
        regfree(&regex);
        return 0;
}
```

11.7 Нерегулярные выражения

Из названия понятно, что это выражения, которые не представляются регулярным образом, в отличие от специальных выражений, повторителей и т. п., не выходящих за рамки автоматов, или регулярных грамматик по Хомскому. Нерегулярные выражения находятся вне области регулярных грамматик. Типичные примеры:

1. нерегулярные выражения PCRE или *re*[39]: нежадные повторители, пред- и постпросмотр;
2. libpcre: в *документации*[40], в частности, приведены синтаксис регулярных выражений pcresyntax и создание шаблонов pcrepattern.

11.8 Домашнее задание

Предварительно необходимо почитать про регулярные выражения в:

1. *документации*[41];
2. *викиучебнике*[42];
3. *уроках*[43];
4. *руководстве*[44];
5. книге Jeffrey Friedl *Mastering Regular Expressions*[45] («книга о сове»).

39 re — Regular expression operations [Электронный ресурс]. URL: https://docs.python.org/3/library/re (дата обращения: 20.01.2025).

40 pcre(3) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man3/pcre.3.html (дата обращения: 20.01.2025).

41 regex(7) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man7/regex.7.html (дата обращения: 20.01.2025).

42 Викиучебник: Регулярные выражения [Электронный ресурс]. URL: https://ru.wikibooks.org/wiki/Регулярные_выражения (дата обращения: 20.01.2025).

43 Уроки по Регулярным Выражениям [Электронный ресурс]. URL: https://ravesli.com/uroki-po-regexp/ (дата обращения: 20.01.2025).

44 Regex Tutorial [Электронный ресурс]. URL: http://www.rexegg.com/ (дата обращения: 20.01.2025).

45 Mastering Regular Expressions [Электронный ресурс]. URL: https://regex.info/book.html (дата обращения: 20.01.2025).

1. Попрактиковаться в написании регулярных выражений, при этом следует учитывать, что большинство сайтов поддерживают python или pcre, или javascript, но не классические регулярные выражения.

2. Написать программу esub с параметрами regexp substitution string, которая работает аналогично echo 'string' | sed -E 's/regexp/substitution/' — однократная замена (необходимы навыки программирования на C), при этом:

- используются *расширенные регулярные выражения*[46] (так меньше «\»);
- программа должна распознавать ошибки в регулярном выражении и выводить соответствующую диагностику с помощью *regerror*[47];
- в строке substitution программа должна распознавать ссылки на соответствующие карманы (самих карманов 9, ссылок может быть не более 100)
- также должны распознаваться ссылки на несуществующие карманы (например, \9 если в выражении нет стольких пар скобок — это ошибка), и конструкция «\\», которая должна заменяться на «\» (необходимо отслеживать количество «\» — обычно оно уменьшается вдвое при каждой обработке, но в разных shell может работать по-разному).

3. Опционально: реализовать режим раскрашивания, аналогично grep, в котором подстановка из каждого кармана окрашивается в свой цвет.

4. Написать Makefile, в который включить сборку, удаление генератов и тесты, где вывод esub сравнивается с выводом соответствующей команды sed -E s/…/…/.

5. Создать в каталоге с домашними заданиями подкаталог 05_Regexps и разместить в нем проект.

46 regex(7) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man7/regex.7.html (дата обращения: 20.01.2025).

47 regex(3) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man3/regerror.3.html (дата обращения: 20.01.2025).

12 Трассировка вызовов

Идентификатор темы для сайта: 06_Tracing

В операционной системе и используемых в ней программах иногда возникают ошибки, причину которых очень сложно понять, анализируя файлы журналов и сообщения об ошибках. Но и для таких ситуаций в Linux есть средства. За процессом работы любой из программ можно проследить, наблюдая системные вызовы, которые использует программа.

Главный интерфейс между программами и ядром ОС в UNIX — системные вызовы (system calls, syscalls), взаимодействие программ с внешним миром происходит исключительно через них.

Трассировка нужна для программиста, чтобы видеть последовательность выполнения команд. Он просматривает с помощью ее инструментов значения переменных на каждом шаге выполняемой программы.

12.1 Трассировка с помощью strace

Команда strace показывает все системные вызовы программы, которые та отправляет к системе во время выполнения, а также их параметры и результат выполнения. Но при необходимости можно подключиться и к уже запущенному процессу. В самом простом варианте strace запускает переданную команду с её аргументами и выводит в стандартный поток ошибок все системные вызовы команды.

Основные источники литературы:

1. *strace.io*[48];
2. *руководство strace*[49];
3. статьи про strace: *устройство и настройка*[50],*отладка*[51], *варианты применения*[52];
4. *ограничения перехвата вызовов*[53] (kernel.yama.ptrace_scope и strace -p PID / gdb -p PID).

Основные возможности strace:

1. отслеживание системных вызовов с помощью ptrace;
2. фильтр на вызовы и группы вызовов;
3. отслеживание подпроцессов;
4. трассировка стека вызовов и показ параметров;
5. внедрение ошибок.

48 Strace. Linux syscall tracer [Электронный ресурс]. URL: https://strace.io/ (дата обращения: 20.01.2025).

49 strace(1) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man1/strace.1.html (дата обращения: 20.01.2025).

50 Strace в Linux: история, устройство и использование [Электронный ресурс]. URL: https://habr.com/ru/company/badoo/blog/493856/ (дата обращения: 20.01.2025).

51 Отлаживаем развертывание ПО со strace [Электронный ресурс]. URL: https://habr.com/ru/companies/slurm/articles/478626 (дата обращения: 20.01.2025).

52 Strace [Электронный ресурс]. URL: https://habr.com/ru/post/215577/ (дата обращения: 20.01.2025).

53 Yama. Linux security module [Электронный ресурс]. URL: https://www.kernel.org/doc/Documentation/security/Yama.txt (дата обращения: 20.01.2025).

12.2 Переменная LD_PRELOAD и подмена библиотек

LD_PRELOAD — это переменная среды, которая содержит список дополнительных разделяемых объектов ELF, загружаемых раньше всех остальных.

Основное предназначение LD_PRELOAD — отладка или проверка функций в динамически подключаемых библиотеках. Использовать LD_PRELOAD можно двумя способами:

1. установить переменную среды LD_PRELOAD с файлом библиотеки: библиотека будет загружена раньше остальных для текущего пользователя и его окружения;
2. записать путь к библиотеке в файл /etc/ld.so.preload: библиотека будет загружена раньше остальных для всех пользователей системы.

LD_PRELOAD также может использоваться злоумышленниками для перехвата некоторых вызовов, таких как чтение файла, листинг каталога, процессов и прочих функций, позволяющих злоумышленнику скрывать своё присутствие в системе.

12.3 Прочие инструменты трассировки

ltrace[54] — это утилита для трассировки и отладки в Linux, которая отображает вызовы приложения к разделяемым библиотекам. Она анализирует информацию о связывании разделяемых библиотек программы и печатает используемые библиотечные функции.

xtruss — простая в использовании программа для отслеживания работы приложения с X-протоколом, предназначенным для отображения и управления графической информацией в UNIX. Он определяет пакеты информации, передаваемые между X-сервером и X-клиентом в обоих направлениях и поддерживает четыре базовых типа сообщений: запрос, ответ, оповещение о событии, сообщение об ошибке.

ftrace (Function Tracer) — трассирующая среда для ядра Linux, позволяющая отслеживать контекстные переключения, измерять время обработки прерываний, высчитывать время на активизацию заданий с высоким приоритетом и многое другое. Это фреймворк, предоставляющий отладочный кольцевой буфер для записи данных. Собирают эти данные встроенные в ядро программы-трассировщики. Чтобы приступить к работе с ftrace, нужно перейти в каталог sys/kernel/debug/tracing (она доступна только для root-пользователя):

SystemTap — это средство, которое позволяет собирать и анализировать информацию о работающей Linux-системе. SystemTap поддерживает интерфейс командной строки и скриптовый язык программирования. Системные администраторы могут использовать SystemTap для мониторинга и анализа производительности системы, а разработчики программного обеспечения могут использовать SystemTap для анализа поведения приложения в работающей системе.

54 ltrace(1) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man1/ltrace.1.html (дата обращения: 20.01.2025).

12.4 Домашнее задание

Предварительно необходимо почитать про strace, ltrace, errno и ld.so.

1. Написать программу move.c, принимающую два параметра в командной строке (move infile outfile), которая перемещает содержимое файла infile в файл outfile (если он существовал, то его содержимое удаляется), и в случае успеха удаляет infile. При этом:

- программа должна проверять по возможности все ошибки и сообщать о них, в том числе выставляя различные коды ошибки;
- программа должна по возможности работать безопасно — не удалять исходный файл, пока целевой не закрыт; удалять целевой файл и сохранять исходный, если есть подозрение, что что-то пошло не так.

2. Написать тесты с помощью strace error injection, которые позволяют проверить как можно больше ошибочных ситуаций (проверяют код ошибки и наличие того файла, который должен был остаться).

3. Написать .so библиотеку для LD_PRELOAD, при подгрузке которой файл, в имени которого есть строка "PROTECT", не удаляется (как минимум — не удаляется при помощи move) и тест, который это проверяет.

4. Написать Makefile со сборкой, запусками тестов и очисткой генератов.

5. Создать в репозитории каталог 06_Tracing и разместить в нем проект.

13 Адаптация к окружению при сборке

Идентификатор темы для сайта: 07_Environmental

Инструмент Make не предоставляет следующие возможности (а их реализация на языке Make затруднительна):

1. опциональная сборка (включение или отключение определенных опций);
2. поиск, в процессе компиляции, расположения библиотек, подключаемых (include) файлов, утилит и т. п.;
3. проверка наличия и/или работоспособности скриптов, библиотек;
4. настройка на различные варианты сборочного инструментария;
5. поддержка процесса разработки (workflow — заранее определённые действия для тестов, для документации, дисциплина хранения исходников и т. п.);
6. возможность скрытия обращения к инструментам командной строки за человекочитаемыми описаниями.

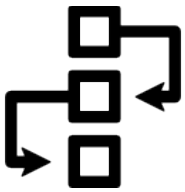
В результате оказывается востребованным высокоуровневый инструмент, для которого Makefile — это генерат.

13.1 Общие понятия Autotools

GNU Autotools — это система сборки проекта GNU, набор программных средств, предназначенных для поддержки переносимости исходного кода программ между UNIX-подобными системами.

Документации по данному инструменту достаточно много, например *перечень наиболее частых вопросов*[55].

Последовательность применения инструментов из состава Autotools:

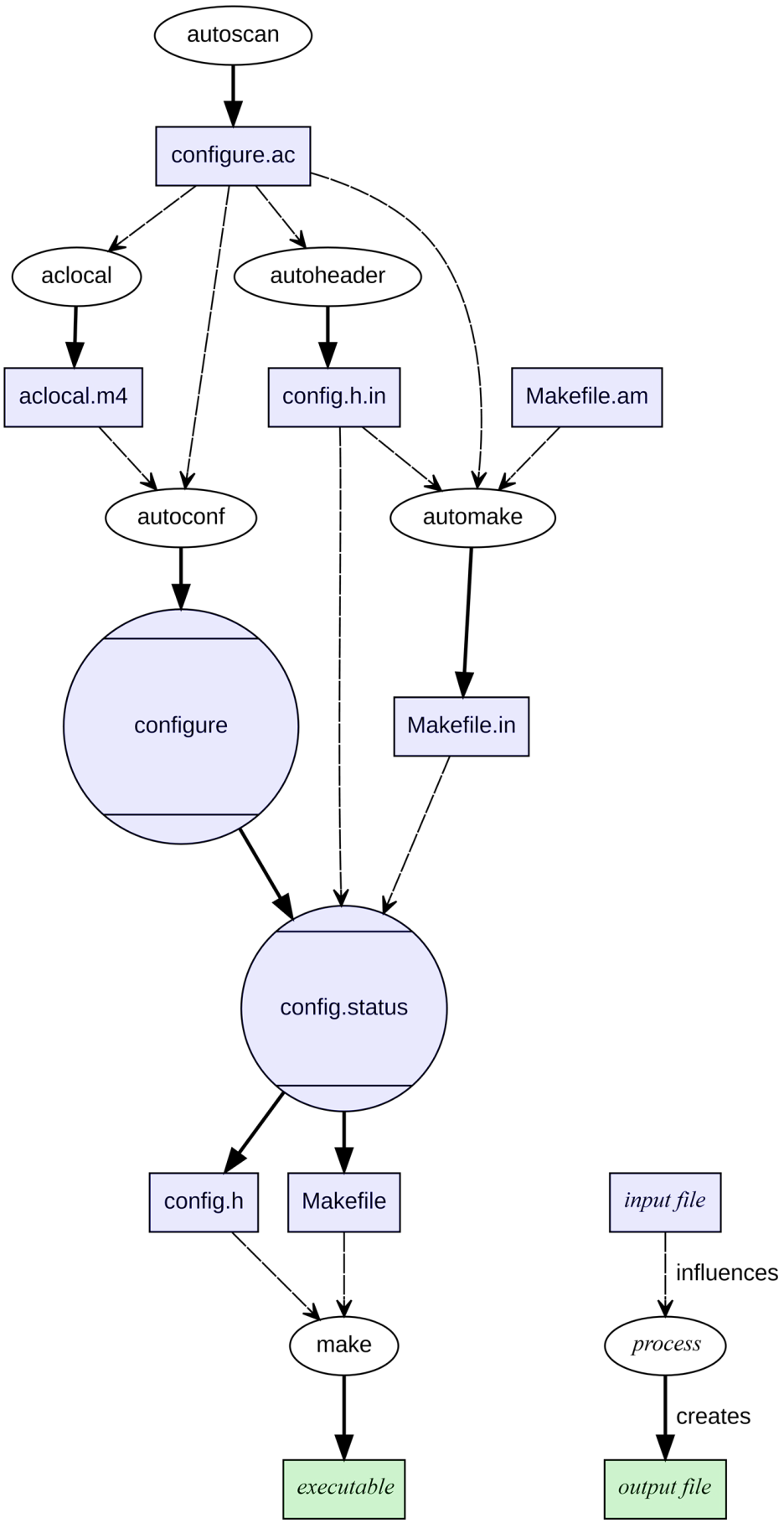


1. autoscan — генерация профиля (отыскивает возможные зависимости и подставляет их в профиль);
2. autoconf — генерация конфигурационного скрипта (configure);
3. autoheader — генерация прототипа общего заголовочного файла config.h.in;
4. automake — формирование Makefile.in по Makefile.am;
5. aclocal — переопределение процедур поиска и инструментов automake для конкретной системы;
6. conf — последовательность вызова утилит сборки;
7. порождение Makefile сценарием-генератом configure;
8. make — создание исполняемого файла;
9. при необходимости возможна сборка и использование библиотек при помощи *libtool*[56], а также использование других надстроек.

55 Autotools FAQ [Электронный ресурс]. URL: https://www.gnu.org/software/automake/faq/autotools-faq.html (дата обращения: 20.01.2025).

56 GNU Libtool - The GNU Portable Library Tool [Электронный ресурс]. URL: https://www.gnu.org/software/libtool/ (дата обращения: 20.01.2025).

На рисунке[57] показана диаграмма применения инструментов Autotools.



57 Wikipedia: Automake [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Automake (дата обращения: 20.01.2025).

13.2 Пример сборки проекта

Тестовым файлом . является простой код на , который отображает код введенной клавиши на экране:

```
// key.c

#include <ncurses.h>
#include <unistd.h>
#include <error.h>

#ifndef DX
#define DX 3
#endif

int main(void) {
        WINDOW *win;
        int c = 0;

        if (!isatty(0))
                error(1, 0, "Not a terminal");
        initscr();
        noecho();
        cbreak();
        printw("Window:");
        refresh();

        win = newwin(LINES-2*DX, COLS-2*DX, DX, DX);
#ifdef KEYPAD
        keypad(win, TRUE);
#else
        keypad(win, FALSE);
#endif
        scrollok (win, TRUE);
        do {
                wprintw(win, "  Key: %d, Name: %s\n", c, keyname(c));
                box(win, 0, 0);
                wrefresh(win);
        } while((c = wgetch(win)) != 27);

        endwin
return
}
```

13.3 Autoscan

Запуск autoscan приводит к формированию двух новых файлов:

```
[alt@localhost 07_env]$ autoscan .
[alt@localhost 07_env]$ ls
autoscan-default.log  configure.scan  key  key.c
```

13.4 Autoconf и configure

Содержимое файла configure.scan содержит список необходимых макросов для скрипта configure и используется как заготовка для файла configure.ac.

Основные макросы, которые могут присутствовать в этом файле:

1. AC_PREREQ — указывает версию autoconf;
2. AC_INIT — в качестве аргументов принимает название программы, её версию и, как необязательный аргумент, email-адрес, куда пользователи смогут отправлять отчёты об ошибках;
3. AC_CONFIG_SRCDIR — принимает в качестве параметра путь к файлу, с которого начинается сборка;
4. AC_CONFIG_HEADERS — создаёт соответствующий заголовочный файл;
5. AC_PROG_CC — ищет компилятор для языка и выставляет переменную CC соответствующим образом;
6. AC_OUTPUT — обязательный макрос, выполняет все действия.

13.5 Autoheader

Это команда Linux, которая генерирует шаблонный заголовочный файл (обычно с именем config.h.in) для использования скриптом configure во время сборки программного обеспечения. Программа сканирует файл configure.ac и находит символы препроцессора C, которые может потребоваться определить. Файл config.h в Linux содержит определения некоторых опций конфигурации (необходимо убрать комментарии у соответствующей строки):

13.6 Config.h

Использование config.h в файле key.c:

```
// key.c

#include "config.h"

printw("Window:");
printw("%s (%s)", PACKAGE_STRING, PACKAGE_BUGREPORT);
```

13.7 Automake

Automake[58] — утилита для автоматической генерации файлов Makefile.in из файлов Makefile.am. Каждый файл Makefile.am фактически является набором макросов для программы make (иногда с несколькими правилами). Полученные таким образом файлы Makefile.in соответствуют стандартам GNU Makefile. Основное назначение Automake в том, чтобы снять с разработчика программ бремя сопровождения нескольких версий Makefile для разных вариантов UNIX-подобных операционных систем.

58 Automake home page [Электронный ресурс]. URL: https://www.gnu.org/software/automake/ (дата обращения: 20.01.2025).

13.8 Autoreconf

Autoreconf[59] — это команда из пакета инструментов системной конфигурации GNU Autoconf, которая автоматически генерирует сценарии конфигурации для исходного кода на основе требований проекта. В частности, autoreconf проверяет код и генерирует необходимые сценарии configure, файлы Makefile.in и другие файлы конфигурации, нужные для правильной компиляции приложения в определённой системе.

13.9 Тестирование

По умолчанию цель пустая:

```
alt@localhost ~/Lin24/07_env $ make check
```

В Makefile.am есть возможность добавления процедур тестирования: в данном случае создается тестовый файл notatty.sh, который запускает файл key с перенаправлением стандартного ввода из /dev/null, перенаправляет стандартный поток ошибок на стандартный вывод и выводит сообщение ”./key: Not a terminal”, если стандартный вывод перенаправлен:

13.10 Сборка в отдельном дереве

Сборка в отдельном дереве каталогов (out of tree) — это процесс, при котором все выходные данные размещаются в отдельном от исходных файлов каталоге (или дереве каталогов). Особенно это удобно, когда впоследствии потребуется удалять все генераты — это можно сделать, удалив содержащий их каталог.

13.11 Pkg-config

Pkg-config[60] — это программа для получения информации об установленных библиотеках в Linux, которая используется при компиляции программного обеспечения, зависящего от них. Она позволяет программистам и сценариям установки работать без явного знания путей к библиотекам.

```
[alt@localhost 07_env]$ pkg-config --libs tinfo
-ltinfo
```

13.12 Cmake

CMake[61] — кроссплатформенное программное средство автоматизации сборки программного обеспечения из исходного кода. Не занимается непосредственно сборкой, а лишь генерирует файлы сборки из предварительно написанного файла сценария

59 Using autoreconf to Update configure Scripts [Электронный ресурс]. URL: https://www.gnu.org/software/autoconf/manual/autoconf-2.71/html_node/autoreconf-Invocation.html (дата обращения: 20.01.2025).

60 Pkg-config home page [Электронный ресурс]. URL: http://www.freedesktop.org/wiki/Software/pkg-config (дата обращения: 20.01.2025).

CMakeLists.txt и предоставляет простой единый интерфейс управления. Помимо этого, способно автоматизировать процесс установки и сборки пакетов.

13.13 Meson и Ninja

Meson[62]— это кроссплатформенная консольная утилита, автоматизирующая процесс сборки программного обеспечения из исходного кода (аналог autotools). Имеет открытый исходный код и распространяется под лицензией Apache.

Ninja[63] — это минималистичный инструмент, простой и мощный, ориентированный на скорость сборки (аналог Make) с двумя основными особенностями:

1. входные файлы генерируются системой сборки более высокого уровня;
2. сборки выполняются как можно быстрее.

В отличие от Make, в Ninja отсутствуют такие функции, как манипулирование строками, поскольку файлы сборки Ninja не предназначены для записи вручную. Вместо этого для генерации файлов сборки Ninja следует использовать «генератор сборки».

Meson использует Ninja в качестве основной системы сборки, но также может использовать Microsoft Visual Studio или Xcode. После первичной инициализации после любого изменения кода достаточно набрать ninja для пересборки проекта. При этом пересоберутся только изменённая и зависимые от неё части кода.

13.14 Домашнее задание

Данное задание можно выполнять с помощью Autotools, а можно — с помощью других инструментариев (CMake, Meson и т. п.).

Предварительно необходимо изучить:

1. простую *методичку*[64] по autotols;
2. *пример*[65] к лекции;
3. *слайды*[66] по autotools;
4. дополнительные материалы: *Autotools Mythbuster*[67], *Autotools Tutorial*[68], *AutoBook*[69];
5. либо аналогичные материалы по CMake, Meson и т. п.

61 CMake: A Powerful Software Build System [Электронный ресурс]. URL: https://cmake.org (дата обращения: 20.01.2025).

62 The Meson Build system [Электронный ресурс]. URL: https://mesonbuild.com/ (дата обращения: 20.01.2025).

63 Ninja home page [Электронный ресурс]. URL: https://ninja-build.org/ (дата обращения: 20.01.2025).

64 Основы сборки проектов при помощи Autotools [Электронный ресурс]. URL: https://eax.me/autotools/ (дата обращения: 20.01.2025).

65 Пример по Autotools [Электронный ресурс]. URL: https://git.sr.ht/~frbrgeorge/AutotoolsExample/log (дата обращения: 20.01.2025).

66 Autotools: a tutorial [Электронный ресурс]. URL: https://pdfslide.net/documents/autotools-a-tutorial-linux-conference-2016-gnu-autotools-a-tutorial-free-electrons.html (дата обращения: 20.01.2025).

67 Autotools Mythbuster [Электронный ресурс]. URL: https://autotools.io/index.html (дата обращения: 20.01.2025).

68 Autotools Tutorial [Электронный ресурс]. URL: https://www.lrde.epita.fr/~adl/autotools.html (дата обращения: 20.01.2025).

Основная цель задания — это создание простой утилиты для подсчёта хешей.

1. Изучить работу библиотеки librhash, разобравшись с:

- вычислением хеша для строки и файла;
- выводом хеша в шестнадцатеричном формате и в Base64.

2. Установить в сборочное окружение devel-версию библиотеки librhash из пакета согласно правилам используемого дистрибутива.

3. Написать программу rhasher.c, организующую простейшую командную строку (REPL):

- первое слово командной строки — имя алгоритма хеша;
- второе слово — либо имя файла, либо строка (строка начинается с «"», файл — нет): для простоты предполагается, что оно не содержит пробелов, и все последующие слова игнорируются, например, можно использовать *strtok*[70];
- в результате работы команды на stdout выводится соответствующий хеш этого файла или строки, причём должны поддерживаться как минимум MD5, SHA1 и TTH; если имя хеша начиналось с большой буквы, то в шестнадцатеричном виде; а если с маленькой — то в Base64 (в примерах есть Base32);
- если команда не опознана или возникает какая-либо ошибка с файлом, выводить на stderr сообщение и продолжать работу;
- интерфейс командной строки можно сделать довольно простым, достаточно getline;
- конец ввода — Ctrl+D.

4. Изучить работу *libreadline*[71] для модифицирования rhasher так, чтобы при сборке можно было использовать и getline(), и readline в зависимости от некоторого #define.

5. Написать профиль (Makefile.am + configure.ac или аналог) для сборки проекта:

- предусмотреть проверку наличия обеих библиотек (rhash обязательна, readline опциональна);
- предусмотреть принудительную сборку без readline, даже если он есть;
- в качестве примера можно использовать упрощенный *вариант*[72] из документации, который довольно сложен, например использует ncurses;
- предусмотреть простейшие тесты, как минимум, сравнение с выводом утилит sha1sum и md5sum;
- возможно сделать ввод не с терминала, не поддерживающий вывод подсказки, а также включить в тест вывод readline();
- предусмотреть полное удаление генератов.

6. Создать в репозитории с домашними заданиями подкаталог 07_Environmental и разместить в нем проект.

---

69 GNU Autoconf, Automake and Libtool [Электронный ресурс]. URL: https://sourceware.org/autobook/ (дата обращения: 20.01.2025).

70 strtok(3) — Linux manual page [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man3/strtok.3.html (дата обращения: 20.01.2025).

71 GNU Readline Library [Электронный ресурс]. URL: https://tiswww.case.edu/php/chet/readline/readline.html (дата обращения: 20.01.2025).

72 Working With External Software [Электронный ресурс]. URL: https://www.gnu.org/software/autoconf/manual/autoconf-2.60/html_node/External-Software.html (дата обращения: 20.01.2025).

14 Интернационализация и локализация

Идентификатор темы для сайта: 08_I18n

Рассмотрим неотъемлемую часть разработки международного продукта или адаптации продукта к местному рынку — *интернационализацию* и *локализацию.* Эти слова принято сокращать до I18n и l10n.

14.1 Область применения

Определим, что представляют собой эти два понятия, где они используются и разработчикам, надо изменять для i18n и l10n проекта.

- Интернационализация (i18n) — приведение исходного кода в готовое для локализации состояние. I18n это не локальное преобразование продукта, а его адаптация для потенциального использования в любом языковом контексте. I18n проводится на начальных этапах разработки.
- Локализация (l10n) — адаптация культурных и языковых предпочтений ПО, т.е. переводы и другие преобразования продукта для локальной площадки.

Для понимания необходимости локализации рассмотрим несколько объектов ПО, которым она обычно необходима:

- Сообщения и тексты. При работе с текстами основной проблемой являются словоформы. Системы падежей и исключений их них — дело непростое, а вот множественные формы вполне поддаются описанию. В каждом языке свои правила преобразования форм в зависимости от количества, также этих множественных форм может быть разное количество в разных языках (например, две множественные формы в английском и три в русском).

- Интерфейсные элементы. Кроме перевода, нужно также учитывать формат информации и особенности интерфейса. Например, для языков, написание в которых ведётся справа налево, нужно также адаптировать вывод.
- Изображения. Кроме изображений с текстом, которые несомненно требуют локализации, разработчикам может понадобиться изменять культурную атрибуцию (например, дорожные знаки, страны-оригинала на знаки страны использования).
- Шрифты также надо адаптировать. Замена оригинальных шрифтов на доступные «по умолчанию» может значительно испортить внешний вид приложения.
- Документация, шаблоны документов и прочие «непрограммные» данные.

И этот список далеко не исчерпывающий.

В Linux множество параметров, чувствительных к локали, можно посмотреть через команду locale:

```
alt@localhost:~> locale
```

```
LANG=en_US.UTF-8 # Язык системы
LC_CTYPE=en_US.UTF-8
LC_NUMERIC="en_US.UTF-8"
LC_TIME="en_US.UTF-8" # Формат вывода времени
LC_COLLATE="en_US.UTF-8"
LC_MONETARY="en_US.UTF-8"
LC_MESSAGES="en_US.UTF-8" # Язык для вывода сообщений (то, с чем мы будем
     работать)
LC_PAPER="en_US.UTF-8"
LC_NAME="en_US.UTF-8"
LC_ADDRESS="en_US.UTF-8"
LC_TELEPHONE="en_US.UTF-8"
LC_MEASUREMENT="en_US.UTF-8"
LC_IDENTIFICATION="en_US.UTF-8"
LC_ALL=
alt@localhost:~>
```

В дальнейшем будем заниматься только переводами непосредственно текстов, в т.ч. множественными формами слов.

14.2 Gettext

Чтобы в системе работали различные локали, необходимо поставить пакет переводов и их поддержки в базовой библиотеке Си — GLibc. В Debian-based дистрибутивах этот пакет называется *locales-all*[73]. В дистрибутивах семейства Альт, пакет glibs-locales установлен по умолчанию.

Для наших задач воспользуемся набором инструментов GNU gettext, который автоматически будет контролировать выбор словоформ при переводах и помогать генерировать переводы похожих элементов.

Рассмотрим, как происходит локализация программ на примере. Локализуем примитивную программу, которая просто печатает строки:

```
// ahello.c
#include <stdio.h>

int main(void)
{
        int i;

        /* Simple text */
        puts("Hello World\n");

        for(i=0; i<5; i++)
                /* Loosy plural example */
                printf("Hello, %d World\n", i);

        return 0;
}
```

73 Пакет locales-all [Электронный ресурс]. URL: https://packages.debian.org/buster/locales-all (дата обращения: 20.01.2025).

Скомпилируем и запустим эту программу:

```
alt@localhost:~/08_I18n/Lection> cc ahello.c
alt@localhost:~/08_I18n/Lection> ./a.out
Hello World

Hello, 0 World
Hello, 1 World
Hello, 2 World
Hello, 3 World
Hello, 4 World
alt@localhost:~/08_I18n/Lection>
```

Добавим конфигурацию Autotools для сборки нашей программы:

```
# configure.ac
#                                               -*- Autoconf -*-
# Process this file with autoconf to produce a configure script.

AC_PREREQ([2.72])
AC_INIT([L10n gettext ex], [0.0], [alt])
AC_INIT_AUTOMAKE([foreign])
AC_CONFIG_SRCDIR([ahello.c])
AC_CONFIG_HEADERS([config.h])

# Checks for programs.
AC_PROG_CC

# Checks for libraries.
# Checks for header files.

# Checks for typedefs, structures, and compiler characteristics.

# Checks for library functions.

AC_CONFIG_FILES([Makefile])
AC_OUTPUT
```

```
# Makefile.am
bin_PROGRAMS=ahello
```

14.3 Создание перевода

Первый шаг — интернационализация. Необходимо подготовить программу к возможности перевода. Для этого все элементы, которые мы хотим локализовать, оборачиваем функциями из библиотеки locale.h, входящей в GNU gettext. Для краткости обозначим функцию-обёртку gettext через её классическое обозначение _.

Важный элемент интернационализации — *домены переводов*. Домен — это отдельное множество переведённых строк, изолированное от других переводов *в ту же локаль*. Домен необходим для разделения разных классов переводов в одном проекте.

Например, в игровом приложении могут встречаться переводы реплик игровых персонажей (например, средневековых пиратов), а могут — переводы сообщений для администраторов или ведущих игру.

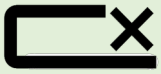
Добавим первую локализацию:

```
// ahello.c
#include <stdio.h>
#include <stdlib.h>
#include <libgen.h>
#include <libintl.h>
#include <locale.h>

#define _(STRING) gettext(STRING)
#define LOCALE_PATH "."

int main(int argc, char *argv[])
{
        int i;
        char *dir;

        dir = dirname(realpath(argv[0], NULL));
        setlocale (LC_ALL, "");
        bindtextdomain ("ahello", LOCALE_PATH); // имя домена выбираем сами
        textdomain ("ahello");

        /* Simple text */
        puts(_("Hello World\n"));

        for(i=0; i<5; i++)
                /* Loosy plural example */
                printf(_("Hello, %d World\n"), i);

        return 0;
}
```

Выполним сборку:

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  configure.ac  LICENSE  Makefile.am
alt@localhost:~/08_I18n/Lection> autoreconf -fivs
```

```
alt@localhost:~/08_I18n/Lection> ./configure
```



```
alt@localhost:~/08_I18n/Lection> make
```

```
alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 World
Hello, 1 World
```

```
Hello, 2 World
Hello, 3 World
Hello, 4 World
alt@localhost:~/08_I18n/Lection>
```

Посмотрим, что делает процесс при обработке программы:

```
Hello World

Hello, 0 World
Hello, 1 World
Hello, 2 World
Hello, 3 World
Hello, 4 World
alt@localhost:~/08_I18n/Lection> cat strace_data | grep -E "(LC)|(\.mo)"
openat(AT_FDCWD, "/usr/lib/locale/en_US.UTF-8/LC_IDENTIFICATION", O_RDONLY|
     O_CLOEXEC) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/usr/lib/locale/en_US.utf8/LC_IDENTIFICATION", O_RDONLY|
     O_CLOEXEC) = 3
```

```
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en_US.UTF-8/LC_MESSAGES/ahel-
     lo.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en_US.utf8/LC_MESSAGES/ahel-
     lo.mo", O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en_US/LC_MESSAGES/ahello.mo",
     O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en.UTF-8/LC_MESSAGES/ahello.mo",
     O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en.utf8/LC_MESSAGES/ahello.mo",
     O_RDONLY) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/home/alt/08_I18n/Lection/./en/LC_MESSAGES/ahello.mo",
     O_RDONLY) = -1 ENOENT (No such file or directory)
alt@localhost:~/08_I18n/Lection>
```

Происходит обращение процесса к .mo-файлам. Это те самые файлы переводов, которые создаются с помощью gettext по нашим переводам. Пришло время написать их.

Следующий шаг после интернационализации программы — *генерация общего шаблона* для поиска переводимых фраз. Она делается с помощью команды xgettext, которой мы указываем ключ (название нашей функции перевода), исходный объект и выходной шаблон. Принято файл входного шаблона называть <src_name>.pot, т. к. он будет шаблоном для файла переводов .po (pot = “po template”).

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  configure.ac  LICENSE  Makefile.am
alt@localhost:~/08_I18n/Lection> xgettext -k_ -c ahello.c -o ahello.pot
alt@localhost:~/08_I18n/Lection> ls
ahello.c  ahello.pot  configure.ac  LICENSE  Makefile.am
alt@localhost:~/08_I18n/Lection>
```

Сформирована заготовка для переводов:

```
# ahello.pot
```

```
# SOME DESCRIPTIVE TITLE.
# Copyright (C) YEAR THE PACKAGE'S COPYRIGHT HOLDER
# This file is distributed under the same license as the PACKAGE package.
# FIRST AUTHOR <alt>, YEAR.
#
#, fuzzy
msgid ""
msgstr ""
"Project-Id-Version: 0.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2024-11-05 17:03+0300\n"
"PO-Revision-Date: YEAR-MO-DA HO:MI+ZONE\n"
"Last-Translator: FULL NAME <EMAIL@ADDRESS>\n"
"Language-Team: LANGUAGE <LL@li.org>\n"
"Language: \n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=CHARSET\n"
"Content-Transfer-Encoding: 8bit\n"

#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr ""

#. Loosy plural example
#: ahello.c:25
#, c-format
msgid "Hello, %d World\n"
msgstr ""
```

В заготовке пока не определён язык перевода, правила множественного перевода, нет информации о переводчиках и т.д.

Следующий шаг локализации — *создание перевода*. Для этого используется команда msginit с указанием исходного шаблона, выходного файла переводов и локали перевода.

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  ahello.pot  configure.ac  LICENSE  Makefile.am
alt@localhost:~/08_I18n/Lection> msginit -i ahello.pot -o ru.po -l ru_RU.UTF-8
The new message catalog should contain your email address, so that users can
give you feedback about the translations, and so that maintainers can contact
you in case of unexpected technical problems.

Is the following your email address?
  alt@localhost
Please confirm by pressing Return, or enter your email address.

Retrieving https://translationproject.org/team/index.html... done.
Please visit your translation team's homepage at
  https://translationproject.org/team/ru.html
  https://translationproject.org/team/index.html
```

```
  https://translationproject.org/html/translators.html
  https://translationproject.org/html/welcome.html
and consider joining your translation team's mailing list
  <gnu@d07.ru>

Created ru.po.
alt@localhost:~/08_I18n/Lection> ls
ahello.c  ahello.pot  configure.ac  LICENSE  Makefile.am  ru.po
alt@localhost:~/08_I18n/Lection>
```

Шаблон перевода (файл .pot) нужен для первоначального создания файлов с переводами на разные языки и для последующего их обновления. В процессе собственно перевода и компиляции он не участвует.

```
# ru.po
# Russian translations for 0.0 package
# Английские переводы для пакета 0.0.
# Copyright (C) 2024 THE 0.0'S COPYRIGHT HOLDER
# This file is distributed under the same license as the 0.0 package.
# FIRST AUTHOR <alt>, 2024.
#
msgid ""
msgstr ""
"Project-Id-Version: 0.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2024-11-05 17:03+0300\n"
"PO-Revision-Date: 2024-11-05 17:07+0300\n"
"Last-Translator: alt <alt@localhost>\n"
"Language-Team: Russian <gnu@d07.ru>\n"
"Language: ru\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"
"Plural-Forms: nplurals=3; plural=(n%10==1 && n%100!=11 ? 0 : n%10>=2 && "
"n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2);\n"

#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr ""

#. Loosy plural example
#: ahello.c:25
#, c-format
msgid "Hello, %d World\n"
msgstr ""
```

Файл приобрёл начальное наполнение — добавились правила вывода множественного числа, явно указан язык перевода. Однако самих переводов всё ещё нет:

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  configure.ac  LICENSE  Makefile.am  ru.po
alt@localhost:~/08_I18n/Lection> autoreconf -fivs
```

```
alt@localhost:~/08_I18n/Lection> ./configure
alt@localhost:~/08_I18n/Lection> make
alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 World
Hello, 1 World
Hello, 2 World
Hello, 3 World
Hello, 4 World
alt@localhost:~/08_I18n/Lection>
```

Добавим переводы:

```
# ru.po
# Russian translations for 0.0 package
# Английские переводы для пакета 0.0.
# Copyright (C) 2024 THE 0.0'S COPYRIGHT HOLDER
# This file is distributed under the same license as the 0.0 package.
# FIRST AUTHOR <alt>, 2024.
#
msgid ""
msgstr ""
"Project-Id-Version: 0.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2024-11-05 17:03+0300\n"
"PO-Revision-Date: 2024-11-05 17:07+0300\n"
"Last-Translator: alt <alt@localhost>\n"
"Language-Team: Russian <gnu@d07.ru>\n"
"Language: ru\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"
"Plural-Forms: nplurals=3; plural=(n%10==1 && n%100!=11 ? 0 : n%10>=2 && "
"n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2);\n"

#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr "Привет, мир\n"

#. Loosy plural example
#: ahello.c:25
#, c-format
msgid "Hello, %d World\n"
msgstr "Привет, %d миров\n"
```

Последний шаг для локализации — *компиляция перевода*. Именно на этом шаге формируются .mo-файлы, к которым обращается процесс. При этом (см. вывод

strace) нужно расположить эти файлы в правильном месте — каталоге <LOCALE_PATH>/<язык перевода>/LC_MESSAGES. Поскольку LOCALE_PATH в программе определён как ".", переводы надо класть в текущий каталог.

За компиляцию перевода отвечает msgfmt.

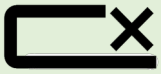
Добавим в Makefile.am отдельный рецепт по компиляции перевода:

```
# Makefile.am
ru/LC_MESSAGES/ahello.mo: ru.po
        mkdir -p `dirname $@`
        msgfmt $< -o $@
```

Выполним сборку с подключением перевода:

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  configure.ac  LICENSE  Makefile.am  ru.po

alt@localhost:~/08_I18n/Lection> autoreconf -fivs
```

```
alt@localhost:~/08_I18n/Lection> ./configure
```



```
alt@localhost:~/08_I18n/Lection> make
```

```
alt@localhost:~/08_I18n/Lection> make ru/LC_MESSAGES/ahello.mo
mkdir -p `dirname ru/LC_MESSAGES/ahello.mo`
msgfmt ru.po -o ru/LC_MESSAGES/ahello.mo

alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 World
Hello, 1 World
Hello, 2 World
Hello, 3 World
Hello, 4 World
alt@localhost:~/08_I18n/Lection> LC_ALL=ru_RU.UTF8 ./ahello
Привет, мир

Привет, 0 миров
Привет, 1 миров
Привет, 2 миров
Привет, 3 миров
Привет, 4 миров
alt@localhost:~/08_I18n/Lection>
```

14.4 Обновление перевода

Поменяем исходник и обновим уже существующий перевод:

```
// ahello.c
#include <stdio.h>
#include <stdlib.h>
#include <libgen.h>
#include <libintl.h>
#include <locale.h>

#define _(STRING) gettext(STRING)
#define LOCALE_PATH "."

int main(int argc, char *argv[])
{
        int i;
        char *dir;

        dir = dirname(realpath(argv[0], NULL));
        setlocale (LC_ALL, "");
        bindtextdomain ("ahello", LOCALE_PATH);
        textdomain ("ahello");

        /* Simple text */
        puts(_("Hello World\n"));

        for(i=0; i<5; i++)
                /* Loosy plural example */
                printf(_("Hello, %d Worlds\n"), i); // Поменяли

        return 0;
}
```

Для обновления перевода нужно вновь собрать переводимые фразы в шаблон с помощью xgettext, и после этого соединить их с уже существующим файлом перевода с помощью msgmerge. При этом:

- Сохранятся не изменившиеся переводы, они будут перенесены в новый файл переводов;
- Автоматически для фраз, *похожих* на уже существовавшие в переводах, сгенерируется «нечёткий» перевод (помеченный значащим комментарием # fuzzy).

```
alt@localhost:~/08_I18n/Lection> ls
ahello.c  configure.ac  LICENSE  Makefile.am  ru.po
alt@localhost:~/08_I18n/Lection> xgettext -k_ -c ahello.c -o ahello.pot
alt@localhost:~/08_I18n/Lection> msgmerge -U ru.po ahello.pot
.... done.
alt@localhost:~/08_I18n/Lection>
```

```
# ru.po
# Russian translations for 0.0 package
# Английские переводы для пакета 0.0.
# Copyright (C) 2024 THE 0.0'S COPYRIGHT HOLDER
# This file is distributed under the same license as the 0.0 package.
# FIRST AUTHOR <alt>, 2024.
```

```
#
msgid ""
msgstr ""
"Project-Id-Version: 0.0\n"
"Report-Msgid-Bugs-To: \n"
"POT-Creation-Date: 2024-11-05 17:27+0300\n"
"PO-Revision-Date: 2024-11-05 17:07+0300\n"
"Last-Translator: alt <alt@localhost>\n"
"Language-Team: Russian <gnu@d07.ru>\n"
"Language: ru\n"
"MIME-Version: 1.0\n"
"Content-Type: text/plain; charset=UTF-8\n"
"Content-Transfer-Encoding: 8bit\n"
"Plural-Forms: nplurals=3; plural=(n%10==1 && n%100!=11 ? 0 : n%10>=2 && "
"n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2);\n"

#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr "Привет, мир\n"

#. Loosy plural example
#: ahello.c:25
#, fuzzy, c-format
msgid "Hello, %d Worlds\n"
msgstr "Привет, %d миров\n"
```

Fuzzy-перевод вполне удачен, но из-за пометки ”fuzzy” не будет использоваться:

```
alt@localhost:~/08_I18n/Lection> autoreconf -fivs
```

```
alt@localhost:~/08_I18n/Lection> ./configure
```

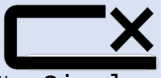
```
alt@localhost:~/08_I18n/Lection> make
```

```
alt@localhost:~/08_I18n/Lection> make ru/LC_MESSAGES/ahello.mo
mkdir -p `dirname ru/LC_MESSAGES/ahello.mo`
msgfmt ru.po -o ru/LC_MESSAGES/ahello.mo

alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 Worlds
Hello, 1 Worlds
Hello, 2 Worlds
Hello, 3 Worlds
Hello, 4 Worlds
alt@localhost:~/08_I18n/Lection> LC_ALL=ru_RU.UTF8 ./ahello
Привет, мир
```

```
Hello, 0 Worlds
Hello, 1 Worlds
Hello, 2 Worlds
Hello, 3 Worlds
Hello, 4 Worlds
alt@localhost:~/08_I18n/Lection>
```

После удаления метки “fuzzy” подключится перевод для множественного числа:

```
# ru.po
```



```
#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr "Привет, мир\n"

#. Loosy plural example
#: ahello.c:25
#, c-format
msgid "Hello, %d Worlds\n"
msgstr "Привет, %d миров\n"
```

```
alt@localhost:~/08_I18n/Lection> make ru/LC_MESSAGES/ahello.mo
mkdir -p `dirname ru/LC_MESSAGES/ahello.mo`
msgfmt ru.po -o ru/LC_MESSAGES/ahello.mo

alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 Worlds
Hello, 1 Worlds
Hello, 2 Worlds
Hello, 3 Worlds
Hello, 4 Worlds
alt@localhost:~/08_I18n/Lection> LC_ALL=ru_RU.UTF8 ./ahello
Привет, мир

Привет, 0 миров
Привет, 1 миров
Привет, 2 миров
Привет, 3 миров
Привет, 4 миров
alt@localhost:~/08_I18n/Lection>
```

Опишем зависимость нашей программы от файла переводов для автоматического включения этой зависимости в Makefile:

```
# Makefile.am
bin_PROGRAMS=ahello
pkgdata_DATA=ru/LC_MESSAGES/ahello.mo

ru/LC_MESSAGES/ahello.mo: ru.po
```

```
        mkdir -p `dirname $@`
        msgfmt $< -o $@

ru.po:  ahello.pot
        msgmerge -U $@ $<

ahello.pot: ahello.c
        xgettext -k_ -c ahello.c -o ahello.pot
```

14.5 Различные формы множественного числа

В английском языке множественная форма одна, а в русском — три. Для учета этого при переводе, в .po-файле используется тернарный оператор для выбора формы.

```
// ahello.c
#include <stdio.h>
#include <stdlib.h>
#include <libgen.h>
#include <libintl.h>
#include <locale.h>

#define _(STRING) gettext(STRING)
#define LOCALE_PATH "."

int main(int argc, char *argv[])
{
        int i;
        char *dir;

        dir = dirname(realpath(argv[0], NULL));
        setlocale (LC_ALL, "");
        bindtextdomain ("ahello", LOCALE_PATH);
        textdomain ("ahello");

        /* Simple text */
        puts(_("Hello World\n"));

        for(i=0; i<6; i++)
                /* Plural example */
                printf(ngettext("Hello, %d World\n", "Hello, %d Worlds\n", i),
i);

        return 0;
}
```

```
alt@localhost:~/08_I18n/Lection> xgettext -k_ -c ahello.c -o ahello.pot
alt@localhost:~/08_I18n/Lection> msgmerge -U ru.po ahello.pot
.... done.
alt@localhost:~/08_I18n/Lection>
```

```
# ru.po
"Plural-Forms: nplurals=3; plural=(n%10==1 && n%100!=11 ? 0 : n%10>=2 && "
"n%10<=4 && (n%100<10 || n%100>=20) ? 1 : 2);\n"

#. Simple text
#: ahello.c:21
msgid "Hello World\n"
msgstr "Привет, мир\n"

#. Plural example
#: ahello.c:25
#, c-format
msgid "Hello, %d World\n"
msgid_plural "Hello, %d Worlds\n"
msgstr[0] "Привет, %d мир\n"
msgstr[1] "Привет, %d мира\n"
msgstr[2] "Привет, %d миров\n"
```

Сборка и запуск программы с подключенным переводом:

```
alt@localhost:~/08_I18n/Lection> autoreconf -fivs
alt@localhost:~/08_I18n/Lection> ./configure
alt@localhost:~/08_I18n/Lection> make
alt@localhost:~/08_I18n/Lection> ./ahello
Hello World

Hello, 0 Worlds
Hello, 1 World
Hello, 2 Worlds
Hello, 3 Worlds
Hello, 4 Worlds
Hello, 5 Worlds
alt@localhost:~/08_I18n/Lection> LC_ALL=ru_RU.UTF8 ./ahello
Привет, мир

Привет, 0 миров
Привет, 1 мир
Привет, 2 мира
Привет, 3 мира
Привет, 4 мира
Привет, 5 миров
alt@localhost:~/08_I18n/Lection>
```

14.6 Раздельное хранение исходных текстов и переводов

Для сложных проектов рекомендуется разделять места хранения исходных текстов и переводов к ним. Покажем это на примере, заодно разделив на части Makefile.am.

```
alt@localhost:~/08_I18n/Lection> tree
.
├── configure.ac
├── LICENSE
├── Makefile.am
├── po
│   ├── Makefile.am
│   └── ru.po
└── src
    ├── ahello.c
    └── Makefile.am

3 directories, 7 files
```

```
# Makefile.am
SUBDIRS = src po
```

```
# src/Makefile.am
bin_PROGRAMS=ahello
```

```
# po/Makefile.am
RU = ru/LC_MESSAGES
PACKAGE = ahello

pkgdata_DATA = $(RU)/$(PACKAGE).mo

$(RU)/$(PACKAGE).mo: ru.po
        mkdir -p `dirname $@`
        msgfmt $< -o $@

$(PACKAGE).pot: ../src/$(PACKAGE).c
        xgettext -k_ -c $< -o $@

ru.po:  $(PACKAGE).pot
        msgmerge -U $@ $<
```

```
# configure.ac
#                                               -*- Autoconf -*-
# Process this file with autoconf to produce a configure script.

AC_PREREQ([2.72])
AC_INIT([L10n gettext ex], [0.0], [alt])
```

```
AM_INIT_AUTOMAKE([foreign])
AC_CONFIG_SRCDIR([src/ahello.c])
AC_CONFIG_HEADERS([config.h])

# Checks for programs.
AC_PROG_CC

# Checks for libraries.

# Checks for header files.

# Checks for typedefs, structures, and compiler characteristics.

# Checks for library functions.

AC_CONFIG_FILES([Makefile po/Makefile src/Makefile])
AC_OUTPUT
```

14.7 Домашнее задание

1. Написать программу на Си, которая угадывает целое число от 1 до 100 методом деления отрезка пополам:
   - просит загадать число от 1 до 100;
   - спрашивает «число больше 50?» (ответ тоже надо переводить);
   - на основании ответа выбирает половину интервала и продолжает деление на нем;
   - когда догадается, какое число загадано — отвечает;
2. Оформить автосборку для этой программы (autotools, CMake, Meson и т. п.)
3. Добавить перевод;
4. Добавить:
   - либо поддержку перевода с помощью autotools;
   - либо автоматическое обновление/пересборку перевода и удаление генератов;
   - обратите внимание: .po-файл (а в примере до перехода на autotools ещё заголовочная часть .pot-файла) — *не генераты*; .mo, временный .pot и прочее — *генераты*;
5. Создать в репозитории с домашними заданиями подкаталог 08_I18n и разместить в нем проект.

15 Работа со сторонними исходными текстами

Идентификатор темы для сайта: 09_PatchDiff

Данная тема посвящена работе с проектами, в которых мы не являемся непосредственным разработчиком или не имеем возможности пользоваться общей системой контроля версий. В таких ситуациях на помощь в разработке приходят *патчи*.

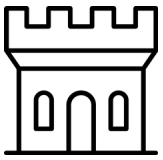
15.1 Патч и его место в процессе разработки

Представим ситуацию, когда мы — не разработчик программного продукта, но работаем с его исходниками, хранящимися где-то в открытом доступе. Бывает, что проект не работает «из коробки». Причин у этого может быть множество: проект изначально писался под какую-то определённую систему, не совпадающую с нашей; в нём присутствуют ошибки, связанные с обновлением каких-то зависимых элементов. Возможно, мы хотим что-то добавить в проект, например, локализацию под нашу площадку, однако разработчики не принимают наши изменения в основной код. В обоих случаях мы можем самостоятельно локально изменить исходники для наших нужд, но тогда непременно столкнёмся с ситуацией, когда у команды разработки продукта вышла новая версия без наших улучшений, и потребуется заново производить модификацию кода для внесения наших надстроек.

Чтобы избежать подобного бесконечного переписывания кода, можно использовать механизм *патчей*. Патч (patch — заплатка, накладка) — это текст специального формата, в котором описывается набор изменений для внесения в другой текст.

При использовании патчей работа со сторонними исходниками становится значительно легче. При создании первой версии изменений мы *исправляем исходники*, как нам необходимо; *оформляем* эти *исправления* в виде патчей; *сохраняем* этот *патч-сет*, а *исправленную версию используем* сами или выкладываем в открытый доступ в зависимости от нужд.
При обновлении исходников мы просто *применяем наши патчи к новой версии* (при этом часть может примениться сразу, а часть — оказаться неприменимой из-за изменений); *адаптируем оставшиеся патчи* для их применимости; *сохраняем новый патч-сет* и снова просто *пользуемся нашими надстройками*.

Работа с патчами не входит в базовый комплект умений программиста, однако на практике встречается очень часто. Кроме того, именно патчирование (с рассылкой патчей по электронной почте) было основным способом синхронизации при совместной разработке до массового использования систем контроля версий.



15.2 Технология патчирования

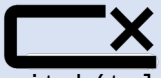
Разберёмся, на чём вообще основана технология создания патчей. Для начала нужно научиться находить различия в программах. Рассмотрим пример, в котором реа-

лизована функция открытия файла с помощью системного вызова open, но с параметром прав доступа из функции fopen:

```
// ffopen1.c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <ctype.h>
/* open()->fopen() wrapper */
static FILE *ffopen(const char *pathname, const char *mode, int flags)
{
  int m;
  switch(tolower(mode[0])+(mode[1]=='+')) {
    case 'b': /* "a+" */
    case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
    case 'r': m = O_RDONLY; break;
    case 's': m = O_RDWR; break; /* "r+" */
    case 'x': /* "w+" */
    case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
    default: m = O_RDONLY; break;
  }
  int fd = open(pathname, flags, m);
  return fd<0? NULL: fdopen(fd, mode);
}
int main(int argc, char *argv[]) {
        FILE *fp;
        fp = ffopen(argv[1], "r", O_NOFOLLOW);
        if(fp == NULL) {
          perror(argv[1]);
          return 1;
        }
        return 0;
}
```

Пусть необходимо изменить реализацию параметра x, который отвечает за открытие файла на чтение и запись одновременно:

```
// ffopen1.c
```



```
switch(tolower(mode[0])+(mode[1]=='+')) {
    case 'b': /* "a+" */
    case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
    case 'r': m = O_RDONLY; break;
    case 's': m = O_RDWR; break; /* "r+" */
    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */ /*Наше
изменение*/
    case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
    default: m = O_RDONLY; break;
  }
```

Чтобы определить, где и как изменился файл по сравнению с исходным вариантом, воспользуемся утилитой diff:

```
alt@localhost:~/09_PatchDiff/Lection> ls
ffopen.c  ffopen_new.c
alt@localhost:~/09_PatchDiff/Lection> diff ffopen.c ffopen_new.c
17c17
<     case 'x': /* "w+" */
---
>    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
```

Идентификатор темы для сайта: 10_LibTesting

В выдаче diff указано, что необходимо сделать с первым файлом, чтобы получить из него второй: удалить 17-ю строку, которая выглядит, как показано на экране, а вместо нее вставить новую (также указанную) строку.

Некоторые флаги diff позволяют получить эту же информацию в других форматах:

-e — в виде скрипта для текстового однострочного редактора ed:

```
alt@localhost:~/09_PatchDiff/Lection> diff -e ffopen.c ffopen_new.c
17c
    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
.
```

-c — для указания изменения вместе с контекстом, где оно проведено (несколько строк до и после):

```
 alt@localhost:~/09_PatchDiff/Lection> diff -c ffopen.c ffopen_new.c
*** ffopen.c    2024-11-12 16:32:10.603958261 +0300
--- ffopen_new.c        2024-11-12 16:32:23.247503694 +0300
***************
*** 14,20 ****
      case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
      case 'r': m = O_RDONLY; break;
      case 's': m = O_RDWR; break; /* "r+" */
!     case 'x': /* "w+" */
      case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
      default: m = O_RDONLY; break;
    }
--- 14,20 ----
      case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
      case 'r': m = O_RDONLY; break;
      case 's': m = O_RDWR; break; /* "r+" */
!     case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
      case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
      default: m = O_RDONLY; break;
    }
```

-u — для показа изменений с однократным отображением контекста:

```
 alt@localhost:~/09_PatchDiff/Lection> diff -u ffopen.c ffopen_new.c
--- ffopen.c    2024-11-12 16:32:10.603958261 +0300
+++ ffopen_new.c        2024-11-12 16:32:23.247503694 +0300
@@ -14,7 +14,7 @@ # Наш контекст из 7 строк от 14ой меняем на 7 строк от 14
     вот так
     case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
```

```
     case 'r': m = O_RDONLY; break;
     case 's': m = O_RDWR; break; /* "r+" */
-    case 'x': /* "w+" */ # Было
+    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */ # Стало
     case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
     default: m = O_RDONLY; break;
   }
```

Добавим ещё одно изменение в программу:

```
// ffopen_new2.c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <ctype.h>
/* open()->fopen() wrapper */
static FILE *ffopen(const char *pathname, const char *mode, int flags)
{
  int m;
  switch(tolower(mode[0])+(mode[1]=='+')) {
    case 'b': /* "a+" */
    case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
    case 'r': m = O_RDONLY; break;
    case 's': m = O_RDWR; break; /* "r+" */
    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
    case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
    default: m = O_RDONLY; break;
  }
  int fd = open(pathname, flags, m);
  return fd<0? NULL: fdopen(fd, mode);
}
int main(int argc, char *argv[]) {
        FILE *fp;
        fp = ffopen(argv[1], "r", O_NOFOLLOW);
        if(fp == NULL) {
          perror(argv[1]);
          return 1;
        }
        else
                fclose(fp);
        return 0;
}
```

Визуально это изменение сложно обнаружить. Необходимо использовать утилиту diff:

```
alt@localhost:~/09_PatchDiff/Lection> diff -u ffopen.c ffopen_new2.c
--- ffopen.c    2024-11-12 16:32:10.603958261 +0300
+++ ffopen_new2.c       2024-11-12 16:44:50.173769006 +0300
@@ -14,7 +14,7 @@
     case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
     case 'r': m = O_RDONLY; break;
     case 's': m = O_RDWR; break; /* "r+" */
-    case 'x': /* "w+" */
```

```
+    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
     case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
     default: m = O_RDONLY; break;
   }
@@ -31,6 +31,7 @@
           perror(argv[1]);
           return 1;
         }
-
+        else
+          fclose(fp);
         return 0;
 }
```

Diff вывел изменения и для каждого из них указал блок контекста. Формат вывода diff -u это и есть формат оформления патчей. Поэтому, сохранив его в виде файла, можно делиться патчем и использовать его для преобразования исходных кодов.

```
alt@localhost:~/09_PatchDiff/Lection> diff -u ffopen.c ffopen_new.c > p0.patch
alt@localhost:~/09_PatchDiff/Lection> ls
a.out  ffopen.c  ffopen_new2.c  ffopen_new.c  p0.patch
alt@localhost:~/09_PatchDiff/Lection> cat p0.patch
--- ffopen.c    2024-11-12 16:32:10.603958261 +0300
+++ ffopen_new.c        2024-11-12 16:32:23.247503694 +0300
@@ -14,7 +14,7 @@
     case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
     case 'r': m = O_RDONLY; break;
     case 's': m = O_RDWR; break; /* "r+" */
-    case 'x': /* "w+" */
+    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
     case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
     default: m = O_RDONLY; break;
   }
```

```
alt@localhost:~/09_PatchDiff/Lection> diff -u ffopen_new.c ffopen_new2.c >
     p1.patch
alt@localhost:~/09_PatchDiff/Lection> ls
a.out  ffopen.c  ffopen_new2.c  ffopen_new.c  p0.patch  p1.patch
alt@localhost:~/09_PatchDiff/Lection> cat p1.patch
--- ffopen_new.c        2024-11-12 16:32:23.247503694 +0300
+++ ffopen_new2.c       2024-11-12 16:44:50.173769006 +0300
@@ -31,6 +31,7 @@
           perror(argv[1]);
           return 1;
         }
-
+        else
+          fclose(fp);
```

```
        return 0;
 }
```

Утилита patch применяет изменения к файлу:

```
alt@localhost:~/09_PatchDiff/Lection> cp ffopen.c ffopen2.c
alt@localhost:~/09_PatchDiff/Lection> patch ffopen2.c < p0.patch
patching file ffopen2.c
alt@localhost:~/09_PatchDiff/Lection> diff ffopen2.c ffopen_new.c
alt@localhost:~/09_PatchDiff/Lection> patch ffopen2.c < p1.patch
patching file ffopen2.c
alt@localhost:~/09_PatchDiff/Lection> diff ffopen2.c ffopen_new2.c
alt@localhost:~/09_PatchDiff/Lection>
```

После применения патчей ffopen2.c стал неотличим от ffopen_new2.c. Наши патчи работают с разными контекстами, поэтому порядок их применения не так важен. Но нередкой бывает работа с патчами, основанными на одном контексте, для которых порядок применения будет играть роль.

15.3 Неточный контекст

Поскольку накладывать патчи приходится на обновляющуюся версию исходников, надо разобрать случай, когда контекст изменяется. Возьмём те же самые патчи и применим их к слегка изменённой программе. При этом одно изменение будет попадать в контекст, а другое — нет.

```
// ffopen3.c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <ctype.h>

/* open()->fopen() wrapper */
static FILE *ffopen(const char *pathname, const char *mode, int flags)
{
  int m;

  switch(tolower(mode[0])+(mode[1]=='+')) {
    case 'b': /* "a+" */
    case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
    case 'r': m = O_RDONLY; break;
    // TODO "b" modifier
    case 's': m = O_RDWR; break; /* "r+" */
    case 'x': /* "w+" */
    case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
    default: m = O_RDONLY; break;
  }

  int fd = open(pathname, flags, m);
  return fd<0? NULL: fdopen(fd, mode);
}
```

```
/* just for test purpose */
int main(int argc, char *argv[]) {
        FILE *fp;

        fp = ffopen(argv[1], "r", O_NOFOLLOW);
        if(fp == NULL) {
          perror(argv[1]);
          return 1;
        }

        return 0;
}
```

Объединим патчи:

```
alt@localhost:~/09_PatchDiff/Lection> cat p*.patch > pall.patch
alt@localhost:~/09_PatchDiff/Lection> cat pall.patch
--- ffopen.c    2024-11-12 16:32:10.603958261 +0300
+++ ffopen_new2.c       2024-11-12 16:44:50.173769006 +0300
@@ -14,7 +14,7 @@
     case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
     case 'r': m = O_RDONLY; break;
     case 's': m = O_RDWR; break; /* "r+" */
-    case 'x': /* "w+" */
+    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
     case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
     default: m = O_RDONLY; break;
   }
@@ -31,6 +31,7 @@
           perror(argv[1]);
           return 1;
         }
-
+        else
+          fclose(fp);
         return 0;
 }
```

Запустим применение патчей с флагом --verbose, чтобы получить дополнительную информацию по применению:

```
alt@localhost:~/09_PatchDiff/Lection> patch --verbose ffopen3.c < pall.patch
Hmm...  Looks like a unified diff to me...
The text leading up to this was:
--------------------------
|--- ffopen.c   2024-11-12 16:32:10.603958261 +0300
|+++ ffopen_new2.c      2024-11-12 16:44:50.173769006 +0300
--------------------------
patching file ffopen3.c
Using Plan A...
Hunk #1 succeeded at 15 with fuzz 2 (offset 1 line).
Hunk #2 succeeded at 33 with fuzz 2 (offset 2 lines).
done
```

Когда программа говорит "Hmm...", это не очень хорошо, но в данном случае это лишь предупреждение. Патчи применились корректно.

Из диагностической выдачи видно, что оба изменения применились с неточностью (fuzz) некоторого уровня, в данном случае это уровень 2. Уровень неточности определяется тем, как сильно обновление изменило контекст. Начиная с некоторого уровня неточности, патч считается неприменимым:

```
// ffopen4.c
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <ctype.h>

/* open()->fopen() wrapper */
static FILE *ffopen(const char *pathname, const char *mode, int flags)
{
  int m;

  switch(tolower(mode[0])+(mode[1]=='+')) {
    case 'b': /* "a+" */
    case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
    case 'r': m = O_RDONLY; break;
    case 's': m = O_RDWR; break; /* "r+" */
    // TODO "b" modifier
    case 'x': /* "w+" */
    case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
    default: m = O_RDONLY; break;
  }

  int fd = open(pathname, flags, m);
  return fd<0? NULL: fdopen(fd, mode);
}

/* just for test purpose */
int main(int argc, char *argv[]) {
        FILE *fp;

        fp = ffopen(argv[1], "r", O_NOFOLLOW);
        if(fp == NULL) {
          perror(argv[1]);
          return 1;
        }

        return 0;
}
```

```
alt@localhost:~/09_PatchDiff/Lection> patch --verbose ffopen4.c < pall.patch
Hmm...  Looks like a unified diff to me...
The text leading up to this was:
--------------------------
|--- ffopen.c   2024-11-12 16:32:10.603958261 +0300
|+++ ffopen_new2.c      2024-11-12 16:44:50.173769006 +0300
```

```
--------------------------
patching file ffopen4.c
Using Plan A...
Hunk #1 FAILED at 14.
Hunk #2 succeeded at 33 with fuzz 2 (offset 2 lines).
1 out of 2 hunks FAILED -- saving rejects to file ffopen4.c.rej
done
```

При этом создаётся .rej-файл для описания неприменённой части патча с изменениями, которые придется вносить вручную.

```
alt@localhost:~/09_PatchDiff/Lection> cat ffopen4.c.rej
--- ffopen.c    2024-11-12 16:32:10.603958261 +0300
+++ ffopen_new2.c       2024-11-12 16:44:50.173769006 +0300
@@ -14,7 +14,7 @@
     case 'a': m = O_WRONLY | O_CREAT | O_APPEND; break;
     case 'r': m = O_RDONLY; break;
     case 's': m = O_RDWR; break; /* "r+" */
-    case 'x': /* "w+" */
+    case 'x': m = O_RDWR | O_CREAT | O_TRUNC; break; /* w+ */
     case 'w': m = O_WRONLY | O_CREAT | O_TRUNC; break;
     default: m = O_RDONLY; break;
   }
```

15.4 Diffutils

Для поддержки работы с патчами предназначен набор утилит diffutils. Рассмотрим его возможности на примере работы с календарём.

```
alt@localhost:~/09_PatchDiff/Lection/calend> LC_TIME=ru_RU.UTF-8 cal -y > cal-
     end
alt@localhost:~/09_PatchDiff/Lection/calend> cat calend
                                2024

       Январь                  Февраль                  Март
Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
 1  2  3  4  5  6  7              1  2  3  4                 1  2  3
 8  9 10 11 12 13 14    5  6  7  8  9 10 11    4  5  6  7  8  9 10
15 16 17 18 19 20 21   12 13 14 15 16 17 18   11 12 13 14 15 16 17
22 23 24 25 26 27 28   19 20 21 22 23 24 25   18 19 20 21 22 23 24
29 30 31               26 27 28 29            25 26 27 28 29 30 31

       Апрель                    Май                    Июнь
Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
 1  2  3  4  5  6  7           1  2  3  4  5                    1  2
 8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
29 30                  27 28 29 30 31         24 25 26 27 28 29 30

        Июль                   Август                  Сентябрь
```

```
Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
 1  2  3  4  5  6  7             1  2  3  4                     1
 8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
15 16 17 18 19 20 21   12 13 14 15 16 17 18    9 10 11 12 13 14 15
22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                              30
       Октябрь                 Ноябрь                 Декабрь
Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
    1  2  3  4  5  6                1  2  3                     1
 7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
14 15 16 17 18 19 20   11 12 13 14 15 16 17    9 10 11 12 13 14 15
21 22 23 24 25 26 27   18 19 20 21 22 23 24   16 17 18 19 20 21 22
28 29 30 31            25 26 27 28 29 30      23 24 25 26 27 28 29
                                              30 31
```

Сделаем несколько патчей для файла с календарём.

Добавим точки между 21 и 13 у двух месяцев:

```
alt@localhost:~/09_PatchDiff/Lection/calend> sed -i.old1 's/21   13/21...13/'
     calend
alt@localhost:~/09_PatchDiff/Lection/calend> diff -u calend.old1 calend >
     p1.patch
alt@localhost:~/09_PatchDiff/Lection/calend> cat p1.patch
--- calend.old1 2024-11-12 17:09:32.096205400 +0300
+++ calend      2024-11-12 17:09:49.309601942 +0300
@@ -12,7 +12,7 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30
```

Изменим названия осенних месяцев:

```
alt@localhost:~/09_PatchDiff/Lection/calend> sed -i.old2 's/ябрь/ёр../g' cal-
     end
alt@localhost:~/09_PatchDiff/Lection/calend> diff -u calend.old2 calend >
     p2.patch
alt@localhost:~/09_PatchDiff/Lection/calend> cat p2.patch
--- calend.old2 2024-11-12 17:09:49.309601942 +0300
+++ calend      2024-11-12 17:10:11.596350457 +0300
@@ -16,7 +16,7 @@
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентябрь
+        Июль                  Август                Сентёр..
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                     1
```

```
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октябрь                Ноябрь                 Декабрь
+       Октёр..                Ноёр..                 Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

Третьим патчем уберём все добавленные точки:

```
alt@localhost:~/09_PatchDiff/Lection/calend> sed -i.old3 's/[.]/ /g' calend
alt@localhost:~/09_PatchDiff/Lection/calend> diff -u calend.old3 calend >
     p3.patch
alt@localhost:~/09_PatchDiff/Lection/calend> cat p3.patch
--- calend.old3 2024-11-12 17:10:11.596350457 +0300
+++ calend      2024-11-12 17:11:01.573216826 +0300
@@ -12,11 +12,11 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                   Август                 Сентёр..
+        Июль                   Август                 Сентёр
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октёр..                Ноёр..                 Декабрь
+       Октёр                  Ноёр                   Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

Отметим особенность работы утилиты sed, с помощью которой мы задавали изменения. С помощью флага -i можно задать постфикс для имени файла, в котором будет сохранена версия файла до изменения. Это полезно, поскольку patch умеет применять патчи к итоговому файлу, указанному в его output-namefile.

Объединим патчи:

```
alt@localhost:~/09_PatchDiff/Lection/calend> cat p?.patch > pall.patch
alt@localhost:~/09_PatchDiff/Lection/calend> cat pall.patch
```

```
--- calend.old1 2024-11-12 17:09:32.096205400 +0300
+++ calend      2024-11-12 17:09:49.309601942 +0300
@@ -12,7 +12,7 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

--- calend.old2 2024-11-12 17:09:49.309601942 +0300
+++ calend      2024-11-12 17:10:11.596350457 +0300
@@ -16,7 +16,7 @@
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентябрь
+        Июль                  Август                Сентёр..
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октябрь                Ноябрь                 Декабрь
+       Октёр..                Ноёр..                 Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
--- calend.old3 2024-11-12 17:10:11.596350457 +0300
+++ calend      2024-11-12 17:11:01.573216826 +0300
@@ -12,11 +12,11 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентёр..
+        Июль                  Август                Сентёр
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
```

```
 29 30 31                26 27 28 29 30 31      23 24 25 26 27 28 29
                                                30
-       Октёр..                 Ноёр..                  Декабрь
+       Октёр                   Ноёр                    Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                 1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

Применим суммарный патч к исходному календарю:

```
alt@localhost:~/09_PatchDiff/Lection/calend> LC_TIME=ru_RU.UTF-8 cal -y > cal-
    end
alt@localhost:~/09_PatchDiff/Lection/calend> patch --verbose < pall.patch
Hmm...  Looks like a unified diff to me...
The text leading up to this was:
--------------------------
|--- calend.old1        2024-11-12 17:09:32.096205400 +0300
|+++ calend     2024-11-12 17:09:49.309601942 +0300
--------------------------
patching file calend
Using Plan A...
Hunk #1 succeeded at 12.
Hmm...  The next patch looks like a unified diff to me...
The text leading up to this was:
--------------------------
|--- calend.old2        2024-11-12 17:09:49.309601942 +0300
|+++ calend     2024-11-12 17:10:11.596350457 +0300
--------------------------
patching file calend
Using Plan A...
Hunk #1 succeeded at 16.
Hunk #2 succeeded at 24.
Hmm...  The next patch looks like a unified diff to me...
The text leading up to this was:
--------------------------
|--- calend.old3        2024-11-12 17:10:11.596350457 +0300
|+++ calend     2024-11-12 17:11:01.573216826 +0300
--------------------------
patching file calend
Using Plan A...
Hunk #1 succeeded at 12.
Hunk #2 succeeded at 24.
done
```

Судя по диагностическим сообщениям, патч применен успешно. Перейдем к собственно рассмотрению набора инструментов diffutils.

combinediff объединяет патчи в единый блок, при этом по возможности объединяя последовательные действия. При использовании этого инструмента важно указывать патчи в правильном порядке, поскольку применение предшествующего патча формирует контекст для последующего патча.

```
alt@localhost:~/09_PatchDiff/Lection/calend> combinediff p1.patch p2.patch
```

```
diff -u calend calend
--- calend      2024-11-12 17:09:49.309601942 +0300
+++ calend      2024-11-12 17:10:11.596350457 +0300
@@ -12,11 +12,11 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентябрь
+        Июль                  Август                Сентёр..
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октябрь                Ноябрь                 Декабрь
+       Октёр..                Ноёр..                 Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

При объединении 1-го патча с 3-м поглощено изменение, вносящее точки между «21» и «13».

```
alt@localhost:~/09_PatchDiff/Lection/calend> combinediff p1.patch p2.patch |
     combinediff - p3.patch
diff -u calend calend
--- calend      2024-11-12 17:10:11.596350457 +0300
+++ calend      2024-11-12 17:11:01.573216826 +0300
@@ -16,7 +16,7 @@
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентябрь
+        Июль                  Август                Сентёр
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октябрь                Ноябрь                 Декабрь
```

```
+       Октёр                   Ноёр                    Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

Если combinediff описывает объединение патчей, то interdiff наоборот — показывает операции, которые надо провести над файлом, где наложен только первый патч, чтобы получить файл где наложен только второй патч (преобразование base+patch1 -> base+patch2) .

```
alt@localhost:~/09_PatchDiff/Lection/calend> interdiff p1.patch p2.patch
diff -u calend calend
--- calend      2024-11-12 17:09:49.309601942 +0300
+++ calend      2024-11-12 17:10:11.596350457 +0300
@@ -12,11 +12,11 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30

-        Июль                  Август                Сентябрь
+        Июль                  Август                Сентёр..
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7             1  2  3  4                      1
  8  9 10 11 12 13 14    5  6  7  8  9 10 11    2  3  4  5  6  7  8
@@ -24,7 +24,7 @@
 22 23 24 25 26 27 28   19 20 21 22 23 24 25   16 17 18 19 20 21 22
 29 30 31               26 27 28 29 30 31      23 24 25 26 27 28 29
                                               30
-       Октябрь                Ноябрь                 Декабрь
+       Октёр..                Ноёр..                 Декабрь
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
     1  2  3  4  5  6                1  2  3                      1
  7  8  9 10 11 12 13    4  5  6  7  8  9 10    2  3  4  5  6  7  8
```

Утилита rediff помогает исправлять ошибки, допущенные при ручном исправлении патча. Её применение по-прежнему основывается на контексте самого патча и никак не зависит от файла, на который предполагается накладывать патч.

Отредактируем первый патч, в т.ч. внесём ошибку в нумерации строк:

```
# p1.patch
--- calend.old1 2024-11-12 17:09:32.096205400 +0300
+++ calend      2024-11-12 17:09:49.309601942 +0300
@@ -16,7 +12,7 @@ # Поменяли 12 на 16
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
```

```
+15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
+Qoo-Qoo # добавили ещё строчку
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30
```

rediff исправил патч:

```
alt@localhost:~/09_PatchDiff/Lection/calend> rediff p1.patch
--- calend.old1 2024-11-12 17:09:32.096205400 +0300
+++ calend      2024-11-12 17:09:49.309601942 +0300
@@ -16,7 +16,8 @@
 Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс   Пн Вт Ср Чт Пт Сб Вс
  1  2  3  4  5  6  7          1  2  3  4  5                   1  2
  8  9 10 11 12 13 14    6  7  8  9 10 11 12    3  4  5  6  7  8  9
-15 16 17 18 19 20 21   13 14 15 16 17 18 19   10 11 12 13 14 15 16
+15 16 17 18 19 20 21...13 14 15 16 17 18 19   10 11 12 13 14 15 16
+Qoo-Qoo
 22 23 24 25 26 27 28   20 21 22 23 24 25 26   17 18 19 20 21 22 23
 29 30                  27 28 29 30 31         24 25 26 27 28 29 30
```

15.5 Git и патчи

На смену пересылке патчей между разработчиками по электронной почте пришли распределенные системы управления версиями, такие как git. Механизм патчей грамотно интегрирован в работу git.

В git есть свой аналог утилиты diff: git diff. Он вычисляет изменения между коммитами или между последним коммитом и рабочей версией кода.

Перенос изменений между ветками кода в git также выполняется при помощи вычисления патча между последовательными версиями кода в исходной ветке и наложения этого патча на код в целевой ветке.

В git имеется ретро-функциональность: если в репозитории есть данные о разработчике (имя и почта), можно прямо из репозитория по предварительно настроенному каналу отправлять письма с патчами. Это позволяет использовать исторически сложившийся способ обмена изменениями — пересылку друг другу патчей. Для того, чтобы применить полученные патчи, необходимо воспользоваться функцией git am (буквально "git apply mailbox").

Патчирование в git более строгое, чем у обычного инструмента patch, и даже при малейшем несовпадении контекста патча git не позволит его применить, поскольку для совместной разработки точность наложения патчей может быть критичной.

15.6 Домашнее задание

Написать на Си программу, которая генерирует случайный лабиринт размера 6×6

1. Минимальные требования:
   - Лабиринт — это последовательность строк из «.» и «#».

- Длина строки — 13 символов, количество строк также 13. Таким образом, он состоит из «комнат» 3×3 с общими «стенами». Пример полностью непроходимого лабиринта 6×6:

```
#############
#.#.#.#.#.#.#
#############
#.#.#.#.#.#.#
#############
#.#.#.#.#.#.#
#############
#.#.#.#.#.#.#
#############
#.#.#.#.#.#.#
#############
#.#.#.#.#.#.#
#############
```

2. Блоки по сторонам от точек — это «стены», а по диагоналям — «углы». Чтобы пройти из комнаты в комнату, надо разрушить стену. Пример лабиринта 6×6 без стен:

```
#############
#...........#
#.#.#.#.#.#.#
#...........#
#.#.#.#.#.#.#
#...........#
#.#.#.#.#.#.#
#...........#
#.#.#.#.#.#.#
#...........#
#.#.#.#.#.#.#
#...........#
#.#.#.#.#.#.#
#...........#
#############
```

3. Лабиринт, генерируемый программой, должен быть проходимым из любой комнаты в любую. Необязательно: путь между комнатами должен быть единственным. Пример:

```
#############
#.......#...#
#######.#.#.#
#...#...#.#.#
#.###.###.#.#
#...#.....#.#
###.#######.#
#.......#...#
#.#######.###
#...#.....#.#
#.#.#.#####.#
#.#.........#
#############
```

4. Приложить к файлу набор из минимум трёх unified-патчей, которые модифицируют программу следующим образом:
   - размер лабиринта задаётся из командной строки первым параметром;
   - первый параметр — это строка из двух символов, «проход» и «стена» выводимого лабиринта, а второй — его размер;

- первый параметр — начальное значение генератора псевдослучайных чисел (для воспроизведения лабиринта), второй и третий — «проход»-«стена» и размер соответственно.

5. Написать Makefile (или аналог), который будет:
   - генерировать три дополнительных исходника, соответствующих добавлению очередного патча;
   - генерировать, соответственно, четыре бинарника;
   - иметь цель run, которая запускает все четыре бинарника с заданными параметрами;
   - иметь цель clean, уничтожающую все генераты.
6. Создать в репозитории с домашними заданиями подкаталог 09_PatchDiff и разместить в нем проект.

16 Библиотеки и тестирование

Идентификатор темы для сайта: 10_LibTesting

16.1 Сборка библиотек

При работе с проектами возникают различные проблемы при подключении библиотек к создаваемой программе. В случае динамических библиотек необходимо указывать места их расположения через LD_PRELOAD и LD_LIBRARY_PATH. При создании библиотеки необходимо выполнить компиляцию её исходных текстов и компоновку. Эти процессы желательно автоматизировать.

Разберём небольшой пример, в котором создадим собственную библиотеку и будем её использовать.

main.c — тестирующая программа, инкрементирующая при помощи функции inc() поступающее ей из аргументов командной строки число.

```
// main.c
#include <stdio.h>
#include <stdlib.h>
#include "lib.h"

int main(int argc, char *argv[]) {
        int n;

        if(argc < 2) {
          fprintf(stderr, "Usage: %s NUMBER\n", argv[0]);
          return 1;
        }

        n = atoi(argv[1]);
        printf("%d\n", inc(n));

 return 0;
}
```

lib.c — содержит определение функции инкрементирования:

```
// lib.c
#include "lib.h"

int inc(int var) {
  return var + inc_var;
}
```

libg.c — содержит определение глобальной переменной для этой функции:

```
// libg.c
#include "lib.h"
int inc_var = 1;
```

lib.h — заголовочный файл библиотеки:

```
// lib.h
```

```
int inc(int);
extern int inc_var;
```

Напишем для нашей библиотеки Makefile, который будет собирать её и тестировать:

```
# Makefile
all:   inc
libinc.so:     lib.c libg.c
        gcc -fPIC -shared $^ -o $@
inc:   main.c libinc.so
        gcc -L. $< -linc -o $@
check: inc
        LD_LIBRARY_PATH=`pwd` ./inc 123
clean:
        rm -f *.so inc
```

Компиляция и запуск тестов библиотеки:

```
alt@localhost:~/10_LibTesting/Lection> make
gcc -fPIC -shared lib.c libg.c -o libinc.so
gcc -L. main.c -linc -o inc
alt@localhost:~/10_LibTesting/Lection> make check
LD_LIBRARY_PATH=`pwd` ./inc 123
124
```

Даже для простого примера пришлось включить в Makefile много команд, специфических для сборки библиотек. Для упрощения разработки библиотек существуют специальные инструменты.

16.2 Libtool

libtool это мощный инструмент для создания и тестированию библиотек. Перепишем Makefile с использованием специальных флагов и команд этой утилиты:

```
# Makefile
CFLAGS = -g -O
LTFLAGS = --tag=CC

all:    inc

%.lo:   %.c
        libtool --mode=compile $(LTFLAGS) $(CC) -c $<
libinc.la: lib.lo libg.lo
        libtool --mode=link $(LTFLAGS) $(CC) -o $@ $^ -rpath /usr/lib64
inc:    main.o libinc.la
        libtool --mode=link $(LTFLAGS) $(CC) -o $@ $^
check:  inc
        ./$< 123
clean:
        rm -rf *.so inc .libs *.l? *.o
```

Разберём использованные команды:

- Не забываем про встроенные команды make, благодаря которым можно не описывать правила компиляции отдельных файлов, а воспользоваться встроенными рецептами и флагами CFLAGS;
- Так как libtool — стандартная утилита работы с библиотеками, в make для неё существует своя переменная окружения LTFLAGS. При этом базовые флаги для работы с библиотеками (например, -fPIC) добавятся автоматически, а вручную нужно указывать специализированные флаги, например, информацию о компиляторе;
- libtool для работы создаёт не совсем обычные объектные файлы, а так называемые libtool objects (%.lo). Для этого указывается специальный режим compile для создания объектных файлов, а также %.lo-файлов;
- Далее libtool на основе указанных нами lo-файлов создаёт libtool archive (.la). Напомним, что расширение .a свойственно статической библиотеке, здесь же это основа сразу для статической и динамической библиотек. Для создания архива указываем режим компоновки link, а также стандартное место расположения библиотек (-rpath), куда будет помещены готовые библиотеки;
- Для создания бинарного файла, в режиме компоновки объединяем архив с объектным файлом main.o тестирующей функции. Это даёт возможность в тестирующем рецепте убрать явное указание места загрузки динамической библиотеки — libtool сам знает, куда положил библиотеку, и сам её подключит.

В результате модификации Makefile увеличился и усложнился. Это цена использования универсального инструмента в простом примере. Проверим, что сохранилась собираемость и работоспособность библиотеки:

```
alt@localhost:~/10_LibTesting/Lection> ls
inc  lib.c  libg.c  lib.h  libinc.so  main.c  Makefile
alt@localhost:~/10_LibTesting/Lection> make clean
rm -rf *.so inc .libs *.l? *.o
alt@localhost:~/10_LibTesting/Lection> ls
lib.c  libg.c  lib.h  main.c  Makefile

alt@localhost:~/10_LibTesting/Lection> make
cc -g -O   -c -o main.o main.c

libtool --mode=compile --tag=CC cc -c lib.c
libtool: compile:  cc -c lib.c  -fPIC -DPIC -o .libs/lib.o
libtool: compile:  cc -c lib.c -o lib.o >/dev/null 2>&1

libtool --mode=compile --tag=CC cc -c libg.c
libtool: compile:  cc -c libg.c  -fPIC -DPIC -o .libs/libg.o
libtool: compile:  cc -c libg.c -o libg.o >/dev/null 2>&1

libtool --mode=link --tag=CC cc -o libinc.la lib.lo libg.lo -rpath /usr/lib64
libtool: link: gcc -shared  -fPIC -DPIC  .libs/lib.o .libs/libg.o      -Wl,-
     soname -Wl,libinc.so.0 -o .libs/libinc.so.0.0.0
libtool: link: (cd ".libs" && rm -f "libinc.so.0" && ln -s "libinc.so.0.0.0"
     "libinc.so.0")
```

```
libtool: link: (cd ".libs" && rm -f "libinc.so" && ln -s "libinc.so.0.0.0"
     "libinc.so")
libtool: link: ar cr .libs/libinc.a  lib.o libg.o
libtool: link: ranlib .libs/libinc.a
libtool: link: ( cd ".libs" && rm -f "libinc.la" && ln -s "../libinc.la" "lib-
     inc.la" )

libtool --mode=link --tag=CC cc -o inc main.o libinc.la
libtool: link: cc -o .libs/inc main.o  ./.libs/libinc.so -Wl,-rpath -Wl,/usr/
     lib64

alt@localhost:~/10_LibTesting/Lection> make check
./inc 123
124
```

Видно, что libtool создал и статическую (.a) и динамическую (.so) библиотеки, разместив их в каталоге .libs. Кроме того, в процессе работы были созданы промежуточные объектные файлы — исследуем их.

```
alt@localhost:~/10_LibTesting/Lection> ls -a
.   inc    libg.c   libg.o  libinc.la  lib.o  main.c  Makefile
..  lib.c  libg.lo  lib.h   lib.lo     .libs  main.o
alt@localhost:~/10_LibTesting/Lection> ls .libs/
inc     libinc.a   libinc.lai  libinc.so.0      lib.o
libg.o  libinc.la  libinc.so   libinc.so.0.0.0
```

Что представляют собой два исполняемых файла inc?

```
alt@localhost:~/10_LibTesting/Lection> diff inc .libs/inc
Binary files inc and .libs/inc differ
alt@localhost:~/10_LibTesting/Lection> ./inc 123
124
alt@localhost:~/10_LibTesting/Lection> ./.libs/inc 123
./.libs/inc: error while loading shared libraries: libinc.so.0: cannot open
     shared object file: No such file or directory
alt@localhost:~/10_LibTesting/Lection> LD_LIBRARY_PATH=`pwd`/.libs ./.libs/inc
     123
124
```

Один исполняемый файл «подтягивает» библиотеку автоматически, второй — требует явного указания пути к ней.

Что из себя представляют .lo-файлы и .la-файлы?

```
alt@localhost:~/10_LibTesting/Lection> cat lib.lo
# lib.lo - a libtool object file
# Generated by libtool (GNU libtool) 2.4.7
#
# Please DO NOT delete this file!
# It is necessary for linking the library.

# Name of the PIC object.
pic_object='.libs/lib.o'
```

```
# Name of the non-PIC object
non_pic_object='lib.o'

alt@localhost:~/10_LibTesting/Lection> cat libinc.la
# libinc.la - a libtool library file
# Generated by libtool (GNU libtool) 2.4.7
#
# Please DO NOT delete this file!
# It is necessary for linking the library.

# The name that we can dlopen(3).
dlname='libinc.so.0'

# Names of this library.
library_names='libinc.so.0.0.0 libinc.so.0 libinc.so'

# The name of the static archive.
old_library='libinc.a'

# Linker flags that cannot go in dependency_libs.
inherited_linker_flags=''

# Libraries that this one depends upon.
dependency_libs=''

# Names of additional weak libraries provided by this library
weak_library_names=''

# Version information for libinc.
current=0
age=0
revision=0

# Is this an already installed library?
installed=no

# Should we warn about portability when linking against -modules?
shouldnotlink=no

# Files to dlopen/dlpreopen
dlopen=''
dlpreopen=''

# Directory that this library needs to be installed in:
libdir='/usr/lib64'
```

Это, как и предполагалось, не обычные объектные файлы, а описания того, как строятся наши библиотеки — «старый» формат (статическая библиотека), информация о предзагрузках, расположение и версии библиотек. Сами же библиотеки хранятся в подкаталоге .libs и подгружаются оттуда. Лежащий вне подкаталога файл inc это то-

же не просто бинарный файл программы, а shell-скрипт, в котором и подключаются все зависимости:

```
alt@localhost:~/10_LibTesting/Lection> cat inc
#! /usr/bin/bash

# inc - temporary wrapper script for .libs/inc
# Generated by libtool (GNU libtool) 2.4.7
#
# The inc program cannot be directly executed until all the libtool
# libraries that it depends on are installed.
#
# This wrapper script should never be moved out of the build directory.
# If it is, it will not operate correctly.

LD_LIBRARY_PATH="/home/alt/10_LibTesting/Lection/.libs:$LD_LIBRARY_PATH"
```

16.3 Версионирование библиотек

Судя по содержимому каталога .libs, libtool создал нам три версии нашей библиотеки — без чисел, с одним числом и с тремя числами в имени файла.

```
alt@localhost:~/10_LibTesting/Lection> cd .libs/
alt@localhost:~/10_LibTesting/Lection/.libs> la -A *.so*
lrwxrwxrwx 1 alt alt    15 Nov 25 07:55 libinc.so -> libinc.so.0.0.0
lrwxrwxrwx 1 alt alt    15 Nov 25 07:55 libinc.so.0 -> libinc.so.0.0.0
-rwxr-xr-x 1 alt alt 16624 Nov 25 07:55 libinc.so.0.0.0
```

Две из них — символьные ссылки на третью. Бесчисловая версия используется для создания зависимостей на библиотеку при работе зависящих утилит, основная («мажорная», с одним числом) используется для указания на основную поддерживаемую версию, а последняя (единственная реальная библиотека) является собственно рабочим объектом.

Что означают числа в версии библиотеки? В случае с библиотеками, числовые версии в именах файлов указывают на изменение ABI (Application Binary Interface — двоичный интерфейс приложения). Этот интерфейс задаёт низкоуровневые аспекты работы ПО на уровне машинного кода и инструкций процессора. В соответствии с *правилами версионирования библиотек*[74], нумерация версий задается в формате current:revision:age. Смысл этих полей следующий:

- current описывает изменения ABI, влияющие на обратную совместимость библиотеки, и увеличивается при каждом вмешательстве в ABI;
- revision описывает изменения, не меняющие ABI, и увеличивается при внесении таких правок, устанавливаясь в 0 при каждом изменении current;

74 Library interface versions [Электронный ресурс]. URL: https://www.gnu.org/software/libtool/manual/libtool.html#Versioning (дата обращения: 20.01.2025).

- age описывает изменения ABI, не влияющие на обратную совместимость. Например, добавление нового поля в конец структуры при определённых дисциплинах программирования не будет ломать совместимость; age всегда должно быть не больше current.

Версионирование библиотек можно переложить на стандартный формат version.minor.subminor для описания основной и двух вторичных нумераций версий, меняющихся в зависимости от глобальности изменений файла, который будет вычисляться как {current-age}.age.revision, однако полноценно считать это версией неправильно, как минимум, из-за неравномерности роста version. Зато такое версионирование библиотек позволяет нам поддерживать сразу диапазон допустимых интерфейсов — от current - age до current.

Семантическое версионирование, к которому мы привыкли, не поддерживает такой диапазон номеров. Номер версии описывается значениями MAJOR.MINOR.PATCH и меняется по следующим правилам:

- Обратно **не**совместимые изменения ABI (удаление / изменение): MAJOR++.MINOR=0.PATCH=0;
- Обратно совместимые изменения ABI (добавление): MAJOR.MINOR++.PATCH=0;
- Обратно совместимые исправления ABI и изменения, не затрагивающие ABI: MAJOR.MINOR.PATCH++.

16.4 Тестирование

Компилятор проверяет лишь синтаксические ошибки кода, в крайнем случае — указывает своими предупреждениями на возможные смысловые ошибки. Для проверки семантики кода, а также того информационного пространства, которое этот код создаёт и с которым взаимодействует (глобальные переменные, макросы, переменные окружения) необходимо использовать тесты. Их прохождение не гарантирует правильность работы функций, однако повышает вероятность обнаружения ошибок в работе программы.

Выделяют 4 вида тестирования:

- **Модульное** — проверяет работу отдельных модулей написанного продукта;
- **Системное** — проверяет работу всей системы изолированно от внешних зависимостей;
- **Интеграционное** — проверяет работу в рамках среды, в которой она запускается, и на, вообще говоря, не нами задаваемых условиях среды;
- **Приёмочное тестирование** — проверяет внешнее воздействие на проект, внесение невстроенных тестов и проверка поведения системы вообще на её взаимодействие с программами.

Сосредоточимся на самом базовом уровне тестирования — модульном (unit-тестирование). В рамках unit-тестов определён test runner — основная программа, запускающая тесты. Она отвечает за проверку подключаемости тестирующего кода (библиотека это или какой-то другой исполняемый объект), фильтрацию тестов, сбор информации по запускам и т.д.
Все тесты (test) объединены в отдельные тестирующие блоки (case), проверяющие

работу какой-то конкретной функции, а множества блоков, описывающих один раздел работы с проектом (например, *работа с файлами*) объединены в test suite. Для более точного понимания можно запомнить, что suite отвечает на вопрос «Какая функциональность сломалась?», case — «Какая функция сломалась?», test — «На каких данных сломалась?».

Следующий важный блок любого тестирования — подготовка и ликвидация тестируемого окружения (*фикстуры*). Фикстуры могут создаваться для каждого из уровней тестирования и описывать все необходимые зависимости и данные для проверки работоспособности системы. Также в рамках тестирования считается специальный параметр — тестовое покрытие (coverage), описывающий полноту тестирования программы. Покрытие может быть рассчитано как процент количества задействованных при прогоне тестов строк программы, процент задействованных путей её выполнения и т. п.

16.5 Check

Одна из классических утилит для тестирования это *Check*[75]. Он написан на Си и поддерживает основные технологии тестирования.

В качестве примера рассмотрим *генератор фэнтезийных имён*[76] и напишем для него тесты. В состав примера первоначально входят следующие файлы:

```
alt@localhost:~/10_LibTesting/Lection/fantasy> tree
.
└── src
    ├── example.c
    ├── Makefile
    ├── namegen.h
    └── tablegen.py

2 directories, 4 files
```

Файл example.c первоначально содержит написанные вручную тесты, использующие определенный в этом же файле макрос TEST.

```
// example.c
#include "namegen.h"
#include <time.h>
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

static unsigned long
hash32(unsigned long a)
{
    a = a & 0xffffffffUL;
    a = (a ^ 61UL) ^ (a >> 16);
```

75 Check. Unit Testing Framework for C [Электронный ресурс]. URL: https://libcheck.github.io/check/ (дата обращения: 20.01.2025).

76 Fantasy Name Generator [Электронный ресурс]. URL: https://github.com/skeeto/fantasyname/tree/master/c (дата обращения: 20.01.2025).

```
    a = (a + (a << 3)) & 0xffffffffUL;
    a = a ^ (a >> 4);
    a = (a * 0x27d4eb2dUL) & 0xffffffffUL;
    a = a ^ (a >> 15);
    return a;
}

#if _WIN32
#  define C_RED(s)     s
#  define C_GREEN(s)   s
#else
#  define C_RED(s)     "\033[31;1m" s "\033[0m"
#  define C_GREEN(s)   "\033[32;1m" s "\033[0m"
#endif

#define TEST(s, x) \
    do { \
        if (x) { \
            printf(C_GREEN("PASS") " %s\n", s); \
            count_pass++; \
        } else { \
            printf(C_RED("FAIL") " %s\n", s); \
            count_fail++; \
        } \
    } while (0)

static void
tests(void)
{
    int r;
    char buf[256];
    int count_pass = 0;
    int count_fail = 0;
    unsigned long rng[] = {0xd22bfb3b};

    {
        int i;
        int mismatch = 0;
        int seen[3] = {0, 0, 0};
        for (i = 0; i < 100; i++) {
            memset(buf, 0x5a, sizeof(buf));
            r = namegen(buf, sizeof(buf), "0|1|2", rng);
            if (r != NAMEGEN_SUCCESS || buf[1] || !strchr("012", buf[0])) {
                mismatch++;
            } else {
                seen[buf[0] - '0']++;
            }
        }
        TEST("alternation", !mismatch && seen[0] && seen[1] && seen[2]);
    }

    {
        int i;
        int mismatch = 0;
```

```
        int seen[6] = {0, 0, 0};
        for (i = 0; i < 100; i++) {
            memset(buf, 0x5a, sizeof(buf));
            r = namegen(buf, sizeof(buf), "(v)v", rng);
            if (r != NAMEGEN_SUCCESS || buf[0] != 'v' || !buf[1] || buf[2]) {
                mismatch++;
            } else {
                char *p = "aeiouy";
                char *s = strchr(p, buf[1]);
                if (!s)
                    mismatch++;
                else
                    seen[s - p]++;
            }
        }
        for (i = 0; i < 6; i++)
            mismatch += !seen[i];
        TEST("substitution", !mismatch);
    }

    {
        r = namegen(buf, sizeof(buf), "a>", rng);
        TEST("closing bracket", r == NAMEGEN_INVALID);
    }

    {
        r = namegen(buf, sizeof(buf), "a)", rng);
        TEST("closing paren", r == NAMEGEN_INVALID);
    }

    {
        r = namegen(buf, sizeof(buf), "<a)", rng);
        TEST("mismatch paren", r == NAMEGEN_INVALID);
    }

    {
        r = namegen(buf, sizeof(buf), "(a>", rng);
        TEST("mismatch bracket", r == NAMEGEN_INVALID);
    }

    {
        r = namegen(buf, sizeof(buf), "<a", rng);
        TEST("unclosed bracket", r == NAMEGEN_INVALID);
    }

    {
        r = namegen(buf, sizeof(buf), "(a", rng);
        TEST("unclosed paren", r == NAMEGEN_INVALID);
    }

    {
        char *p = "(((((<((((((((((((((a)))))))))))))))>)))))";
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, sizeof(buf), p, rng);
```

```
        TEST("deep nesting", r == NAMEGEN_SUCCESS && !strcmp(buf, "a"));
    }

    {
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, 4, "(abcdefg)", rng);
        TEST("truncation 1",
             r == NAMEGEN_TRUNCATED && !strcmp(buf, "abc") && buf[4] == 0x5a);
    }

    {
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, 2, "i", rng);
        TEST("truncation 2",
             r == NAMEGEN_TRUNCATED && buf[0] && !buf[1] && buf[2] == 0x5a);
    }

    {
        r = namegen(buf, sizeof(buf), "!(foo)", rng);
        TEST("capitalization 1", r == NAMEGEN_SUCCESS && !strcmp(buf, "Foo"));
    }

    {
        r = namegen(buf, sizeof(buf), "(!foo)", rng);
        TEST("capitalization 2", r == NAMEGEN_SUCCESS && !strcmp(buf, "Foo"));
    }

    {
        r = namegen(buf, sizeof(buf), "(f!oo)", rng);
        TEST("capitalization 3", r == NAMEGEN_SUCCESS && !strcmp(buf, "fOo"));
    }

    {
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, sizeof(buf), "", rng);
        TEST("empty", r == NAMEGEN_SUCCESS && !buf[0] && buf[1] == 0x5a);
    }

    if (count_fail)
        exit(EXIT_FAILURE);
    printf("All %d tests passed\n", count_pass);
    exit(EXIT_SUCCESS);
}

int
main(int argc, char **argv)
{
    int i;
    int count = 1;
    FILE *urandom;
    unsigned char randbuf[4];
    unsigned long seed[] = {0x8af611acUL};

    /* Parse command line arguments */
```

```
    if (argc < 2 || argc > 3) {
        printf("Usage: %s <pattern> [num]\n", argv[0]);
        printf("  pattern   template for names to generate\n");
        printf("  num       number of names to generate\n");
        exit(EXIT_FAILURE);
    } else if (argc == 3) {
        count = atoi(argv[2]);
    }

    if (!strcmp(argv[1], "--test"))
        tests();

    /* Shuffle up the seed a bit */
    urandom = fopen("/dev/urandom", "rb");
    if (urandom && fread(randbuf, sizeof(randbuf), 1, urandom)) {
        unsigned long rnd =
            (unsigned long)randbuf[0] <<  0 |
            (unsigned long)randbuf[1] <<  8 |
            (unsigned long)randbuf[2] << 16 |
            (unsigned long)randbuf[3] << 24;
        *seed ^= rnd;
        fclose(urandom);
    } else {
        void *p = malloc(4UL * 1024 * 1024);
        *seed ^= hash32(time(0));             /* Current time */
        *seed ^= hash32((ptrdiff_t)main);     /* ASLR entopy */
        *seed ^= hash32((ptrdiff_t)seed);     /* Stack gap entropy */
        *seed ^= hash32((ptrdiff_t)p);        /* Allocator entropy */
        *seed ^= hash32((ptrdiff_t)malloc);   /* C library ASLR */
        free(p);
    }

    /* Generate some names */
    for (i = 0; i < count; i++) {
        char buf[256];
        int r = namegen(buf, sizeof(buf), argv[1], seed);
        if (r == NAMEGEN_INVALID) {
            fprintf(stderr, "%s: invalid pattern\n", argv[0]);
            exit(EXIT_FAILURE);
        }
        printf("%s\n", buf);
    }
    return 0;
}
```

Первоначально namegen.h содержит огромные таблицы символов. Сразу вынесем их в файл const.h, генерируемый программой tablegen.py. Эту программу потребуется один раз запустить вручную.

```
alt@localhost:~/10_LibTesting/Lection/fantasy/src> python3 tablegen.py > con-
     st.h
```

```
// namegen.h
/* Fantasy name generator ANSI C header library
```

```
 * This is free and unencumbered software released into the public domain.
 *
 * The compile() function creates a name generator based on an input
 * pattern. The letters s, v, V, c, B, C, i, m, M, D, and d represent
 * different types of random replacements. Everything else is emitted
 * literally.
 *
 *   s - generic syllable
 *   v - vowel
 *   V - vowel or vowel combination
 *   c - consonant
 *   B - consonant or consonant combination suitable for beginning a word
 *   C - consonant or consonant combination suitable anywhere in a word
 *   i - insult
 *   m - mushy name
 *   M - mushy name ending
 *   D - consonant suited for a stupid person's name
 *   d - syllable suited for a stupid person's name (begins with a vowel)
 *
 * All characters between parenthesis () are emitted literally. For
 * example, the pattern "s(dim)", emits a random generic syllable
 * followed by "dim".
 *
 * Characters between angle brackets <> emit patterns from the table
 * above. Imagine the entire pattern is wrapped in one of these.
 *
 * In both types of groupings, a vertical bar | denotes a random choice.
 * Empty groups are allowed. For example, "(foo|bar)" emits either "foo"
 * or "bar". The pattern "<c|v|>" emits a constant, vowel, or nothing at
 * all.
 *
 * An exclamation point ! means to capitalize the component that follows
 * it. For example, "!(foo)" will emit "Foo" and "v!s" will emit a
 * lowercase vowel followed by a capitalized syllable, like "eRod".
 *
 * This library is based on the RinkWorks Fantasy Name Generator.
 * http://www.rinkworks.com/namegen/
 */
#ifndef NAMEGEN_H
#define NAMEGEN_H

#define NAMEGEN_MAX_DEPTH  32 /* Cannot exceed bits in a long */

/* Return codes */
#define NAMEGEN_SUCCESS    0
#define NAMEGEN_TRUNCATED  1  /* Output was truncated */
#define NAMEGEN_INVALID    2  /* Pattern is invalid */
#define NAMEGEN_TOO_DEEP   3  /* Pattern exceeds maximum nesting depth */
#include "const.h"

/* Generate a name into DST of LEN bytes from PATTERN and using SEED.
 *
 * The length must be non-zero. For best results, the lower 32 bits of
 * the seed should be thoroughly initialized. A particular seed will
```

```
 * produce the same results on all platforms.
 *
 * The return value is one of the above codes, indicating success or
 * that something went wrong. Truncation occurs when DST was too short.
 * Pattern is validated even when the output has been truncated.
 */
static int
namegen(char *dst, unsigned long len, const char *pattern, unsigned long
*seed);

/* Implementation */

/* Rather than compile the pattern into some internal representation,
 * the name is generated directly from the pattern in a single pass
 * using reservoir sampling. If an alternate option is selected, the
 * output pointer is reset to "undo" the output for the previous group.
 * This means the output buffer may be written beyond the final output
 * length (but never beyond the buffer length).
 *
 * The substitution templates are stored in an efficient, packed form
 * that contains no pointers. This is to avoid cluttering up the
 * relocation table, but without any additional run-time overhead.
 */

/* Return offsets table offset for C, or -1 if not special.
 * The return value is suitable for namegen_offsets().
 */
static int
namegen_special(int c)
{
    if (c >= 0 && c < 128)
        return special[c];
    return -1;
}

/* Return offsets and number of offsets for special N.
 * Offsets point into namegen_argz.
 */
static int
namegen_offsets(int n, const short **offsets)
{
    *offsets = offsets_table + off_len[n * 2 + 0];
    return off_len[n * 2 + 1];
}

static unsigned long
namegen_rand32(unsigned long *s)
{
    unsigned long x = *s;
    x ^= x << 13;
    x ^= (x & 0xffffffffUL) >> 17;
    x ^= x << 5;
    return (*s = x) & 0xffffffffUL;
}
```

```
static int
namegen_cap(int c, int capitalize)
{
    return capitalize && c >= 'a' && c <= 'z' ? c & ~0x20 : c;
}

/* Copy a random substitution for template C into P, but only before E.
 */
static char *
namegen_copy(char *p, char *e, int c, unsigned long *seed, int capitalize)
{
    int n = namegen_special(c);
    if (n == -1) {
        if (p != e)
            *p++ = namegen_cap(c, capitalize);
    } else {
        const short *offsets;
        int count = namegen_offsets(n, &offsets);
        int select = namegen_rand32(seed) % count;
        const char *s = namegen_argz + offsets[select];
        while (*s) {
            int r = *s++;
            if (p != e)
                *p++ = namegen_cap(r, capitalize);
            capitalize = 0;
        }
    }
    return p;
}

static int
namegen(char *dst, unsigned long len, const char *pattern, unsigned long
*seed)
{
    int depth = 0;               /* Current nesting depth */
    char *p = dst;               /* Current output pointer */
    char *e = dst + len;         /* Maxiumum output pointer */
    int capitalize = 0;          /* Capitalize next item */

    /* Stacks */
    char *reset[NAMEGEN_MAX_DEPTH];     /* Reset pointer (undo generate) */
    unsigned long n[NAMEGEN_MAX_DEPTH]; /* Number of groups */
    unsigned long silent = 0;    /* Actively generating? */
    unsigned long literal = 0;   /* Current "mode" */
    unsigned long capstack = 0;  /* Initial capitalization state */

    n[0] = 1;
    reset[0] = dst;
    for (; *pattern; pattern++) {
        unsigned long bit; /* Bit for current depth */
        int c = *pattern;
        switch (c) {
            case '<':
```

```
            if (++depth == NAMEGEN_MAX_DEPTH) {
                *dst = 0;
                return NAMEGEN_TOO_DEEP;
            }
            bit = 1UL << depth;
            n[depth] = 1;
            reset[depth] = p;
            literal &= ~bit;
            silent &= ~bit;
            silent |= (silent << 1) & bit;
            capstack &= ~bit;
            capstack |= (unsigned long)capitalize << depth;
            break;

        case '(':
            if (++depth == NAMEGEN_MAX_DEPTH) {
                *dst = 0;
                return NAMEGEN_TOO_DEEP;
            }
            bit = 1UL << depth;
            n[depth] = 1;
            reset[depth] = p;
            literal |= bit;
            silent &= ~bit;
            silent |= (silent << 1) & bit;
            capstack &= ~bit;
            capstack |= (unsigned long)capitalize << depth;
            break;

        case '>':
            if (depth == 0) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            bit = 1UL << depth--;
            if (literal & bit) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            break;

        case ')':
            if (depth == 0) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            bit = 1UL << depth--;
            if (!(literal & bit)) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            break;

        case '|':
```

```
                bit = 1UL << depth;
                /* Stay silent if parent group is silent */
                if (!(silent & (bit >> 1))) {
                    if (namegen_rand32(seed) < 0xffffffffUL / ++n[depth]) {
                        /* Switch to this option */
                        p = reset[depth];
                        silent &= ~bit;
                        capitalize = !!(capstack & bit);
                    } else {
                        /* Skip this option */
                        silent |= bit;
                    }
                }
                break;

            case '!':
                capitalize = 1;
                break;

            default:
                bit = 1UL << depth;
                if (!(silent & bit)) {
                    if (literal & bit) {
                        /* Copy value literally */
                        if (p != e)
                            *p++ = namegen_cap(c, capitalize);
                    } else {
                        /* Copy a substitution */
                        p = namegen_copy(p, e, c, seed, capitalize);
                    }
                }
                capitalize = 0;
        }
    }

    if (depth) {
        *dst = 0;
        return NAMEGEN_INVALID;
    } else if (p == e) {
        p[-1] = 0;
        return NAMEGEN_TRUNCATED;
    } else {
        *p = 0;
        return NAMEGEN_SUCCESS;
    }
}

#endif
```

Makefile содержит цель check, в которой программа вызывается с ключом, запускающим тесты.

```
# Makefile
.POSIX:
CC      = cc
```

```
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3 -fpie
LDFLAGS = -pie

namegen: example.c namegen.h
        $(CC) $(LDFLAGS) $(CFLAGS) -o $@ example.c $(LDLIBS)

test: check
check: namegen
        @./namegen --test

clean:
        rm -f namegen
```

Выполним сборку и тестирование:

```
alt@localhost:~/10_LibTesting/Lection/fantasy> cd src/
alt@localhost:~/10_LibTesting/Lection/fantasy/src> make
cc -pie -ansi -pedantic -Wall -Wextra -O3 -g3 -fpie -o namegen example.c
alt@localhost:~/10_LibTesting/Lection/fantasy/src> make check
PASS alternation
PASS substitution
PASS closing bracket
PASS closing paren
PASS mismatch paren
PASS mismatch bracket
PASS unclosed bracket
PASS unclosed paren
PASS deep nesting
PASS truncation 1
PASS truncation 2
PASS capitalization 1
PASS capitalization 2
PASS capitalization 3
PASS empty
All 15 tests passed
```

Прежде чем перевести тесты на использование инструмента check, организуем стандартный autotools-проект.

```
# Makefile.am
SUBDIRS = src

demo:   all
        ./src/namegen '!BVCs !DdM'
```

Исходный файл example.c переименуем в namegen.c и учтем это в конфигурационных файлах.

```
# configure.ac
#                                               -*- Autoconf -*-
# Process this file with autoconf to produce a configure script.

AC_PREREQ([2.69])
AC_INIT([namegen], [0.0.1], [george@altlinux.org])
AM_INIT_AUTOMAKE([foreign])
AC_CONFIG_SRCDIR([src/namegen.c])
```

```
AC_CONFIG_HEADERS([config.h])

# Checks for programs.
AC_PROG_CC

# Checks for libraries.

# Checks for header files.
AC_CHECK_HEADERS([stddef.h stdlib.h string.h])

# Checks for typedefs, structures, and compiler characteristics.
AC_CHECK_TYPES([ptrdiff_t])

# Checks for library functions.
AC_FUNC_MALLOC
AC_CHECK_FUNCS([memset strchr])

AC_CONFIG_FILES([Makefile
                 src/Makefile])
AC_OUTPUT
```

Makefile переместим в src/Makefile.in, при этом добавим цель для генерации const.h

```
# Makefile.in
.POSIX:
CC      = cc
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3 -fpie
LDFLAGS = -pie

all:    namegen

const.h: tablegen.py
        python3 $< > $@

namegen: namegen.c namegen.h
        $(CC) $(LDFLAGS) $(CFLAGS) -o $@ example.c $(LDLIBS)

test: check
check: namegen
        @./namegen --test

clean:
        rm -f namegen
```

Получим следующее размещение файлов:

```
alt@localhost:~/10_LibTesting/Lection/fantasy> tree
.
├── configure.ac
├── Makefile.am
└── src
    ├── example.c
    ├── Makefile.in
    ├── namegen
    ├── namegen.h
    └── tablegen.py
```

```
2 directories, 7 files
```

Выполним сборку и запуск:

```
alt@localhost:~/10_LibTesting/Lection/fantasy> autoreconf -fisv
```

```
alt@localhost:~/10_LibTesting/Lection/fantasy> ./configure
```

```
alt@localhost:~/10_LibTesting/Lection/fantasy> make demo
make  all-recursive
```

```
./src/namegen '!BVCs !DdM'
Thestit Thulfboo
alt@localhost:~/10_LibTesting/Lection/fantasy> make demo
make  all-recursive
```

```
./src/namegen '!BVCs !DdM'
Craiwar Gongpie
```

Настроим Automake в каталоге src:

```
# src/Makefile.am
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3

bin_PROGRAMS=namegen

namegen.o: const.h

const.h: tablegen.py
        python3 $< > $@

check: namegen
        @./namegen --test
```

Разделим namegen.h на «собственно» заголовочный файл и файл с реализацией библиотечных функций. Это деление необходимо отразить в Makefile.am:

```
src/Makefile.am
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3

bin_PROGRAMS=namegen
namegen_SOURCES=namegen.c libnamegen.c

libnamegen.c: const.h

const.h: tablegen.py
        python3 $< > $@

check: namegen
        @./namegen --test
```

Заголовочный файл:

```
// src/namegen.h
#ifndef NAMEGEN_H
#define NAMEGEN_H

#define NAMEGEN_MAX_DEPTH  32 /* Cannot exceed bits in a long */

/* Return codes */
#define NAMEGEN_SUCCESS    0
#define NAMEGEN_TRUNCATED  1  /* Output was truncated */
#define NAMEGEN_INVALID    2  /* Pattern is invalid */
#define NAMEGEN_TOO_DEEP   3  /* Pattern exceeds maximum nesting depth */

int namegen(char *, unsigned long, const char *, unsigned long *);

#endif
```

Файл с реализацией, но уже без «ручных» тестов:

```
// src/libnamegen.c
/* Fantasy name generator ANSI C header library
 *
 */
#include "namegen.h"
#include "const.h"

/* Generate a name into DST of LEN bytes from PATTERN and using SEED.
 *
 */

/* Implementation */

/* Rather than compile the pattern into some internal representation,
 *
 */

/* Return offsets table offset for C, or -1 if not special.
 * The return value is suitable for namegen_offsets().
 */
static int
namegen_special(int c)
{
    if (c >= 0 && c < 128)
        return special[c];
    return -1;
}

/* Return offsets and number of offsets for special N.
 * Offsets point into namegen_argz.
 */
static int
namegen_offsets(int n, const short **offsets)
{
```

```
    *offsets = offsets_table + off_len[n * 2 + 0];
    return off_len[n * 2 + 1];
}

static unsigned long
namegen_rand32(unsigned long *s)
{
    unsigned long x = *s;
    x ^= x << 13;
    x ^= (x & 0xffffffffUL) >> 17;
    x ^= x << 5;
    return (*s = x) & 0xffffffffUL;
}

static int
namegen_cap(int c, int capitalize)
{
    return capitalize && c >= 'a' && c <= 'z' ? c & ~0x20 : c;
}

/* Copy a random substitution for template C into P, but only before E.
 */
static char *
namegen_copy(char *p, char *e, int c, unsigned long *seed, int capitalize)
{
    int n = namegen_special(c);
    if (n == -1) {
        if (p != e)
            *p++ = namegen_cap(c, capitalize);
    } else {
        const short *offsets;
        int count = namegen_offsets(n, &offsets);
        int select = namegen_rand32(seed) % count;
        const char *s = namegen_argz + offsets[select];
        while (*s) {
            int r = *s++;
            if (p != e)
                *p++ = namegen_cap(r, capitalize);
            capitalize = 0;
        }
    }
    return p;
}

int
namegen(char *dst, unsigned long len, const char *pattern, unsigned long
*seed)
{
    int depth = 0;               /* Current nesting depth */
    char *p = dst;               /* Current output pointer */
    char *e = dst + len;         /* Maxiumum output pointer */
    int capitalize = 0;          /* Capitalize next item */

    /* Stacks */
```

```
    char *reset[NAMEGEN_MAX_DEPTH];     /* Reset pointer (undo generate) */
    unsigned long n[NAMEGEN_MAX_DEPTH]; /* Number of groups */
    unsigned long silent = 0;    /* Actively generating? */
    unsigned long literal = 0;   /* Current "mode" */
    unsigned long capstack = 0;  /* Initial capitalization state */

    n[0] = 1;
    reset[0] = dst;
    for (; *pattern; pattern++) {
        unsigned long bit; /* Bit for current depth */
        int c = *pattern;
        switch (c) {
            case '<':
                if (++depth == NAMEGEN_MAX_DEPTH) {
                    *dst = 0;
                    return NAMEGEN_TOO_DEEP;
                }
                bit = 1UL << depth;
                n[depth] = 1;
                reset[depth] = p;
                literal &= ~bit;
                silent &= ~bit;
                silent |= (silent << 1) & bit;
                capstack &= ~bit;
                capstack |= (unsigned long)capitalize << depth;
                break;

            case '(':
                if (++depth == NAMEGEN_MAX_DEPTH) {
                    *dst = 0;
                    return NAMEGEN_TOO_DEEP;
                }
                bit = 1UL << depth;
                n[depth] = 1;
                reset[depth] = p;
                literal |= bit;
                silent &= ~bit;
                silent |= (silent << 1) & bit;
                capstack &= ~bit;
                capstack |= (unsigned long)capitalize << depth;
                break;

            case '>':
                if (depth == 0) {
                    *dst = 0;
                    return NAMEGEN_INVALID;
                }
                bit = 1UL << depth--;
                if (literal & bit) {
                    *dst = 0;
                    return NAMEGEN_INVALID;
                }
                break;
```

```
        case ')':
            if (depth == 0) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            bit = 1UL << depth--;
            if (!(literal & bit)) {
                *dst = 0;
                return NAMEGEN_INVALID;
            }
            break;

        case '|':
            bit = 1UL << depth;
            /* Stay silent if parent group is silent */
            if (!(silent & (bit >> 1))) {
                if (namegen_rand32(seed) < 0xffffffffUL / ++n[depth]) {
                    /* Switch to this option */
                    p = reset[depth];
                    silent &= ~bit;
                    capitalize = !!(capstack & bit);
                } else {
                    /* Skip this option */
                    silent |= bit;
                }
            }
            break;

        case '!':
            capitalize = 1;
            break;

        default:
            bit = 1UL << depth;
            if (!(silent & bit)) {
                if (literal & bit) {
                    /* Copy value literally */
                    if (p != e)
                        *p++ = namegen_cap(c, capitalize);
                } else {
                    /* Copy a substitution */
                    p = namegen_copy(p, e, c, seed, capitalize);
                }
            }
            capitalize = 0;
    }
}

if (depth) {
    *dst = 0;
    return NAMEGEN_INVALID;
} else if (p == e) {
    p[-1] = 0;
    return NAMEGEN_TRUNCATED;
```

```
    } else {
        *p = 0;
        return NAMEGEN_SUCCESS;
    }
}
```

Теперь непосредственно подключим в проект libtool: из libnamegen.c будет собираться динамическая библиотека, configure.ac задействует libtool (дополнительно мы запрещаем сборку статической библиотеки, поскольку она не нужна).

src/Makefile.am настроим на использование .la-файлов и, как мы уже видели раньше, генерируемый файл src/namegen не бинарный исполняемый файл, а сценарий на shell, автоматически устанавливающий LD_PRELOAD для запуска бинарного файла. Бинарная библиотека и исполняемый файл собираются в src/.libs/

```
# Патч для configure.ac
@@ 4,6 4,7 @@
 AC_PREREQ([2.69])
 AC_INIT([namegen], [0.0.1], [george@altlinux.org])
 AM_INIT_AUTOMAKE([foreign])
+LT_INIT([disable-static])
 AC_CONFIG_SRCDIR([src/namegen.c])
 AC_CONFIG_HEADERS([config.h])
```

```
# src/Makefile.am
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3

lib_LTLIBRARIES = libnamegen.la
bin_PROGRAMS = namegen

namegen_LDADD = libnamegen.la

libnamegen.c: const.h

const.h: tablegen.py
        python3 $< > $@

check: namegen
        @./namegen --test
```

Отметим компактность конструкций Makefile, задействующих libtool. Видно, что этот инструмент существенно упрощает настройку для сборки библиотеки.

Добавим в каталог tests простейшую программу, проверяющую подключение библиотеки.

```
# Патч для Makefile.am
@@ 1,4 1,4 @@
-SUBDIRS = src
+SUBDIRS = src tests

 demo:  all
        ./src/namegen '!BVCs !DdM'
```

```
# Патч для configure.ac
```

```
@@ 24,5 24,6 @@ AC_FUNC_MALLOC
 AC_CHECK_FUNCS([memset strchr])

 AC_CONFIG_FILES([Makefile
+                 tests/Makefile
                  src/Makefile])
 AC_OUTPUT
```

```
# tests/Makefile.am
TESTS = dummy
bin_PROGRAMS = dummy

AM_CFLAGS = -I$(top_builddir)/src
LDADD = $(top_builddir)/src/libnamegen.la
```

Простейшая тестовая программа:

```
// tests/dummy.c
#include <stdio.h>
#include <assert.h>
#include "namegen.h"

int main(int argc, char *argv[]) {
        assert(namegen != NULL);
        return 0;
}
```

Проверка подключения библиотеки заключается в том, что программа dummy.c успешно собирается.

Теперь подключим собственно инструмент check: добавим модуль check в конфигурацию Automake, а файл с тестами напишем с использованием возможностей check.

```
# Патч для configure.ac
@@ 11,6 11,9 @@ AC_CONFIG_HEADERS([config.h])
 # Checks for programs.
 AC_PROG_CC

+# Joint pkgconfig library/include check and variable definition.
+PKG_CHECK_MODULES([CHECK],[check])
+
 # Checks for libraries.

 # Checks for header files.
```

```
# tests/Makefile.am
TESTS = dummy
check_PROGRAMS = dummy

AM_CFLAGS = -I$(top_builddir)/src @CHECK_CFLAGS@
LDADD = $(top_builddir)/src/libnamegen.la @CHECK_LIBS@
```

Код с тривиальным тестом, использующим check:

```
// tests/dummy.c
```

```
#include <stdio.h>
#include <assert.h>
#include <check.h>
#include "namegen.h"

START_TEST(test_dummy) {
        assert(namegen != NULL);
}
END_TEST

int main(int argc, char *argv[]) {
        Suite *suite = suite_create("Dummy"); // создаём suite
        TCase *testcase = tcase_create("Dummy"); // создаём case
        SRunner *runner = srunner_create(suite); // для suite делаем runner
        int ret;

        suite_add_tcase(suite, testcase); // добавляем case в suite
        tcase_add_test(testcase, test_dummy); // добавляем test в case
        srunner_run_all(runner, CK_ENV); // запускаем все suite
        ret = srunner_ntests_failed(runner); // собираем информацию о тестах
        srunner_free(runner);
        return ret != 0;
}
```

Для построения даже простейшего теста требуется довольно много вызовов функций check. Сделаем описание теста более компактным, используя утилиту checkmk.

```
# tests/Makefile.am
TESTS = dummy
check_PROGRAMS = dummy

.ts.c:
        checkmk $< > $@

AM_CFLAGS = -I$(top_builddir)/src @CHECK_CFLAGS@
LDADD = $(top_builddir)/src/libnamegen.la @CHECK_LIBS@
```

Описание простейшего теста на входном языке check:

```
// tests/dummy.ts
#include <check.h>
#include "namegen.h"

#test dummy
        ck_assert_ptr_nonnull(namegen);
```

Реализуем авторские тесты для namegen средствами check:

```
# src/Makefile.am
CFLAGS  = -ansi -pedantic -Wall -Wextra -O3 -g3

lib_LTLIBRARIES = libnamegen.la
bin_PROGRAMS = namegen

namegen_LDADD = libnamegen.la

libnamegen.c: const.h
```

```
const.h: tablegen.py
        python3 $< > $@
```

```
# tests/Makefile.am
TESTS = dummy upstream
check_PROGRAMS = dummy upstream

.ts.c:
        checkmk $< > $@

AM_CFLAGS = -I$(top_builddir)/src @CHECK_CFLAGS@
LDADD = $(top_builddir)/src/libnamegen.la @CHECK_LIBS@
```

Основное описание тестов:

```
// tests/upstream.ts
#include <check.h>
#include "namegen.h"

int r;
char buf[256];
int count_pass = 0;
int count_fail = 0;
unsigned long rng[] = {0xd22bfb3b};

#suite Tools
#tcase Alternation
#test alternation_test
        int i;
        int mismatch = 0;
        int seen[3] = {0, 0, 0};
        for (i = 0; i < 100; i++) {
            memset(buf, 0x5a, sizeof(buf));
            r = namegen(buf, sizeof(buf), "0|1|2", rng);
            if (r != NAMEGEN_SUCCESS || buf[1] || !strchr("012", buf[0])) {
                mismatch++;
            } else {
                seen[buf[0] - '0']++;
            }
        }
        ck_assert_int_eq(mismatch, 0);
        ck_assert_int_ne(seen[0], 0);
        ck_assert_int_ne(seen[1], 0);
        ck_assert_int_ne(seen[2], 0);

#tcase Substitution
#test substitution_test
        int i;
        int mismatch = 0;
        int seen[6] = {0, 0, 0};
        for (i = 0; i < 100; i++) {
            memset(buf, 0x5a, sizeof(buf));
            r = namegen(buf, sizeof(buf), "(v)v", rng);
            if (r != NAMEGEN_SUCCESS || buf[0] != 'v' || !buf[1] || buf[2]) {
                mismatch++;
```

```
            } else {
                char *p = "aeiouy";
                char *s = strchr(p, buf[1]);
                if (!s)
                    mismatch++;
                else
                    seen[s - p]++;
            }
        }
        for (i = 0; i < 6; i++)
            mismatch += !seen[i];
        ck_assert_int_eq(mismatch, 0);

#suite Errors
#tcase bracket
#test closing_bracket_test
        r = namegen(buf, sizeof(buf), "a>", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#test mismatch_bracket_test
        r = namegen(buf, sizeof(buf), "(a>", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#test unclosed_bracket_test
        r = namegen(buf, sizeof(buf), "<a", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#tcase paren
#test closing_paren_test
        r = namegen(buf, sizeof(buf), "a)", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#test mismatch_paren_test
        r = namegen(buf, sizeof(buf), "<a)", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#test unclosed_paren_test
        r = namegen(buf, sizeof(buf), "(a", rng);
        ck_assert_int_eq(r, NAMEGEN_INVALID);

#suite Tools2
#tcase Deep_nesting
#test deep_nesting_test
        char *p = "(((((<((((((((((((((a)))))))))))))))>)))))";
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, sizeof(buf), p, rng);
        ck_assert_int_eq(r, NAMEGEN_SUCCESS);
        ck_assert_str_eq(buf, "a");

#tcase Truncation
#test truncation_1_test
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, 4, "(abcdefg)", rng);
        ck_assert_int_eq(r, NAMEGEN_TRUNCATED);
```

```
        ck_assert_str_eq(buf, "abc");
        ck_assert_uint_eq(buf[4], 0x5a);

#test truncation_2_test
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, 2, "i", rng);
        ck_assert_int_eq(r, NAMEGEN_TRUNCATED);
        ck_assert_uint_ne(buf[0], 0);
        ck_assert_uint_eq(buf[2], 0x5a);

#tcase Capitalization
#test capitalization_1_test
        r = namegen(buf, sizeof(buf), "!(foo)", rng);
        ck_assert_int_eq(r, NAMEGEN_SUCCESS);
        ck_assert_str_eq(buf, "Foo");

#test capitalization_2_test
        r = namegen(buf, sizeof(buf), "(!foo)", rng);
        ck_assert_int_eq(r, NAMEGEN_SUCCESS);
        ck_assert_str_eq(buf, "Foo");

#test capitalization_3_test
        r = namegen(buf, sizeof(buf), "(f!oo)", rng);
        ck_assert_int_eq(r, NAMEGEN_SUCCESS);
        ck_assert_str_eq(buf, "fOo");

#tcase Empty
#test empty_test
        memset(buf, 0x5a, sizeof(buf));
        r = namegen(buf, sizeof(buf), "", rng);
        ck_assert_int_eq(r, NAMEGEN_SUCCESS);
        ck_assert_uint_eq(buf[0], 0);
        ck_assert_uint_eq(buf[1], 0x5a);
```

Сборка и запуск:

```
alt@localhost:~/10_LibTesting/Lection/fantasy> make check
Making check in src
make[1]: Entering directory '/home/alt/10_LibTesting/Lection/fantasy/src'
PASS alternation
PASS substitution
PASS closing bracket
PASS closing paren
PASS mismatch paren
PASS mismatch bracket
PASS unclosed bracket
PASS unclosed paren
PASS deep nesting
PASS truncation 1
PASS truncation 2
PASS capitalization 1
PASS capitalization 2
PASS capitalization 3
PASS empty
```

```
All 15 tests passed
make[1]: Leaving directory '/home/alt/10_LibTesting/Lection/fantasy/src'
Making check in tests
make[1]: Entering directory '/home/alt/10_LibTesting/Lection/fantasy/tests'
make  check-TESTS
make[2]: Entering directory '/home/alt/10_LibTesting/Lection/fantasy/tests'
make[3]: Entering directory '/home/alt/10_LibTesting/Lection/fantasy/tests'
PASS: dummy
============================================================================
Testsuite summary for namegen 0.0.1
============================================================================
# TOTAL: 1
# PASS:  1
# SKIP:  0
# XFAIL: 0
# FAIL:  0
# XPASS: 0
# ERROR: 0
============================================================================
```

Тесты успешно выполнены и сформирован минималистичный отчет о тестировании. Чтобы добавить информативности, подключим утилиту gcov, которая оценит покрытие тестами путей в тестируемой программе.

```
# Makefile.am
SUBDIRS = src tests

demo:   all
        ./src/namegen '!BVCs !DdM'

checklog:       check
        cat tests/*.log

gcov:   check
        $(MAKE) $(MAKEFLAGS) -C src gcov
```

```
# configure.ac
#                                               -*- Autoconf -*-
# Process this file with autoconf to produce a configure script.

AC_PREREQ([2.69])
AC_INIT([namegen], [0.0.1], [george@altlinux.org])
AM_INIT_AUTOMAKE([foreign])
LT_INIT([disable-static])
AC_CONFIG_SRCDIR([src/namegen.c])
AC_CONFIG_HEADERS([config.h])

# Checks for programs.
AC_PROG_CC

# Joint pkgconfig library/include tests
PKG_CHECK_MODULES([CHECK],[check])
```

```
# Variable definitions
AC_SUBST(CK_VERBOSITY, verbose)
AC_ARG_VAR(CK_VERBOSITY, [Default: "verbose", can be "silent", "minimal" or
"normal")])
# Enabe/disable things
AC_ARG_ENABLE([gcov],
              [AS_HELP_STRING([--enable-gcov], [use Gcov to test the test
suite])],
              [], [enable_gcov=no])
AM_CONDITIONAL([COND_GCOV],[test '!' "$enable_gcov" = no])

# Checks for libraries.

# Checks for header files.
AC_CHECK_HEADERS([stddef.h stdlib.h string.h])

# Checks for typedefs, structures, and compiler characteristics.
AC_CHECK_TYPES([ptrdiff_t])

# Checks for library functions.
AC_FUNC_MALLOC
AC_CHECK_FUNCS([memset strchr])

AC_CONFIG_FILES([Makefile
                 tests/Makefile
                 src/Makefile])
AC_OUTPUT
```

```
# src/Makefile.am
CFLAGS = -ansi -pedantic -Wall -Wextra
if COND_GCOV
CFLAGS  += -O0 -g -fprofile-arcs -ftest-coverage
else
CFLAGS  += -O3 -g3
endif

lib_LTLIBRARIES = libnamegen.la
bin_PROGRAMS = namegen

namegen_LDADD = libnamegen.la

libnamegen.c: const.h

const.h: tablegen.py
        python3 $< > $@

gcov:
        gcov -o .libs libnamegen -t -k -b | egrep -v '[[:digit:]]:[ /][*]'
```

16.6 Домашнее задание

Перед выполнением задания следует выбрать и изучить фреймворк тестирования программ на Си. Это может быть Check или другой инструмент на ваш выбор.

С использованием этого фреймворка написать тесты для *Growable Memory Buffers for C99*[77]:

1. Превратить в библиотеку:
   - функция в ней всего одна, остальное макросы, и переделывать это не надо;
   - все макросы перемещаются в .h-файл;
2. *Приложение*-пример можно не писать;
3. Тесты взять из авторского файла tests.c, тесты на память с setjmp() *можно* отбросить;
4. Оформить тесты сообразно правилам фреймворка в несколько отдельных suite-ов. Необязательно: попробовать добиться как можно более полного покрытия (предельные тесты реализовать, например, с помощью *prlimit*[78]);
5. Сделать примитивную поддержку проверки покрытия (чтобы проценты показывало);
6. Создать в репозитории с домашними заданиями подкаталог 10_LibTesting и разместить в нем проект.

77 Growable Memory Buffers for C99 [Электронный ресурс]. URL: https://github.com/skeeto/growable-buf (дата обращения: 20.01.2025).

78 getrlimit(2) — Linux manual page [Электронный ресурс]. URL: http://man7.org/linux/man-pages/man2/prlimit.2.html (дата обращения: 20.01.2025).

17 Информационное пространство и документирование

Идентификатор темы для сайта: 11_Documenting

Информационное пространство позволяет взаимодействующим в рамках проекта и с проектом людям ориентироваться в нём, разбираться с доступным функционалом проекта и с его организацией.

Для разработчиков основное инфопространство занимают описания внутренних элементов проекта — используемые функции, методы, типы и значения переменных. Для сообщества и пользователей основной является информация о возможностях самого продукта и о правилах взаимодействия с ним. При поддержке продукта основную часть инфопространства заполняет описание и структуризация всех проводящихся изменений проекта.

Информационное пространство — это огромный пласт разработки, охватывающий каждую её часть. Далее сделаем упор на техническое документирование проекта.

17.1 Источники информации

Как в проекте собирается информация по его внутреннему и внешнему функционалу? Основной источник информации — документирование самих исходников. Его можно разбить на два основных вида:

- *Комментарии* — Для их осмысленного написания существует множество методик. Одна из классических — *Literate Programming*[79] — была в 1981 предложена Дональдом Кнутом. По этой методике для проекта пишется текст с описанием всех примеров, затем в него добавляются вставки на языке программирования;
- *Документирование API* — Очень важная часть документирования. В первую очередь продукт разрабатывается для пользователей, и для его успешного применения необходима информация о том, как им пользоваться (в противном случае пользоваться им не будут). Для библиотеки эта информация должна в первую очередь описывать программный интерфейс (API). Документирование API может поддерживаться внутренними средствами языка (например, в *Python* поддерживается автодокументирование функций за счёт размещения в коде строк специального формата) или с помощью систем синтаксического разбора кода, причём как универсальных (*Sphinx*[80], *Doxygen*[81]), так и ориентированных на конкретный язык программирования.

Не менее важным является обеспечение общего взаимодействия и унифицированное ведение документации в сообществе. Большую работу в этой сфере провёл *Po-*

79 Wikipedia: Грамотное программирование [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Грамотное_программирование (дата обращения: 20.01.2025).

80 Sphinx. Create intelligent and beautiful documentation with ease [Электронный ресурс]. URL: https://www.sphinx-doc.org/ (дата обращения: 20.01.2025).

81 Doxygen. Code Documentation. Automated [Электронный ресурс]. URL: https://www.doxygen.nl/ (дата обращения: 20.01.2025).

берт Мартин[82], выпустивший множество работ по дисциплине разработки, стилю оформления проектов, структуре каталогов, наименованию и назначению файлов (классический пример оформления — GNU Hello[83]).

Непосредственное оформление справочников с документацией также имеет свои особенности. Говоря об offline-справочниках, нельзя не упомянуть классические help и man, в которых подробно и без лишней информации описываются все основные параметры и результаты работы описываемых утилит. Ещё одним форматом справочника выступают разного рода генераты и гипертексты, полученные из документированных исходников. Систем документирования бесчисленное количество, каждая из них поддерживает разные форматы выходной документации, включая LaTeX, HTML и так далее. В online-документировании полученные справочники могут выкладываться самостоятельно с помощью сгенерированной HTML-документации, с помощью специализированных движков (wiki, Pelican[84] и т.д.), сервисов совместной разработки (README.md на Gitlab/Github, Github Pages на Github.io и др.) или с помощью специализированных сервисов (ReadTheDocs[85], Git Book[86], ...).

17.2 Doxygen

В данном курсе будем разбираться именно с этим документатором. Doxygen[87] — система с широким спектром возможностей и поддержкой нескольких языков программирования. Работает по принципу синтаксического анализатора — комментарии специального вида (форматные) размещаются прямо в коде, откуда считываются и обрабатываются системой. На выходе у Doxygen — документация в формате HTML (для сайта), PDF и т. д., включая описание API и диаграммы классов.

82 Wikipedia: Мартин, Роберт (инженер) [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Мартин%2C_Роберт_%28инженер%29 (дата обращения: 20.01.2025).

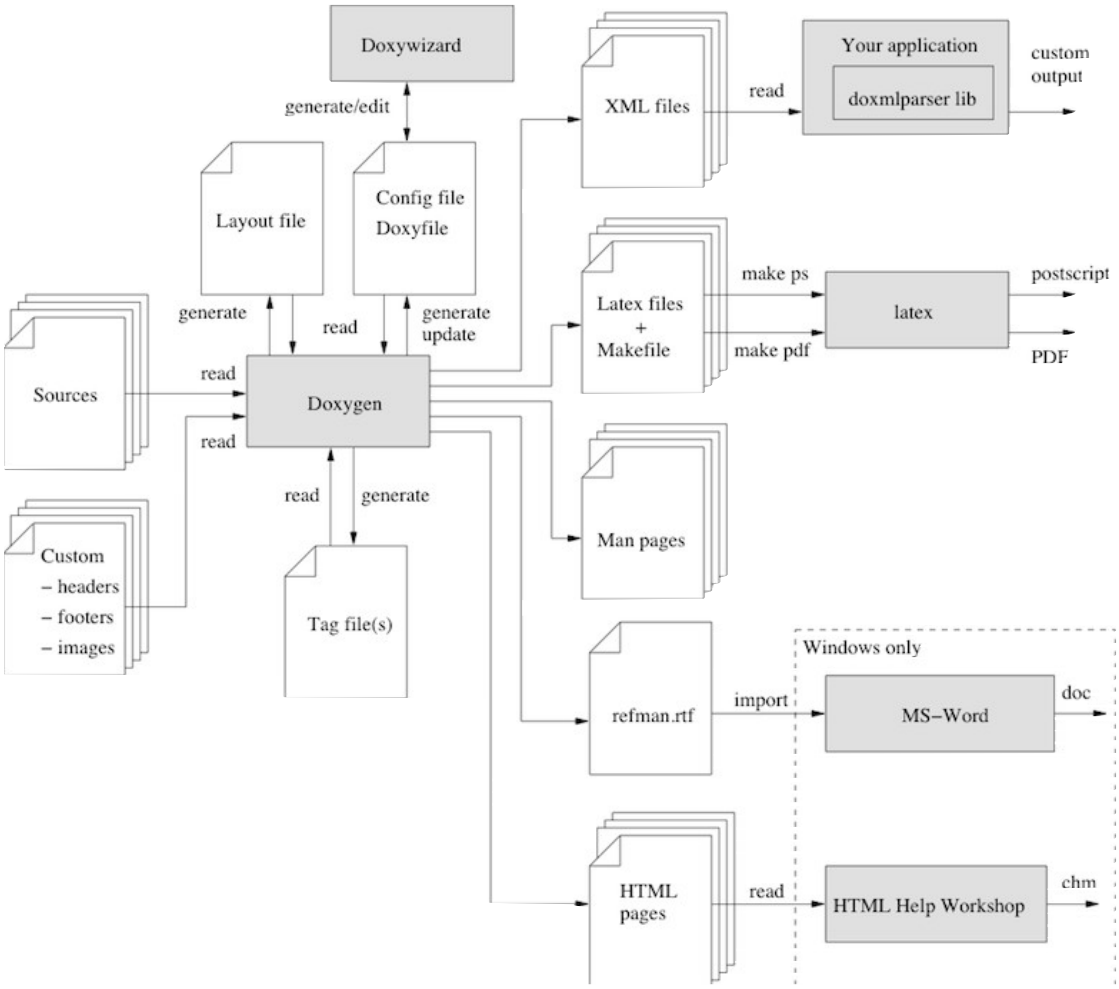
83 GNU Hello [Электронный ресурс]. URL: https://git.savannah.gnu.org/cgit/hello.git/tree/ (дата обращения: 20.01.2025).

84 Pelican. Static site generator powered by Python [Электронный ресурс]. URL: https://blog.getpelican.com/ (дата обращения: 20.01.2025).

85 Read the Docs. Documentation simplified [Электронный ресурс]. URL: https://readthedocs.com/ (дата обращения: 20.01.2025).

86 Gitbook. Build docs as a product [Электронный ресурс]. URL: https://www.gitbook.com/ (дата обращения: 20.01.2025).

87 Doxygen. Code Documentation. Automated [Электронный ресурс]. URL: https://www.doxygen.nl/index.html (дата обращения: 20.01.2025).



На рисунке[88] показана схема работы Doxygen, включая виды входных и выходных данных и связи между ними.

Процесс создания документации укладывается в привычный процесс автосборки проекта через configure. Для начала из исходных файлов сформируем шаблонные файлы для параметров документации, заполним их, укажем формат итоговой документации, затем соберем проект.

Рассмотрим пример с namegen из предыдущей темы, дополнив его документацией.

Для начала создадим шаблонный файл-описатель Doxyfile.in

```
alt@localhost:~/11_Documenting/Lection/namegen> doxygen -g Doxyfile.in

Configuration file 'Doxyfile.in' created.

Now edit the configuration file and enter

  doxygen Doxyfile.in

to generate the documentation for your project

alt@localhost:~/11_Documenting/Lection/namegen>
```

88 Getting started [Электронный ресурс]. URL: https://www.doxygen.nl/manual/starting.html (дата обращения: 20.01.2025).

```
alt@localhost:~/11_Documenting/Lection/namegen> cat Doxyfile.in | wc
   2884   21302  129104
```

Почти 3000 строк — в большинстве своём комментарии, описывающие параметры, которые можно менять. Вот некоторые поля из этого файла:

```
# Doxyfile.in
# The OUTPUT_DIRECTORY tag is used to specify the (relative or absolute) path
# into which the generated documentation will be written. If a relative path
is
# entered, it will be relative to the location where Doxygen was started. If
# left blank the current directory will be used.

OUTPUT_DIRECTORY       =

# If the CREATE_SUBDIRS tag is set to YES then Doxygen will create up to 4096
# sub-directories (in 2 levels) under the output directory of each output for-
mat
# and will distribute the generated files over these directories. Enabling
this
# option can be useful when feeding Doxygen a huge amount of source files,
where
# putting all generated files in the same directory would otherwise causes
# performance problems for the file system. Adapt CREATE_SUBDIRS_LEVEL to
# control the number of sub-directories.
# The default value is: NO.

CREATE_SUBDIRS         = NO

# Controls the number of sub-directories that will be created when
# CREATE_SUBDIRS tag is set to YES. Level 0 represents 16 directories, and ev-
ery
# level increment doubles the number of directories, resulting in 4096
# directories at level 8 which is the default and also the maximum value. The
# sub-directories are organized in 2 levels, the first level always has a
fixed
# number of 16 directories.
# Minimum value: 0, maximum value: 8, default value: 8.
# This tag requires that the tag CREATE_SUBDIRS is set to YES.

CREATE_SUBDIRS_LEVEL   = 8

# If the ALLOW_UNICODE_NAMES tag is set to YES, Doxygen will allow non-ASCII
# characters to appear in the names of generated files. If set to NO, non-
ASCII
# characters will be escaped, for example _xE3_x81_x84 will be used for Uni-
code
# U+3044.
# The default value is: NO.

ALLOW_UNICODE_NAMES    = NO

# The OUTPUT_LANGUAGE tag is used to specify the language in which all
# documentation generated by Doxygen is written. Doxygen will use this
# information to generate all constant output in the proper language.
```

```
# Possible values are: Afrikaans, Arabic, Armenian, Brazilian, Bulgarian,
# Catalan, Chinese, Chinese-Traditional, Croatian, Czech, Danish, Dutch, Eng-
lish
# (United States), Esperanto, Farsi (Persian), Finnish, French, German, Greek,
# Hindi, Hungarian, Indonesian, Italian, Japanese, Japanese-en (Japanese with
# English messages), Korean, Korean-en (Korean with English messages), Lat-
vian,
# Lithuanian, Macedonian, Norwegian, Persian (Farsi), Polish, Portuguese,
# Romanian, Russian, Serbian, Serbian-Cyrillic, Slovak, Slovene, Spanish,
# Swedish, Turkish, Ukrainian and Vietnamese.
# The default value is: English.

OUTPUT_LANGUAGE        = English

# If the BRIEF_MEMBER_DESC tag is set to YES, Doxygen will include brief mem-
ber
# descriptions after the members that are listed in the file and class
# documentation (similar to Javadoc). Set to NO to disable this.
# The default value is: YES.

BRIEF_MEMBER_DESC      = YES

# If the REPEAT_BRIEF tag is set to YES, Doxygen will prepend the brief
# description of a member or function before the detailed description
#
# Note: If both HIDE_UNDOC_MEMBERS and BRIEF_MEMBER_DESC are set to NO, the
# brief descriptions will be completely suppressed.
# The default value is: YES.

REPEAT_BRIEF           = YES
```

Добавим отсылки к Doxygen в файл настроек Autotools:

```
# configure.ac
#                                               -*- Autoconf -*-
# Process this file with autoconf to produce a configure script.

AC_PREREQ([2.69])
AC_INIT([Stupid fantasy name generator], [0.0.1], [george@altlinux.org],
[namegen])
AM_INIT_AUTOMAKE([foreign])
DX_INIT_DOXYGEN([namegen]) # внесли описание Doxygen
LT_INIT([disable-static])
AC_CONFIG_SRCDIR([src/namegen.c])
AC_CONFIG_HEADERS([config.h])

# Checks for programs.
AC_PROG_CC

# Joint pkgconfig library/include tests
PKG_CHECK_MODULES([CHECK],[check])

# Variable definitions
AC_SUBST(CK_VERBOSITY, verbose)
```

```
AC_ARG_VAR(CK_VERBOSITY, [Default: "verbose", can be "silent", "minimal" or
"normal")])
# Enabe/disable things
AC_ARG_ENABLE([gcov],
              [AS_HELP_STRING([--enable-gcov], [use Gcov to test the test
suite])],
              [], [enable_gcov=no])
AM_CONDITIONAL([COND_GCOV],[test '!' "$enable_gcov" = no])

# Checks for libraries.

# Checks for header files.
AC_CHECK_HEADERS([stddef.h stdlib.h string.h])

# Checks for typedefs, structures, and compiler characteristics.
AC_CHECK_TYPES([ptrdiff_t])

# Checks for library functions.
AC_FUNC_MALLOC
AC_CHECK_FUNCS([memset strchr])

AC_CONFIG_FILES([Makefile
                 tests/Makefile
                 src/Makefile
                 Doxyfile # добавили генерат
                 ])
AC_OUTPUT
```

Добавляем рецепты для Doxygen в Makefile.am:

```
#  Makefile.am
SUBDIRS = src tests

@DX_RULES@ # этот макрос притянет все правила документирования

all-local:      doxygen-doc

demo:   all
        ./src/namegen '!BVCs !DdM'

checklog:       check
        cat tests/*.log

gcov:   check
        $(MAKE) $(MAKEFLAGS) -C src gcov

http:   doxygen-doc # этот рецепт с помощью python запустит сервер с
документацией
        python3 -m http.server --directory $</html
```

В Doxyfile.in зададим значения параметров:

```
# Патч для Doxyfile.in
@@ -42,19 +42,19 @@ DOXYFILE_ENCODING      = UTF-8
 # title of most generated pages and in a few other places.
 # The default value is: My Project.
```

```
-PROJECT_NAME           = "@PACKAGE_TARNAME@"
+PROJECT_NAME           = "My Project"

 # The PROJECT_NUMBER tag can be used to enter a project or revision number.
This
 # could be handy for archiving the generated documentation or if some version
 # control system is used.

-PROJECT_NUMBER         = @PACKAGE_VERSION@
+PROJECT_NUMBER         =
```

Теперь добавим сами форматные комментарии, по которым Doxygen будет формировать документацию. Инструмент имеет поддержку и однострочных, и многострочных комментариев. Для разных языков программирования формат комментариев различается.

```
// libnamegen.c
/* Fantasy name generator ANSI C header library
 * This is free and unencumbered software released into the public domain.
 *
 */
#include "namegen.h"
#include "const.h"

/** @file libnamegen.c
 * Implementation
 *
 */

/** Calculate offset for @p c.
 *
 * @param c special character in from table
 * @return offsets table offset for @p c, or -1 if not special
 *
 * The return value is suitable for namegen_offsets().
 */
static int
namegen_special(int c)
{
    if (c >= 0 && c < 128)
        return special[c];
    return -1;
}

/** Calculate all offsets for @p n
 *
 * @param n special
 * @param offsets all offsets for @n buffer
 * @return offsets and number of offsets for special @p n.
 * @p offsets point into namegen_argz.
 */
static int
```

```
namegen_offsets(int n, const short **offsets)
{
    *offsets = offsets_table + off_len[n * 2 + 0];
    return off_len[n * 2 + 1];
}

static unsigned long
namegen_rand32(unsigned long *s)
{
    unsigned long x = *s;
    x ^= x << 13;
    x ^= (x & 0xffffffffUL) >> 17;
    x ^= x << 5;
    return (*s = x) & 0xffffffffUL;
}

/** Capitalize character @p c.
 *
 * @param c character
 * @param capitalize where capitalization is needed
 */
static int
namegen_cap(int c, int capitalize)
{
    return capitalize && c >= 'a' && c <= 'z' ? c & ~0x20 : c;
}

/** Copy a random substitution for template @p c into @p p, but only before @p
e.
 *
 * @param p destinatiornb buffer
 * @param e stop pointer in template
 * @param c source template buffer
 * @param seed random seed
 * @param capitalize where to capitalize substitution
 * @return destination buffer @p
 */
static char *
namegen_copy(char *p, char *e, int c, unsigned long *seed, int capitalize)
{
    int n = namegen_special(c);
    if (n == -1) {
        if (p != e)
            *p++ = namegen_cap(c, capitalize);
    } else {
        const short *offsets;
        int count = namegen_offsets(n, &offsets);
        int select = namegen_rand32(seed) % count;
        const char *s = namegen_argz + offsets[select];
        while (*s) {
            int r = *s++;
            if (p != e)
                *p++ = namegen_cap(r, capitalize);
            capitalize = 0;
```

```
        }
    }
    return p;
}

/** Generate a name into @p dst of @p len bytes from @p pattern and using @p
seed.
 *
 * The length must be non-zero. For best results, the lower 32 bits of
 * the seed should be thoroughly initialized. A particular seed will
 * produce the same results on all platforms.
 *
 * @param dst buffer to put generated name to
 * @param len buffer length (in bytes)
 * @param pattern name pattern
 * @param seed random seed
 *
 * The return value is one of the above codes, indicating success or
 * that something went wrong. Truncation occurs when @p dst was too short.
 * Pattern is validated even when the output has been truncated.
 *
 * @return error code
 */
int
namegen(char *dst, unsigned long len, const char *pattern, unsigned long
*seed)
{
    int depth = 0;               /* Current nesting depth */
    char *p = dst;               /* Current output pointer */
    char *e = dst + len;         /* Maxiumum output pointer */
    int capitalize = 0;          /* Capitalize next item */
    //
}
```

```
// namegen.h
#ifndef NAMEGEN_H
#define NAMEGEN_H

#define NAMEGEN_MAX_DEPTH  32 /**< Cannot exceed bits in a long */

/* Return codes */
#define NAMEGEN_SUCCESS    0
#define NAMEGEN_TRUNCATED  1  /**< Output was truncated */
#define NAMEGEN_INVALID    2  /**< Pattern is invalid */
#define NAMEGEN_TOO_DEEP   3  /**< Pattern exceeds maximum nesting depth */

int namegen(char *, unsigned long, const char *, unsigned long *);

#endif
```

В программе на Python форматные комментарии выглядят иначе:

```
#!/usr/bin/env python3
# tablegen.py
```

```
## @package tablegen
# This script generates the lookup tables in namegen.h. To change those
# tables, modify the symbols dictionary in this script, execute it to
# generate the new tables, then replace the tables in the C source.

Symbols = {
#
}

# Форматные комментарии

## Concatenation of all strings, including null terminators
argz = []
## Offset into argz of the Nth string
offsets = []
## Offset/length into offsets of the Nth special character
off_len = []
## ASCII table of offsets into off_len
special = []

offset = 0
keys = sorted(symbols.keys())
for c in keys:
    options = symbols[c]
    off_len.append(len(argz))
    off_len.append(len(options))
    for option in options:
        offsets.append(offset)
        offset = offset + len(option) + 1
        argz.append(option)

for c in range(0, 128):
    try:
        special.append(keys.index(chr(c)))
    except ValueError:
        special.append(-1)

# Prints a C array initializer
def dump(values, fmt, width):
    for i, v in enumerate(values):
        if i > 0:
            print(',', end='')
        if i % width == 0:
            print('\n    ', end='')
        else:
            print(' ', end='')
        print(fmt(v), end='');
    print('\n};\n')

print('static const signed char special[] = {', end='')
dump(special, lambda v: '  -1' if v < 0 else '0x%02x' % v, 8)
```

namegen 0.0.1

Stupid fantasy name generator

Main Page | Related Pages | Namespaces | Files | Search

Functions | Variables

tablegen Namespace Reference

This script generates the lookup tables in **namegen.h**. More...

Functions

dump (values, fmt, width)

Variables

dict **symbols**

list **argz** = []
Concatenation of all strings, including null terminators.

list **offsets** = []
Offset into argz of the Nth string.

list **off_len** = []
Offset/length into offsets of the Nth special character.

list **special** = []
ASCII table of offsets into off_len.

int **offset** = 0

keys = sorted(symbols.keys())

dict **options** = **symbols**[c]

end

```
print('static const short offsets_table[] = {', end='')
dump(offsets, lambda v: '0x%04x' % v, 8)

print('static const short off_len[] = {', end='')
dump(off_len, lambda v: '0x%04x' % v, 8)

print('static const char namegen_argz[] = {', end='')
dump('\0'.join(argz) + '\0', lambda v: ' 0 ' if v == '\0' else "'%c'" % v, 15)
```

Генерируемая Doxygen документация в формате HTML выглядит так, как показана на снимках экрана ниже. Документация содержит гиперссылки, а также (при предустановленной утилите *Graphviz*[89]) диаграммы зависимостей между файлами исходных текстов.

89 Graphviz home page [Электронный ресурс]. URL: https://graphviz.org/ (дата обращения: 20.01.2025).

namegen 0.0.1

Stupid fantasy name generator

Main Page | Related Pages | Namespaces | Files | Search

src

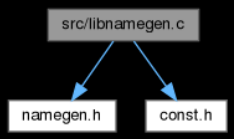
Functions

libnamegen.c File Reference

Implementation. More...

```
#include "namegen.h"
#include "const.h"
```

Include dependency graph for libnamegen.c:



Functions

static int	**namegen_special** (int c) Calculate offset for c.
static int	**namegen_offsets** (int n, const short **offsets) Calculate all offsets for n.
static unsigned long	**namegen_rand32** (unsigned long *s)
static int	**namegen_cap** (int c, int capitalize) Capitalize character c.
static char *	**namegen_copy** (char *p, char *e, int c, unsigned long *seed, int capitalize) Copy a random substitution for template c into p, but only before e.
int	**namegen** (char *dst, unsigned long len, const char *pattern, unsigned long *seed) Generate a name into dst of len bytes from pattern and using seed.

Документация по исходным текстам и программному интерфейсу сформирована, но главная страница документации пустует. Добавим для некоторых комментари-

ев метку @mainpage, чтобы заполнить главную страницу. Также настроим запуск HTTP-сервера, чтобы сделать документацию доступной извне. Соответствующий рецепт указываем в Makefile.am

Главная страница документации теперь выглядит так:

namegen 0.0.1

Stupid fantasy name generator

Main Page | Related Pages | Namespaces | Files | Search

namegen

Fantasy name generator library

This is free and unencumbered software released into the public domain.

The compile() function creates a name generator based on an input pattern. The letters s, v, V, c, B, C, i, m, M, D, and d represent different types of random replacements. Everything else is emitted literally.

- s - generic syllable
- v - vowel
- V - vowel or vowel combination
- c - consonant
- B - consonant or consonant combination suitable for beginning a word
- C - consonant or consonant combination suitable anywhere in a word
- i - insult
- m - mushy name
- M - mushy name ending
- D - consonant suited for a stupid person's name
- d - syllable suited for a stupid person's name (begins with a vowel)

All characters between parenthesis ( ) are emitted literally. For example, the pattern "s(dim)", emits a random generic syllable followed by "dim".

Characters between angle brackets <> emit patterns from the table above. Imagine the entire pattern is wrapped in one of these.

In both types of groupings, a vertical bar | denotes a random choice. Empty groups are allowed. For example, "(foo|bar)" emits either "foo" or "bar". The pattern "<c|v|>" emits a constant, vowel, or nothing at all.

An exclamation point ! means to capitalize the component that follows it. For example, "!(foo)" will emit "Foo" and "v!s" will emit a lowercase vowel followed by a capitalized syllable, like "eRod".

This library is based on the RinkWorks Fantasy Name Generator and subsequent Christopher Wellons Fantasy Name Generator header library

Generated by doxygen 1.12.0

```
Makefile.am
SUBDIRS = src tests

@DX_RULES@

all-local:      doxygen-doc

install-data-local: # вот он
        mkdir -p $(DESTDIR)/$(docdir)
        cp -a doxygen-doc/html $(DESTDIR)/$(docdir)

demo:   all
        ./src/namegen '!BVCs !DdM'
```

```
checklog:       check
        cat tests/*.log

gcov:   check
        $(MAKE) $(MAKEFLAGS) -C src gcov

http:   doxygen-doc
        python3 -m http.server --directory $</html 5000
```

Добавим генерацию документации в других форматах, например, *man*[90]. Документацию в этом формате можно формировать при помощи различных инструментов (например, *troff*[91], txt2man, *scdoc*[92]), но мы воспользуемся Doxygen.

Для начала включим параметр в Doxyfile.in

```
# Патч для Doxyfile.in
@@ -2226,7 +2226,7 @@ RTF_EXTRA_FILES        =
 # classes and files.
 # The default value is: NO.

-GENERATE_MAN           = NO
+GENERATE_MAN           = YES

 # The MAN_OUTPUT tag is used to specify where the man pages will be put. If a
 # relative path is entered the value of OUTPUT_DIRECTORY will be put in front
of
@@ -2244,7 +2244,7 @@ MAN_OUTPUT             = man
 # The default value is: .3.
 # This tag requires that the tag GENERATE_MAN is set to YES.

-MAN_EXTENSION          = .3
+MAN_EXTENSION          = .1
```

Внесём man-информацию в файл с кодом:

```
namegen.c
#include "config.h"
#include "namegen.h"
#include <time.h>
#include <stdio.h>
#include <stddef.h>
#include <stdlib.h>
#include <string.h>

static unsigned long
hash32(unsigned long a)
{
    a = a & 0xffffffffUL;
    a = (a ^ 61UL) ^ (a >> 16);
    a = (a + (a << 3)) & 0xffffffffUL;
```

90 man(1) — Linux manual page [Электронный ресурс]. URL: https://man7.org/linux/man-pages/man1/man.1.html (дата обращения: 20.01.2025).

91 Writing manual pages [Электронный ресурс]. URL: https://liw.fi/manpages/ (дата обращения: 20.01.2025).

92 Scdoc. A simple man page generator [Электронный ресурс]. URL: https://git.sr.ht/~sircmpwn/scdoc (дата обращения: 20.01.2025).

```
    a = a ^ (a >> 4);
    a = (a * 0x27d4eb2dUL) & 0xffffffffUL;
    a = a ^ (a >> 15);
    return a;
}

/*Добавили man*/

/** @page namegen
 * Fantasy name generator
 * @section SYNOPSIS
 * `namegen` \a pattern [ \a number ]
 * @section DESCRIPTION
 * Generate a fantasy name (or a \a number of names, if \a number is provided)
according to \a pattern.
 *
 * @copydetails library
 */
int
main(int argc, char **argv)
{
//
}
```

Добавим в Makefile.am рецепт по генерации man:

```
# Makefile.am
SUBDIRS = src tests

@DX_RULES@

all-local:      doxygen-doc

doxygen-doc/man/man1/namegen.1: doxygen-doc

man1_MANS = doxygen-doc/man/man1/namegen.1

install-data-local:
        mkdir -p $(DESTDIR)/$(docdir)
        cp -a doxygen-doc/html $(DESTDIR)/$(docdir)

demo:   all
        ./src/namegen '!BVCs !DdM'

checklog:       check
        cat tests/*.log

gcov:   check
        $(MAKE) $(MAKEFLAGS) -C src gcov

http:   doxygen-doc
        python3 -m http.server --directory $</html 5000
```

Проверим, что страница man сформирована и выводится:

```
alt@localhost:~/11_Documenting/Lection/namegen> man ./doxygen-doc/man/man1/
     namegen.c.1


src/namegen.c(1)            General Commands Manual            src/
     namegen.c(1)

NAME
       src/namegen.c

SYNOPSIS
       #include 'config.h'
       #include 'namegen.h'
       #include <time.h>
       #include <stdio.h>
       #include <stddef.h>
       #include <stdlib.h>
       #include <string.h>

   Functions
       static unsigned long hash32 (unsigned long a)
       int main (int argc, char **argv)

Function Documentation
   static unsigned long hash32 (unsigned long a) [static]
   int main (int argc, char ** argv)
Author
       Generated automatically by Doxygen for namegen from the source code.

namegen                          Version 0.0.1                 src/
     namegen.c(1)
```

17.3 Домашнее задание

1. За основу взять программу для угадывания чисел из предыдущего домашнего задания;
2. Дописать к ней две функции — перевод из римской системы счисления и обратно;
   - Поскольку диапазон 1...100, проще всего сделать табличку на 100 элементов при помощи *roman*[93]);
3. Научить саму программу принимать из командной строки параметр -r для работы в римской системе счисления;
4. Задокументировать эти функции (и другие, если они есть), а также макросы и глобальные переменные (которые сочтёте нужными);
5. Обеспечить программе вменяемый —help (на двух языках) и man (на одном);
6. Заполнить этим help-ом титульную страницу документации.

Необходимые пояснения:

93 Roman. Integer to Roman numerals converter [Электронный ресурс]. URL: https://pypi.python.org/pypi/roman (дата обращения: 20.01.2025).

1. Титульную страничку придётся генерировать чем-то боле сложным, чем LC_ALL=C ./number-game --help. Например, научить саму программу чему-то вроде --help-md (в который она подставляет команды Doxygen) или обработать --help чем-то ещё.
2. В крайнем случае можно скопировать текст. Для учебного примера допустимо, что титульная страница не будет генератом. В реальной разработке она со временем начнёт расходиться с --help и man.
3. (необязательно, для тех, кто хочет поупражняться программировать на Си) Увеличить диапазон до 1...3999.

18 Установка и пакетирование

Идентификатор темы для сайта: 12_InstallPackaging

В рамках данной темы рассмотрим принятые в Linux принципы установки (инсталляции) программного обеспечения и его пакетирования (в смысле оформление дистрибутива по правилам GNU).

18.1 Понятие установки

В результате разработки программного обеспечения создается проект, состоящий из множества связанных элементов. Теоретически, для того, чтобы поделиться созданной утилитой, мы можем разместить каталог с исходниками в публичном репозитории и оставить все остальные действия для непосредственных пользователей. В такой ситуации отсутствует понятие *установки* как таковой: пользователь должен сам не только скачать архив с исходными файлами, но и скомпилировать их, собрать проект (пусть и с помощью автосборочных команд типа configure/make); в процессе сборки может оказаться, что у пользователя не хватает библиотек для работы, не работают какие-то связи из-за неправильной сборки. Поэтому для использования программы пользователю придётся самому редактировать конфигурационные файлы сборки и, возможно, даже исходные тексты программы.

Установка же должна обеспечить работу «из коробки» (или «почти из коробки») уже собранных бинарников, распределённых по системе в процессе установки. Для распределения файлов по системе для корректной их работы в UNIX-подобных системах введён стандарт иерархии файлов в файловой системе (*Filesystem_Hierarchy_Standard*[94], FHS), на которую настроена работа операционной системы.

18.2 Варианты установки

Рассмотрим основные используемые в Linux варианты установки проектов в систему.

Классический вариант — *установка в полном соответствии с FHS* данных проекта в системные папки /usr/bin, /usr/lib64 и т.д. Этот способ гарантирует работу всех систем, и, собственно, потому и является классическим, поскольку работа со всеми системными утилитами настроена под именно такое распределение файлов. Однако данный способ имеет одну большую проблему — установка в системные папки требует прав root при работе, а это, потенциально, даёт полный доступ к любым вторжениям в систему со стороны устанавливаемого ПО. И только разработчику известно, что установит или удалит в системе его make install (*знаменитое удаление французского языка из системы*). Кроме того, даже рабочее и безопасное ПО может конфликтовать с уже установленным ПО дистрибутива.

Для избежания конфликтов с системой появился ещё один метод установки — *в специальные выделенные каталоги* /usr/local/bin, /usr/local/lib64 и т. д. Такая установ-

94 Wikipedia: Filesystem Hierarchy Standard [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Filesystem_Hierarchy_Standard (дата обращения: 20.01.2025).

ка, вообще говоря, не соответствует FHS, она также требует root-прав для установки и, кроме того, вероятность конфликтов нового ПО с уже установленным в эти каталоги более вероятен. Но данный способ не конфликтует с базовыми компонентами операционной системы и поэтому используется в больших системах, подразумевающих установку огромного количества пакетов.

Для установки проекта без прав root-а располагать файлы можно *в домашнем каталоге пользователя*, точнее — в подкаталогах $HOME/.local/. Такие установки являются локальными и доступными каждому пользователю по отдельности, отчего в системе могут происходить дублирующиеся установки одних и тех же приложений. Кроме того, «дополнительные действия» программ из-за зависимостей от локальных установок каждого пользователя могут не заработать. Такой способ установки полезен для систем, ориентированных на разработчика (поскольку для именно разработки требуется отдельное, а иногда и не одно, независимое пространство установки), а вот для пользователей работа с таким относительно изолированным пространством, для которого надо отдельно прописать все связи файлов, будет проблематичной.

Гарантированно неконфликтующий ни с какими утилитами способ установки — *под правами root в каталог /opt/<приложение>/*. Из минусов такой установки — нужно явно указывать в $PATH /bin-каталог программы, чтобы запускать ее, а также проводить дополнительную настройку при старте (подтягивание всех зависимостей из нестандартных расположений), обычно — через shell-сценарии. Такая схема используется в сложных проектах, требующих подтягивания с собой разных библиотек и исполняемых файлов.

Все предыдущие схемы предполагали работу с динамическими библиотеками. Однако иногда удобнее в небольших проектах описывать статические зависимости от специфичных элементов. В таком случае используется *гибридная схема*, сочетающая статическую компоновку с частью библиотек и одну из вышеперечисленных схем для учета остальных зависимостей. Гибридная схема вносит все проблемы статической компоновки (как минимум, огромные размеры итогового исполняемого файла), а также такая схема может вызвать проблемы версионирования между статически подключенными библиотеками и остальными библиотеками, подключенными динамически.

Самая прямолинейная и независимая ни от чего стратегия — *полная статическая компоновка бинарного исполняемого файла* и установка его. Такой бинарник не требует никаких прав root, библиотек-зависимостей (и правильной их установки), поскольку буквально заработает из любого места системы. Статическая компоновка подходит для либо очень больших, либо очень маленьких приложений, которые должны работать везде, лишь бы ядро ОС могло их исполнять. Кроме того, такой схемой можно удобно собирать разные версии приложений. Важное уточнение: статическая компоновка не решает проблемы обращения к внешним файлам данных, которые установленная программа может «ожидать» расположенными по конкретным путям (например: файлы перевода, иконки графического интерфейса).

18.3 Поддержка установки при сборке

Существует несколько способов описания установки при сборке проекта.

Классический способ — вручную описать процедуру установки. Такой способ часто используется для сборки статического бинарника.

Можно воспользоваться параметрами autotools для описания параметров установки:

- Указывается макрос каталога итогового расположения данных $DESTDIR;
- Используются ключи configure для тонкой настройки выбора установочных данных;
- Статическая сборка требует сноровки при описании задаваемых параметров.

Для тестовой установки после локальной сборки хорошо подойдут сценарии из libtool, поддерживающий описание процедуры установки за счёт .la-сценариев и т. д.

18.4 Введение в пакетирование

Чтобы начать работу с готовым продуктом, следует установить его любым из указанных выше методов. Однако при разработке под Linux самым правильным способом подготовки приложения к установке пользователем является *создание пакета*.

Пакетирование как способ работы с проектами обязан во многом *Свободному Лицензированию и Открытой разработке*. Именно они позволяют создавать огромные неконфликтующие инфраструктуры пакетов, основываясь на открытом исходном коде собираемых компонентов системы, для которой пакет предназначается.

Сам *пакет* — это архив с составляющими исходниками и/или бинарными файлами, которые раскладываются по основным каталогам системы (под root-ом). Предшествующая этому проверка данных и сборка бинарных файлов из исходников, а также предустановочные сценарии, выполняются без прав root-а.

Для проверки корректности сборки пакета используются специальные *репозитории пакетов*. Туда отправляются исходники, а все сборки проводятся в изолированном пространстве репозитория. Это помогает исключить случай работоспособности пакета только в специфичном окружении разработчика. Все данные подписываются (исходники — разработчиком, результаты сборки — сборщиком), чтобы гарантировать безопасность работы с исходниками и исключить подмену данных при обработке. Также к собственно процедуре сборки добавляются встроенные проверки на права доступа, работоспособность собранного проекта.

Множество пакетов объединяется в *дистрибутивы* — срезы из общего множества пакетов работающих наборов, корректно взаимодействующих друг с другом и обеспечивающих функциональность целой системы (приложения, операционной системы и т. д.)

18.5 Работа системы с пакетами

Пакет представляет собой архив данных, предназначенный для распространения в составе *дистрибутива* — рабочего набора установочных программ из *репозитория пакетов*, оформленного в виде комплекта для распространения и установки ОС. Дистрибутивом иногда называют саму ветку репозитория, с которой делается этот «срез». Помимо самих файлов и правил раскладывания их по системе при установке, в пакет входит информация о типах файлов. В зависимости от того, какую роль исполняет тот или иной элемент пакета, с ним необходимо поступать по-разному. В случае с бинарниками нужно производить замену старой версии на новую. Если же файл является конфигурационным, то правильнее будет сохранить предыдущую его

версию при установке новой, или же внедрить пользовательские изменения старой версии в новую.

Чтобы сохранять целостность системы при работе с пакетами, каждый устанавливаемый пакет регистрируется в системе с указанием всех своих зависимостей, файлов, мест установки и т.д. Соответственно при удалении пакета система сама очищает все данные, возвращает системные данные в предыдущее состояние (если происходит откат к прежней версии пакета: например, возвращает старые сохранённые конфигурационные файлы) и дерегистрирует пакет. Также в пакетах передаются сценарии пред- и постустановки, выполняющие какие-либо системные функции. Среди них есть автоматически запускающиеся (например, добавление в систему информации об иконке устанавливаемого приложения, установление доступа к новой странице man), а также специализированные сценарии, требующие явного указания к исполнению при установке.

Кроме вышеперечисленного, пакет хранит все инструкции по сборке приложения из содержимого пакета. Эти инструкции могут быть достаточно нетривиальными. Деятельность разработчика, создающего пакет из исходников и конфигурационных файлов сборки, заключается в переводе сборки из первоначального вида в тот, что поддерживается сообществом, для которого разрабатывается пакет.

18.6 Сборка пакета при помощи hasher

Сборка пакета для включения в дистрибутив выполняется средствами репозитория. Как правило, это делает один из участников соответствующего сообщества. Перед отправкой пакета в репозиторий, нам (разработчику программы) необходимо подготовить сборочное окружение, чтобы проверить пакет.

Для создания изолированного сборочного окружения воспользуемся разработанным командой *ALT Linux Team*[95] инструментом hasher. Этот инструмент обращается к репозиторию, собирает необходимый минимум данных для сборки, распаковывает их и собирает пакет в изолированной среде.

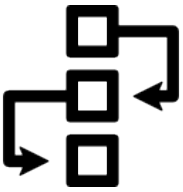
Hasher имеет следующие особенности:

- Окружение собирается на основе снимков (снапшотов, срезов веток разработки) репозитория, что позволяет собирать пакеты на любых его версиях и тестировать их самостоятельно друг от друга;
- hasher позволяет изолировать сборку даже от подключения к сети, что помогает проводить более тщательное тестирование самого продукта, на которое точно не повлияют внешние факторы;
- hasher собирает лишь необходимый минимум, необходимый для работы и тестирования продукта, избегая ненужных зависимостей;
- hasher имитирует режим суперпользователя внутри изолированной среды, что позволяет проводить с его помощью тестирование итогового продукта, как будто он уже установлен в систему и имеет к ней полный доступ. На самом же деле все root-команды лишь вносятся в журнал и транслируются, как выполненные.

95 ALT Linux Team [Электронный ресурс]. URL: https://www.altlinux.org/ALT_Linux_Team (дата обращения: 20.01.2025).

Для проверки и сборки пакета возьмём репозиторий утилиты *nfcollect*[96], пишущей информацию о работе портов в специальном формате, а также умеющей преобразовывать информацию из этого формата в доступную для чтения базу данных.

Процедура работы с hasher следующая:



1. Сделать git clone проекта и развернуть его в hasher-е. Мы специально выбрали проект со знакомым нам autotools (configure.ac и Makefile.am указывают на это), соответственно, первое, нужно сделать с проектом, это попробовать собрать его и проверять работоспособность.
2. Важное правило при проверке будущего пакета — искать в логах работы configure информацию на необнаруженные данные (строки вида check <...> ...no). Это поможет найти неучтённые автором репозитория зависимости или выявить проблемы сборки. В нашем случае обнаружена неучтённая зависимость от sqlite и нескольких других библиотек. Выполним установку их devel-библиотек через hsh-install.
3. В общем случае мы не можем гарантировать безошибочную сборку проекта из репозитория, поскольку какие-то её особенности могли зависеть от конкретной системы разработчика (например, у разработчика были установлены отсутствующие в перечне зависимостей библиотеки, и проблема не проявилась). В таком случае необходимо провести патчирование исходного кода для будущего пакета. В нашем случае всё собралось корректно.
4. После того как проект собрался из пакета, необходимо проверить работоспособность проекта. Правильным было бы собрать пакет, попробовать его поставить и проверить, заработает ли он. Однако так как проект небольшой (всего два бинарника), просто попробуем явно их запустить и протестировать. Здесь открывается особенность программы: в коде есть фиксированная константа, которая записывает сообщения в память только при накоплении 10922 сообщений. Для проверки работы заменим константу на меньшее значение, но не просто поменяем значение в коде, а сделаем ключ для уменьшения значения. Чтобы оставить сделанное изменение в итоговом пакете, необходимо будет его добавить в рецепт создания пакета.

Теперь попробуем сами сделать пакет из авторских исходников. Для этого скачиваем последний релиз проекта (необходимо из Git-репозитория получить последнюю версию ветки). Для описания паспорта пакета, а также правил сборки и установки его в систему, необходимо заполнить .spec-файл.

```
nfcollect.spec
Name:        nfcollect
Version:     0.2
Release:     alt1
Summary:     Collect Netfilter NFLOG log entries and commit them to stable
storage
Source:      %name-%version.tar.gz
Patch0:      <...>
Patch1:      <...>
URL:         https://github.com/yunchih/nfcollect
Group:       Networking/Other
License:     MIT
```

96 Nfcollect home page [Электронный ресурс]. URL: https://github.com/yunchih/nfcollect (дата обращения: 20.01.2025).

```
BuildRequires: <...>

%description
Collect packets from _Netfilter_ netlink kernel interface. Packets are aggre-
gated onto a memory region (we call it _a trunk_), until the _trunk_ is full.
A full _trunk_ will be committed to disk by configurable means (cur-
rently `zstd` compression and no compression is implemented). Trunks will be
stored in a specific directory, which will be scanned by `nfextract` to ex-
tract all trunks.

%prep # Секция подготовки
%setup
%patch0 -p1
%patch1 -p1

%build # секция сборки
%autoreconf
%configure
%make_build

%install # секция установки
%makeinstall
```

Разберем основные составляющие файла. Первые позиции — это паспорт пакета. В него входят имя, версия сборки, номер и индекс релиза, краткое описание пакета; источник данных (архив исходников) и наши патчи, которые мы хотим применить к ним; ссылка на репозиторий, принадлежность к группе пакетов, а также лицензия.

Пакеты-зависимости следует не устанавливать вручную, а подтягивать в среду сборки автоматически, указав в описании сборочного окружения. Для этого в поле BuildRequires указываем все требуемые библиотеки.

Далее описываются секции и макросы для установки данных. В секции %description описывается более подробно функционал пакета, секция %prep задаёт подготовительные действия над пакетом (в нашем примере — разархивирование и применение патчей), в секциях %build и %install описываются процедуры сборки и установки соответственно. Обработчики .spec-файлов пакетных менеджеров умеют работать с различными системами автосборки и поддерживают множество макросов для упрощения записи рецептов. Вот, например, некоторые из них:

```
alt@localhost:~> rpm --eval %autoreconf
autoreconf -fisv
alt@localhost:~> rpm --eval %configure
  CONFIG_SHELL="${CONFIG_SHELL:-/usr/bin/bash}" ; export CONFIG_SHELL ;
  CFLAGS="${CFLAGS:--O2 -g -m64 -fmessage-length=0 -D_FORTIFY_SOURCE=2 -fs-
     tack-protector -funwind-tables -fasynchronous-unwind-tables}" ; export
     CFLAGS ;
  CXXFLAGS="${CXXFLAGS:--O2 -g -m64 -fmessage-length=0 -D_FORTIFY_SOURCE=2 -
     fstack-protector -funwind-tables -fasynchronous-unwind-tables}" ; export
     CXXFLAGS ;
  FFLAGS="${FFLAGS:--O2 -g -m64 -fmessage-length=0 -D_FORTIFY_SOURCE=2 -fs-
     tack-protector -funwind-tables -fasynchronous-unwind-tables }" ; export
     FFLAGS ;
```

```
  FCFLAGS="${FCFLAGS:--O2 -g -m64 -fmessage-length=0 -D_FORTIFY_SOURCE=2 -fs-
     tack-protector -funwind-tables -fasynchronous-unwind-tables }" ; export
     FCFLAGS ;
  LDFLAGS="${LDFLAGS:-}" ; export LDFLAGS;
  ./configure --host=x86_64-suse-linux --build=x86_64-suse-linux \
        --program-prefix= \
        --disable-dependency-tracking \
        --prefix=/usr \
        --exec-prefix=/usr \
        --bindir=/usr/bin \
        --sbindir=/usr/sbin \
        --sysconfdir=/etc \
        --datadir=/usr/share \
        --includedir=/usr/include \
        --libdir=/usr/lib64 \
        --libexecdir=/usr/libexec \
        --localstatedir=/var \
        --sharedstatedir=/var/lib \
        --mandir=/usr/share/man \
        --infodir=/usr/share/info
alt@localhost:~>
alt@localhost:~> rpm --eval %makeinstall
make DESTDIR="/home/alt/rpmbuild/BUILDROOT/%{NAME}-%{VERSION}-%
     {RELEASE}.x86_64" install
```

Сборка пакета проводится с помощью пакетного менеджера с указанием .spec-файла:

```
alt@localhost:~> rpmbuild -ba RPM/SPECS/nfcollect.spec
```

18.7 Домашнее задание

Сделать полностью укомплектованный проект: взять одно из домашних заданий и оснастить его, в произвольном невырожденном объёме, следующими возможностями:

1. Адаптация к окружению (autotools или аналоги);
2. Перевод сообщений;
3. Unit-тесты;
4. Manpage;
5. Генерация документации из исходников;
6. Установка в систему от root-а и удаление оттуда (лучше сначала потренироваться на установке от пользователя в /tmp/qq);
   - Установленная программа должна работать в двух локалях, man должен читаться (выводиться командой man);
7. Необязательно: в состав проекта может входить динамическая библиотека.

19 Материально-технические условия проведения курса

19.1 Аппаратное обеспечение

Для проведения очных лекций, преподавателю требуются:

- Персональный компьютер (ПК) с доступом в Интернет и программной средой согласно разделу 19.2 ;
- Проектор и проекционный экран для демонстраций.

Для выполнения домашних заданий, обучающемуся требуется ПК с доступом в Интернет и программной средой согласно разделу 19.2 .

Требования к аппаратным характеристикам ПК (не менее):

- процессор: частота 1,1 ГГц, 2 ядра;
- оперативная память: 4 Гб;
- свободное место на диске: 10 Гб;
- монитор: диагональ 13”, разрешение 1366*768.

19.2 Программная среда для демонстраций и выполнения домашних заданий

Для проведения демонстраций лектором и выполнения домашних заданий слушателями рекомендуется использовать среду ALT Linux. Рекомендуемые дистрибутивы:

- Альт Образование, версия не ниже 10.1;
- Альт Рабочая станция, версия не ниже 10.1.

Возможно также использование иных дистрибутивов Linux с версиями, актуальными на период чтения курса. При этом потребуется дополнительная работа по установке инструментов, специфических для ОС «Альт», таких как hasher.

В установленной конфигурации Linux должны быть доступны:

- библиотеки: glibc, libncurses, libncursesw, libpcre;
- утилиты: ar, autotools, bash, cc, check, cmake, cut, dialog, diffutils, doxygen, file, find, gdb, gdbfrontend, gdbgui, gettext, git, graphviz, grep, hasher, hexdump, intltool, libtool, make, meson, mktemp, ninja, nm, objdump, od, pkg-config, readelf, sed, shuf, strace, termutils, tr, troff, xargs, xdd, xterm, zsh.

Доступность библиотек и утилит обеспечивается за счет установки соответствующих пакетов. Для каждой из библиотек должен быть установлен как базовый, так и -devel пакет.

В среде для демонстраций должен быть обеспечен доступ к ресурсам сети Интернет, в т.ч. для показа страниц на сайте курса. В среде для выполнения практикума

доступ к сети Интернет необходим для размещения (git push) выполненных заданий в публичном репозитории.